



Rapport de Projet de Fin d'Etudes

présenté en vue de l'obtention du diplôme de

LICENCE NATIONALE EN TECHNOLOGIES DE L'INFORMATIQUE

PARCOURS : Développement des Systèmes d'Informations (DSI)

**Développement d'un assistant conversationnel basé
sur une architecture RAG pour le support des
consultants SAP BASIS**

Entreprise : Avaxia

Réalisé par :

Yassine Ben hamad et Ghada Azizi

Encadré par :

Mme. Msakni Imen: Encadrant ISET

M. Ben Salem Amenallah : Encadrant Entreprise

Code PFE : DSI-26-R487

Année universitaire : 2025/2026

Table des matières

Dédicace	7
Remerciements	8
Liste des figures	9
Liste des tableaux	11
Liste des abréviations	12
Introduction Générale	16
Objectifs du projet	17
Structure du rapport	17
1 Cadre général du projet	19
Introduction	19
1.1 Présentation de l'organisme d'accueil	19
1.1.1 AVAXIA GROUP	19
1.1.2 Philosophie et approche	20
1.1.3 Portefeuille de services	20
1.1.4 Structure organisationnelle	20
1.2 Analyse de l'existant	21
1.2.1 Étude de l'existant	21
1.2.2 Solutions existantes	23
1.2.3 Critique de l'existant	25
1.2.4 Critique des solutions SAP existantes	26
1.2.5 Solution proposée	27
1.3 Méthodologie de développement	28
1.3.1 Présentation des méthodologies candidates	29
1.3.2 Comparaison des méthodologies	32
1.3.3 Méthodologie retenue : CRISP-DM	32

Conclusion	33
2 Planification et spécification des besoins	34
Introduction	34
2.1 Vision et objectifs du système futur	34
2.2 Spécification des besoins	35
2.2.1 Identification des acteurs	35
2.2.2 Besoins fonctionnels	37
2.2.3 Besoins Non Fonctionnels	38
2.3 Gestion de projet avec la méthode CRISP-DM	38
2.4 Diagramme de Gantt	39
3 Compréhension métier	41
Introduction	41
3.1 Contexte métier SAP	41
3.1.1 Qu'est-ce qu'un ERP ?	41
3.1.2 Le consultant SAP BASIS	42
3.1.3 Les codes de transaction SAP (T-codes)	42
3.2 État de l'art	46
3.2.1 Les Fondements Technologiques de l'IA Générative	46
3.2.2 Machine Learning et Intelligence Artificielle	47
3.2.3 Deep Learning	47
3.2.4 Architecture Transformer	47
3.2.5 Traitement Automatique du Langage Naturel (NLP)	47
3.2.6 Les Modèles de Langage de Grande Taille : Capacités et Limites	48
3.2.7 Architecture de RAG (Retrieval-Augmented Generation)	48
3.2.8 L'architecture de CAG (Cache-Augmented Generation) : une amélioration du RAG	49
3.2.9 L'architecture de Fine-Tuning : personnalisation des modèles IA	50
3.2.10 Comparaison des approches Fine-Tuning, RAG et CAG	50
3.3 Le choix du RAG pour ce projet	51
Conclusion	52
4 Compréhension des données	53
4.1 Collecte des Données	53
4.1.1 Données de Surveillance SAP : Extraction via PyRFC	53
4.1.2 Les données documentaires pour l'architecture de RAG	54
4.1.3 Ingestion des données	56
4.2 Description des données à exploiter	57
4.2.1 Nature des données résultantes de l'extraction	57

4.2.2	Exploration des données résultantes de l'extraction	58
4.3	Qualité des données	61
4.3.1	Volumétrie du corpus	61
4.3.2	Valeurs manquantes	61
4.3.3	Doublons et redondances	61
4.3.4	Bruit textuel	62
5	Préparation des données	63
	Introduction	63
5.1	Vue d'ensemble du pipeline de préparation	63
5.2	Nettoyage, normalisation et déduplication	65
5.2.1	Nettoyage des données du portail SAP Help	65
5.2.2	Nettoyage des données des livres techniques PDF	66
5.2.3	Normalisation commune des deux sources	66
5.2.4	Suppression des doublons et du bruit	68
5.3	Découpage des documents (Chunking)	70
5.3.1	Qu'est-ce que le découpage	70
5.3.2	Pourquoi découper les documents ?	70
5.3.3	Stratégie hiérarchique parent-enfant	71
5.3.4	Algorithme de découpage et chevauchement	72
5.3.5	Résultats du découpage	73
5.4	Transformation des données en embeddings vectoriels	73
5.4.1	Choix du modèle d'embedding	73
5.4.2	Processus de génération des vecteurs	74
5.5	Structuration des métadonnées	74
5.5.1	Rôle des métadonnées dans un système RAG	74
5.5.2	Schéma des métadonnées	75
5.5.3	Stockage et indexation dans PostgreSQL	76
	Conclusion	77
6	Modélisation	78
6.1	Architecture globale du système RAG	78
6.1.1	Vue d'ensemble	78
6.1.2	Flux de données	79
6.2	Pipeline RAG : Description détaillée	80
6.2.1	Vue séquentielle du pipeline	80
6.2.2	Normalisation de la requête	80
6.2.3	Compréhension sémantique de la requête (NLP)	80
6.2.4	Vectorisation de la requête	82
6.2.5	Recherche hybride parallèle	82

6.2.6	Fusion par Reciprocal Rank Fusion (RRF)	83
6.2.7	Injection CAG (Context-Augmented Generation)	83
6.3	Modélisation des données	84
6.3.1	Structure de la table <code>sap_chunks</code>	84
6.3.2	Hiérarchie des chunks et stratégie Small-to-Big	85
6.4	Techniques d’Embeddings	86
6.4.1	Modèle retenu : BAAI/bge-large-en-v1.5	86
6.4.2	Méthode de vectorisation	86
6.5	Base Vectorielle	87
6.5.1	PostgreSQL avec l’extension pgvector	87
6.5.2	Indexation vectorielle HNSW	87
6.6	Mécanisme de Retrieval	88
6.6.1	Recherche sémantique vectorielle	88
6.7	Mécanisme de Reranking par Cross-Encoder	88
6.7.1	Modèle cross-encoder ms-marco-MiniLM-L-6-v2	89
6.7.2	Seuillage et déduplication	89
6.8	Large Language Model	90
6.8.1	Modèle et infrastructure d’exécution locale	90
6.8.2	Mode streaming et intégration API	90
6.9	Prompt Engineering	90
6.9.1	Structure du prompt système	90
6.9.2	Construction dynamique du prompt	91
6.9.3	Pré-synthèse des chunks (<code>synthesis_text</code>)	91
6.9.4	Injection de l’historique conversationnel	92
6.9.5	Contrôle anti-hallucination post-génération	93
6.10	Workflow global d’une requête utilisateur	93
6.11	Algorithme général du pipeline	95
6.11.1	Limites identifiées	96
7	Évaluation	97
	Introduction	97
7.1	Protocole d’évaluation	97
7.1.1	Dataset de test	97
7.1.2	Méthode d’auto-labellisation	99
7.2	Évaluation du Retrieval	100
7.2.1	Métriques implémentées	100
7.2.2	Configuration d’évaluation	101
7.2.3	Résultats globaux	102
7.2.4	Résultats par type d’intention	103

7.2.5	Couverture des signaux	104
7.2.6	Limites de validité du protocole d'évaluation	105
7.3	Niveau 2 : Évaluation LLM-as-Judge avec RAGAS	105
7.3.1	Présentation et métriques RAGAS	105
7.3.2	Configuration de l'évaluation RAGAS	106
7.3.3	Résultats RAGAS	107
7.3.4	Analyse des résultats	109
7.4	Comparaison Hybride vs Baseline	109
7.5	Analyse visuelle des performances	110
7.6	Justification de la double approche d'évaluation	111
7.7	Détection d'hallucinations	112
7.8	Synthèse et discussion	112
7.8.1	Points forts du système	112
7.8.2	Limites identifiées	112
	Conclusion	113
8	Déploiement	114
8.1	Architecture globale de déploiement	114
8.1.1	Vue d'ensemble	114
8.1.2	Technologies utilisées	115
8.2	Déploiement du système RAG	116
8.2.1	Configuration de l'environnement	116
8.2.2	Infrastructure de la base de données vectorielle	116
8.2.3	Schéma de la base de données PostgreSQL	117
8.3	Diagramme de cas d'utilisation général	117
8.3.1	Démarrage de l'API FastAPI	120
8.3.2	Endpoints REST exposés	121
8.3.3	Paramètres clés du pipeline	121
8.4	Intégration du chatbot dans l'application web	122
8.4.1	Vue d'ensemble de l'intégration	122
8.4.2	Définition du service OData (CAP CDS)	123
8.4.3	Composants React du chatbot	123
8.4.4	Chatbot et évaluation intégrée en temps réel	124
8.4.5	Intelligence de navigation — T-codes vers pages monitoring	126
8.5	Interface utilisateur — Présentation	126
8.5.1	Page chatbot dédiée	126
8.5.2	ChatPanel : Tiroir latéral universel	127
8.6	Sélection du modèle LLM et benchmarking	128
8.6.1	Protocole d'évaluation	128

8.6.2	Résultats	129
8.6.3	Analyse des résultats	129
8.7	Limites et perspectives	130
8.7.1	Limites du déploiement actuel	130
8.7.2	Perspectives d'amélioration	130
Conclusion générale et perspectives		132
Bibliographie		135

Dédicace

Nous dédions ce modeste travail

À nos très chers parents,

En témoignage de notre profonde gratitude, de notre respect et de notre amour.

À nos pères, qui nous ont appris le sens du travail, de la persévérance et de la responsabilité. Leur soutien indéfectible, leurs conseils précieux et leurs sacrifices ont été une source permanente de motivation tout au long de notre parcours.

À nos mères, symboles de tendresse, de patience et de dévouement. Leur amour inconditionnel, leurs encouragements constants et leurs prières nous ont accompagnés à chaque étape de notre vie.

Aucune parole ne saurait exprimer pleinement notre reconnaissance pour tout ce qu'ils ont accompli pour nous. Que Dieu leur accorde santé, bonheur et longue vie.

À nos chers frères et sœurs,

Pour leur affection, leur soutien moral et leur présence réconfortante. Nous leur souhaitons un avenir rempli de réussite, de bonheur et d'épanouissement.

Enfin,

Nous adressons nos remerciements à toutes les personnes qui nous ont aidés, soutenus et encouragés, de près ou de loin, dans la réalisation de ce projet.

Azizi Ghada & Ben Hamad Yassine

Remerciements

Dans le cadre de la réalisation de ce projet de fin d'études sur la conception et le développement de la plateforme AutoMonitoring de surveillance intelligente des systèmes SAP, nous tenons à exprimer notre profonde reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, à son aboutissement.

Nous tenons à exprimer nos sincères remerciements à Madame Imen Msakni pour son encadrement de qualité, sa rigueur et sa bienveillance tout au long de ce projet de fin d'études. Nous lui dédions cette modeste réalisation en témoignage de notre gratitude pour ses conseils précieux, son accompagnement attentif et son soutien constant, qui ont grandement contribué à la réussite de ce travail.

Nous tenons également à adresser nos sincères remerciements à Monsieur Amenallah Ben Salem, notre encadrant professionnel au sein de la société Avaxia Group, pour sa disponibilité, son expertise technique et son accompagnement tout au long de cette expérience enrichissante.

Nous remercions également l'ensemble de l'équipe d'Avaxia Group pour leur accueil chaleureux, leur collaboration, et pour nous avoir permis d'évoluer dans un environnement professionnel exigeant, au cœur des enjeux technologiques de la gestion des systèmes SAP.

Enfin, nous exprimons toute notre gratitude envers nos collègues, amis et proches, dont le soutien et les encouragements nous ont accompagnés tout au long de ce parcours.

Azizi Ghada & Ben Hamad Yassine

Table des figures

1.1	Logo Avaxia Group	19
1.2	Processus de surveillance manuelle des systèmes SAP	23
1.3	SAP Leonardo ML	24
1.4	SAP Joule au sein de l'écosystème SAP	24
1.5	Cycle de vie du framework CRISP-DM	29
1.6	Étapes du processus KDD	30
1.7	Cycle de vie de la méthodologie Microsoft TDSP	31
1.8	Le processus utilisé le plus fréquemment dans les projets de data science [3]	33
2.1	Diagramme de Gantt	39
3.1	Logo SAP	42
3.2	SM37 — Vue d'ensemble des jobs batch	44
3.3	DB02 — Vue d'ensemble de l'espace base de données	45
3.4	ST22 — Liste des erreurs d'exécution ABAP	46
3.5	Les LLMs à l'intersection du <i>Deep Learning</i> et du <i>NLP</i>	48
3.6	Architecture du système RAG	49
4.1	Structure des données extraites depuis la transaction ST22	54
4.2	Structure des données extraites depuis la transaction SM37	54
4.3	les sites de SAP Help	56
4.4	Pipeline d'ingestion et de traitement des données SAP	57
4.5	Données des livres PDF SAP	59
4.6	Données de SAP Help	60
4.7	Données de surveillance SAP	60
5.1	Pipeline complet de préparation des données	64
5.2	Algorithme de déduplication en deux passes	69
5.3	Pipeline de recherche hybride exploitant les index GIN et HNSW lors d'une requête RAG	77
6.1	Pipeline RAG SAP V2	95

7.1	Analyse des performances du système RAG SAP	111
8.1	Architecture de déploiement AutoMoni	115
8.2	Diagramme de cas d'utilisation	118
8.3	Diagramme de séquence : interaction complète	119
8.4	Diagramme de classes (PostgreSQL)	120
8.5	Interface chatbot avec historique	127
8.6	ChatPanel en tiroir latéral	128
8.7	Benchmark RAG — LLaMA 3.2 :3b vs Qwen 2.5 :7b	129

Liste des tableaux

1.1	Exemples de T-codes utilisés pour la surveillance SAP	22
1.2	Comparatif entre la supervision manuelle, SAP Leonardo ML et SAP Joule	26
1.3	Comparaison des méthodologies CRISP-DM, TDSP, KDD et Scrum . .	32
2.1	Acteurs du système AutoMoni et leurs responsabilités	36
3.1	Comparaison entre Fine-Tuning, RAG et RAG+CAG	51
4.1	Volumétrie du corpus par source	61
5.1	Les opérations de normalisation appliquées à chaque document	67
5.2	Filtres de qualité appliqués après fusion des sources	70
5.3	Niveaux de découpage par ordre de priorité	72
5.4	Statistiques de découpage du corpus SAP	73
5.5	Comparaison des modèles d’embedding évalués	74
5.6	Champs d’identification et de source	75
5.7	Champs de structure hiérarchique et d’entités SAP	75
5.8	Champs de liaison parent-enfant	76
6.1	Colonnes de la table <code>sap_chunks</code>	85
7.1	Distribution du dataset de test par type d’intention et module SAP . .	99
7.2	Résultats du retrieval : Système hybride vs baseline vectorielle pure . .	103
7.3	Performance du retrieval décomposée par type d’intention	104
7.4	Description des métriques RAGAS utilisées	106
7.5	Résultats RAGAS — Évaluation LLM-as-Judge (11ama3.2:3b) sur 3 requêtes SAP	107
7.6	Résultats RAGAS détaillés par requête	108
7.7	Gain apporté par le système hybride par rapport à la baseline	109
8.1	Pile technologique du système déployé	116
8.2	Endpoints de l’API RAG (api.py)	121
8.3	Paramètres de configuration du pipeline RAG (rag_pipeline.py)	122

Liste des abréviations

Abréviation	Signification
ABAP	Advanced Business Application Programming — langage de programmation propriétaire de SAP
API	Application Programming Interface — Interface de Programmation d'Application
AR	Augmented Reality — Réalité Augmentée
BAAI	Beijing Academy of Artificial Intelligence
BASIS	Business Application Software Integrated Solution — couche technique d'administration SAP
BGE	BAAI General Embedding — famille de modèles d'embedding de BAAI
BM25	Best Matching 25 — algorithme de recherche plein texte (ranking lexical)
BW	Business Warehouse — module SAP dédié à l'entreposage et à l'analyse de données
CAG	Cache-Augmented Generation — Génération Augmentée par Cache
CAP	Cloud Application Programming — modèle de programmation d'applications SAP
CRISP-DM	Cross-Industry Standard Process for Data Mining — processus standard inter-industries pour la fouille de données
ECC	Enterprise Central Component — composant central de l'ERP SAP

(suite...)

Abréviation	Signification
ERP	Enterprise Resource Planning — Planification des Ressources d'Entreprise
ETL	Extract, Transform, Load — processus d'Extraction, Transformation et Chargement des données
EWM	Extended Warehouse Management — gestion avancée d'entrepôt (module SAP)
GPT	Generative Pre-trained Transformer — architecture de modèle de langage génératif
GUI	Graphical User Interface — Interface Graphique Utilisateur
HANA	High-performance ANALytic Appliance — base de données en mémoire de SAP
HNSW	Hierarchical Navigable Small World — structure d'index pour la recherche approximative de voisins
IA	Intelligence Artificielle
IT	Information Technology — Technologies de l'Information
JSON	JavaScript Object Notation — format léger d'échange de données
KDD	Knowledge Discovery in Databases — Découverte de Connaissances dans les Bases de Données
LLM	Large Language Model — Grand Modèle de Langage
LSTM	Long Short-Term Memory — type de réseau de neurones récurrents à mémoire longue
ML	Machine Learning — Apprentissage Automatique
MRR	Mean Reciprocal Rank — Rang Réciproque Moyen (métrique d'évaluation du retrieval)
nDCG	normalized Discounted Cumulative Gain — Gain Cumulatif Actualisé Normalisé

(suite...)

Abréviation	Signification
NLP	Natural Language Processing — Traitement Automatique du Langage Naturel
PDF	Portable Document Format — format de fichier document portable
PyRFC	Python Remote Function Call — connecteur Python pour les appels de fonctions SAP distantes
RAG	Retrieval-Augmented Generation — Génération Augmentée par Récupération
RAGAS	RAG Assessment Score — framework d'évaluation automatique des systèmes RAG
REST	Representational State Transfer — style d'architecture pour les API web
RFC	Remote Function Call — Appel de Fonction Distante (mécanisme de communication SAP)
RNN	Recurrent Neural Network — Réseau de Neurones Récurrent
RRF	Reciprocal Rank Fusion — algorithme de fusion de listes de résultats par rang réciproque
SAP	Systems, Applications and Products in Data Processing
SGI	Système de Gestion Intégré
SOLMAN	Solution Manager — outil central de gestion et de surveillance SAP
SQL	Structured Query Language — langage de requête structuré pour les bases de données relationnelles
T-code	Transaction Code — code de transaction SAP permettant d'accéder rapidement à une fonction
TDSP	Team Data Science Process — processus Microsoft de science des données en équipe
UI	User Interface — Interface Utilisateur

(suite...)

Abréviation	Signification
UI5	SAP User Interface 5 — framework de développement d’interfaces web SAP
UML	Unified Modeling Language — Langage de Modélisation Unifié
URL	Uniform Resource Locator — adresse d’une ressource sur le web
VR	Virtual Reality — Réalité Virtuelle

Introduction Générale

Dans un environnement où la transformation numérique est devenue cruciale pour les organisations, les systèmes SAP se sont affirmés comme des outils stratégiques pour la gestion des processus commerciaux.

En tant que logiciels de gestion intégrés (SGI), ils pilotent des activités essentielles telles que la gestion des finances, la logistique, les ressources humaines, la production et la chaîne d'approvisionnement. Leur accessibilité et leur bon fonctionnement sont donc vitaux pour la continuité des opérations de nombreuses entreprises dans le monde entier.

Néanmoins, l'administration et la surveillance de ces systèmes restent des tâches difficiles. Les administrateurs BASIS et les consultants SAP doivent constamment surveiller un grand nombre d'indicateurs techniques concernant l'état du système, tout en étant en mesure d'identifier et de résoudre rapidement les problèmes.

Cette obligation est accentuée par la richesse de la documentation technique SAP. Celle-ci se compose de plusieurs centaines de manuels de formation officiels, d'une documentation en ligne particulièrement vaste ainsi que d'un ensemble de notes techniques mises à jour régulièrement. Lorsqu'un incident se produit, les équipes doivent fréquemment rechercher, analyser et interpréter rapidement les informations pertinentes pour déterminer la procédure de résolution appropriée. Dans ce cadre, plusieurs difficultés majeures peuvent être mises en évidence :

- **Centralisation des informations de supervision et de la documentation :** l'analyse d'un incident requiert généralement la consultation simultanée des données de monitoring et des ressources documentaires correspondantes. Étant souvent éparpillées dans divers outils, leur utilisation devient plus compliquée et prend plus de temps.
- **Gestion d'un volume conséquent de connaissances :** la richesse du corpus documentaire SAP complique la recherche d'informations pertinentes, surtout lorsqu'une réponse rapide est nécessaire dans un cadre opérationnel.
- **Mise en valeur de l'expertise métier :** une grande partie des connaissances accumulées par les consultants expérimentés reste difficile à accéder pour les nouveaux employés, ce qui entrave la diffusion et la capitalisation du savoir au sein des équipes.

- **Diminution du temps de réponse face aux incidents** : toute baisse de performances ou indisponibilité d'un système SAP en production peut entraîner des impacts significatifs sur les activités de l'entreprise. Par conséquent, il est crucial de disposer de mécanismes permettant d'identifier rapidement les anomalies et d'assister efficacement les équipes dans leur résolution.

L'objectif principal de ce projet de fin d'études, réalisé au sein d'AVAXIA GROUP, est de développer un assistant intelligent destiné à la supervision des systèmes SAP. Cet assistant a pour vocation d'accompagner les administrateurs et les consultants SAP dans leurs activités quotidiennes en leur fournissant une assistance rapide et pertinente basée sur les connaissances métier et les informations de supervision disponibles.

Ce projet vise également à intégrer cet assistant au sein de la plateforme de supervision **AutoMoni**, déjà utilisée par l'entreprise. Cette intégration permettra d'enrichir les fonctionnalités existantes de la plateforme en offrant aux utilisateurs un accès centralisé aux outils de supervision ainsi qu'à une assistance documentaire intelligente.

Afin d'atteindre cet objectif, les travaux réalisés portent sur la constitution d'une base de connaissances SAP, le développement d'un mécanisme intelligent de recherche et de génération de réponses, ainsi que son intégration dans l'environnement AutoMoni.

Ce rapport est organisé en huit chapitres structurés selon les phases de la méthodologie CRISP-DM, encadrés par une introduction et une conclusion générales.

Chapitre 1 : Cadre général du projet

Présente l'organisme d'accueil, l'analyse de l'existant, la problématique et la méthodologie adoptée.

Chapitre 2 : Planification et spécification des besoins

Définit les besoins fonctionnels et non fonctionnels, les acteurs et les cas d'utilisation du système.

Chapitre 3 : Compréhension métier

Aligne les objectifs techniques sur les enjeux métier des équipes SAP BASIS et fixe les critères de succès.

Chapitre 4 : Compréhension des données

Décrit les sources de données, leur structure et leur qualité en vue de la modélisation.

Chapitre 5 : Préparation des données

Couvre le nettoyage et la transformation des documents SAP en embeddings pour le moteur de recherche.

Chapitre 6 : Modélisation

Détaille la mise en place du pipeline RAG hybride et l'architecture du système intelligent.

Chapitre 7 : Évaluation

Mesure la qualité du système en évaluant la pertinence de la recherche et la fidélité des réponses.

Chapitre 8 : Déploiement

Présente l'intégration de la solution dans la plateforme AutoMoni et son exposition aux utilisateurs.

Le rapport se clôture par une **conclusion générale** dressant le bilan du projet et ouvrant des perspectives d'évolution futures.

Chapitre 1

Cadre général du projet

Introduction

Ce chapitre a pour objectif de présenter le cadre général dans lequel s'inscrit ce projet de fin d'études. Nous commençons par une présentation de l'organisme d'accueil, puis nous effectuons une étude de l'existant afin d'identifier les lacunes des solutions actuelles de surveillance des systèmes SAP. Nous détaillons ensuite la problématique retenue, avant de présenter la solution proposée et la méthodologie de développement adoptée.

1.1 Présentation de l'organisme d'accueil

1.1.1 AVAXIA GROUP

AVAXIA GROUP est une société de conseil en technologies de l'information à dimension internationale, fondée en 2006. Elle dispose d'une présence stratégique dans plusieurs régions, avec son siège social à Dubaï et des centres opérationnels en Tunisie, au Japon et au Canada. L'organisation est spécialisée dans les solutions technologiques d'entreprise, avec un accent particulier sur l'implémentation des systèmes ERP (*Enterprise Resource Planning*), l'intégration de systèmes et les services gérés pour les applications métiers critiques. Son expertise technique s'étend à l'écosystème SAP ainsi qu'à des plateformes complémentaires telles que Salesforce.

La figure 1.1 présente le logo officiel d'Avaxia Group.



FIGURE 1.1 – Logo Avaxia Group

1.1.2 Philosophie et approche

Avaxia Group adopte une méthodologie résolument centrée sur le client dans son modèle de prestation de services, en privilégiant des partenariats collaboratifs plutôt que des relations purement transactionnelles.

1.1.3 Portefeuille de services

Avaxia met à disposition un portefeuille complet de services numériques et de conseil, avec une forte orientation vers les solutions d'entreprise, le support d'infrastructure et les technologies émergentes. Le groupe est structuré autour de deux départements spécialisés, chacun offrant des domaines de services distincts :

- **Consultance fonctionnelle** : Conception de solutions métier pour le déploiement de nouveaux modules SAP fonctionnels, analyse et optimisation des processus de bout en bout, ainsi que la gestion du déploiement fonctionnel et de la planification des tests.
- **Big Data** : Services incluant la gestion, la transformation, l'intégration, le stockage et l'analyse des données. Les compétences couvrent le développement de pipelines de données, les processus ETL et la gouvernance des données.
- **VR / AR** : Développement d'environnements VR interactifs permettant aux utilisateurs de naviguer, d'interagir avec des objets et de collaborer en temps réel. Les applications comprennent les systèmes de surveillance, les simulations, les programmes de formation et les visites virtuelles.

1.1.4 Structure organisationnelle

L'architecture organisationnelle d'Avaxia Group est conçue pour faciliter une expertise spécialisée tout en maintenant une collaboration transversale. La structure du groupe s'articule autour de trois niveaux :

- **Organisation divisionnelle** : Assure les responsabilités de chaque département en fonction de son rôle, en gérant le développement commercial et les décisions stratégiques.
- **Organisation opérationnelle** : Se concentre sur les défis techniques et les expertises qui pilotent les projets et services (planification, exécution, livraison et support). Elle permet également de suivre, évaluer et optimiser les opérations de projets.
- **Structure corporative** : Supporte le front office en représentant les opérations internes (Finance, RH, Formation, Marketing, Qualité de service, Informatique), afin d'assurer la continuité de l'activité dans son ensemble.

Ce projet se déroule au sein de l'équipe Quadient, rattachée au département *SAP Infrastructure and BASIS* du bureau tunisien d'Avaxia Group. Les responsabilités de cette équipe englobent la surveillance et le support des systèmes SAP critiques auprès de plusieurs clients, ce qui en fait un environnement idéal pour développer et valider un assistant intelligent de surveillance SAP basé sur la technologie RAG.

1.2 Analyse de l'existant

1.2.1 Étude de l'existant

Dans le cadre du développement d'un système intelligent de surveillance au sein d'Avaxia Group, une étude de l'existant a été réalisée pour comprendre le processus actuel de monitoring de l'équipe SAP BASIS.

Le processus de surveillance commence généralement par la consultation des alertes ou incidents signalés durant le shift précédent. Cela assure la continuité du suivi. Ensuite, des vérifications générales liées à l'infrastructure et à l'état des environnements sont faites rapidement avant de commencer les opérations de monitoring principales.

L'activité principale du consultant SAP BASIS consiste à surveiller les environnements SAP à travers plusieurs transactions SAP (T-codes). Cette surveillance permet de contrôler l'état des systèmes, d'identifier les anomalies éventuelles et d'assurer le bon fonctionnement des environnements de production.

Pour le client Quadient, l'équipe BASIS supervise environ dix systèmes SAP sur différents serveurs comme P01, PWD et PW1...

Chaque système doit être vérifié individuellement, ce qui rend le processus de surveillance long et répétitif. Pour chaque environnement, le consultant exécute plusieurs T-codes pour analyser les informations liées au fonctionnement du système. Après cette phase d'analyse, il interprète les résultats pour déterminer si une intervention est nécessaire.

Selon les anomalies détectées, il peut relancer certaines opérations échouées, suivre les incidents et collaborer avec les équipes concernées.

Par exemple, lorsque des traitements lourds ou des jobs affectant les performances sont identifiés dans le système SAP de production, le consultant BASIS discute avec les développeurs ABAP pour apporter les corrections nécessaires.

De même, certaines anomalies peuvent nécessiter l'intervention des consultants fonctionnels, notamment dans les domaines financiers, lorsque des changements liés aux règles de l'entreprise ou aux exigences du client doivent être pris en compte. La durée de surveillance varie selon l'état du système et le nombre d'alertes détectées. En moyenne, l'analyse complète d'un système prend jusqu'à 30 minutes, davantage pour les systèmes de production critiques. Étant donné que le client a environ dix systèmes,

le monitoring global nécessite plusieurs heures de travail par shift.

Cependant, cette approche a ses limites. La multiplicité des systèmes et la répétition des tâches rendent le processus lourd et chronophage. Le consultant doit constamment naviguer entre plusieurs systèmes et transactions, ce qui augmente le risque d'oubli, de confusion ou de perte de temps dans le traitement des anomalies.

Description de la surveillance des systèmes SAP

La surveillance quotidienne des systèmes SAP s'appuie sur l'utilisation de nombreux codes de transaction (T-codes), chacun offrant une visibilité sur un aspect spécifique du système. À titre illustratif, les T-codes suivants figurent parmi les plus couramment utilisés :

T-code	Fonction	Description
SM37	Jobs en arrière-plan	Suivi et analyse des jobs batch (terminé, en cours, échoué).
ST22	Dumps ABAP	Analyse des erreurs critiques du système (ABAP dumps).
SP01	Spools	Consultation et gestion des demandes d'impression.
DB02	Base de données	Informations sur l'espace et l'état de la base de données.
SM66	Processus actifs	Supervision des processus en cours en temps réel.

TABLE 1.1 – Exemples de T-codes utilisés pour la surveillance SAP

Il est important de noter que ces exemples ne représentent qu'une infime partie des milliers de T-codes disponibles dans l'environnement SAP. Dans ce contexte, il est difficile pour un consultant SAP de maîtriser l'ensemble de ces transactions.

Ces transactions sont utilisées par les consultants BASIS¹ pour superviser les systèmes. Certaines analyses impliquent également ABAP².

-
1. SAP BASIS : couche technique assurant l'administration et la maintenance des systèmes SAP.
 2. ABAP : langage de programmation SAP utilisé pour développer et personnaliser les applications.

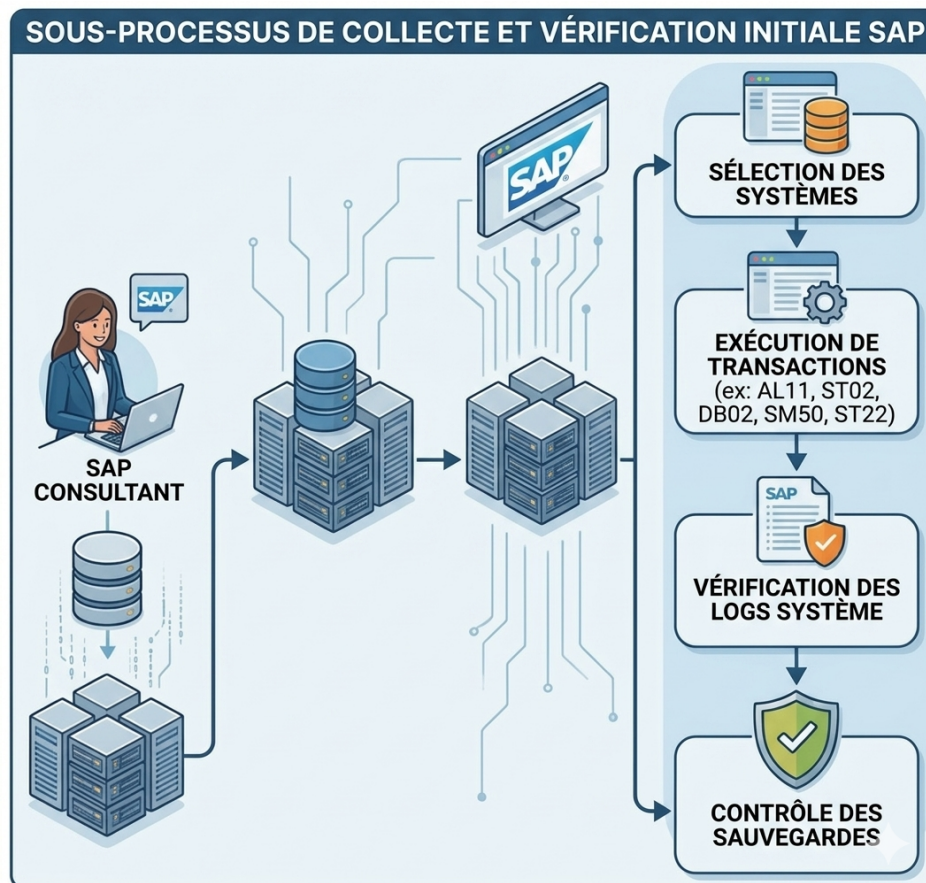


FIGURE 1.2 – Processus de surveillance manuelle des systèmes SAP

1.2.2 Solutions existantes

SAP, en plus des outils de supervision classiques tels que les tableaux de bord et les alertes techniques, propose désormais des solutions qui utilisent le Machine Learning et l'intelligence artificielle. Ces solutions permettent de surveiller les systèmes SAP en temps réel et d'analyser les données passées pour détecter des anomalies, prévoir des incidents et améliorer les décisions. Parmi ces solutions, on peut citer SAP Leonardo ML et SAP Joule. Elles offrent des fonctionnalités avancées pour une meilleure gestion des systèmes SAP.

SAP Leonardo ML

SAP Leonardo ML est une solution de Machine Learning développée par SAP qui permet d'utiliser les données provenant des systèmes SAP. Elle détecte les comportements inhabituels et identifie des modèles récurrents. Puisqu'elle fait partie intégrante des systèmes ERP SAP, SAP Leonardo ML peut accéder directement aux données issues de divers départements de l'entreprise, comme la finance, les ressources humaines ou la logistique.

Cela signifie que les données peuvent être exploitées de manière continue et centralisée.

De plus, les analyses sont plus cohérentes. SAP Leonardo ML propose également des fonctionnalités qui permettent de prédire certaines choses en analysant les données du passé. Cela aide à prévoir les incidents, à améliorer les processus de travail et à mieux surveiller les systèmes dans leur ensemble.

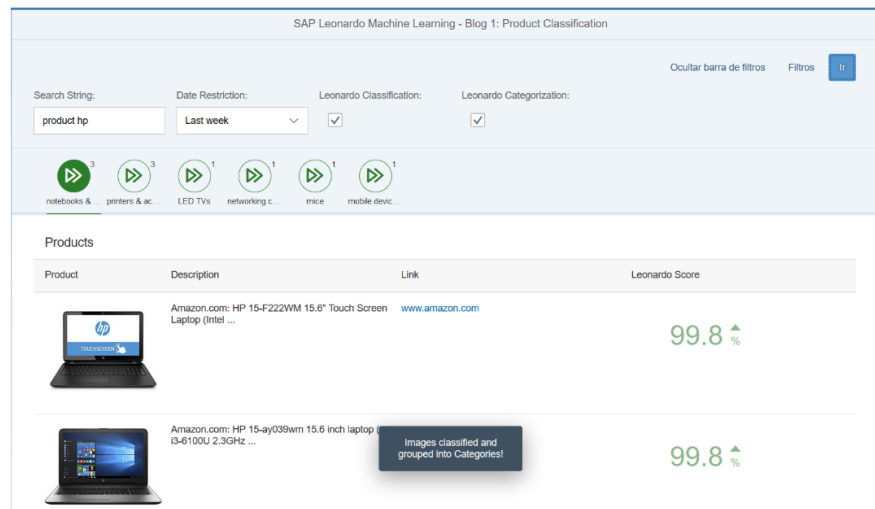


FIGURE 1.3 – SAP Leonardo ML

SAP JOULE

SAP Joule est un copilote génératif intégré à l'écosystème SAP. Il permet d'interroger les applications en langage naturel, sans connaître les commandes techniques, et s'appuie sur les données métier et techniques pour fournir des réponses, automatiser des tâches simples et faciliter l'analyse. Cette interaction conversationnelle réduit le temps de recherche et améliore la productivité des utilisateurs.

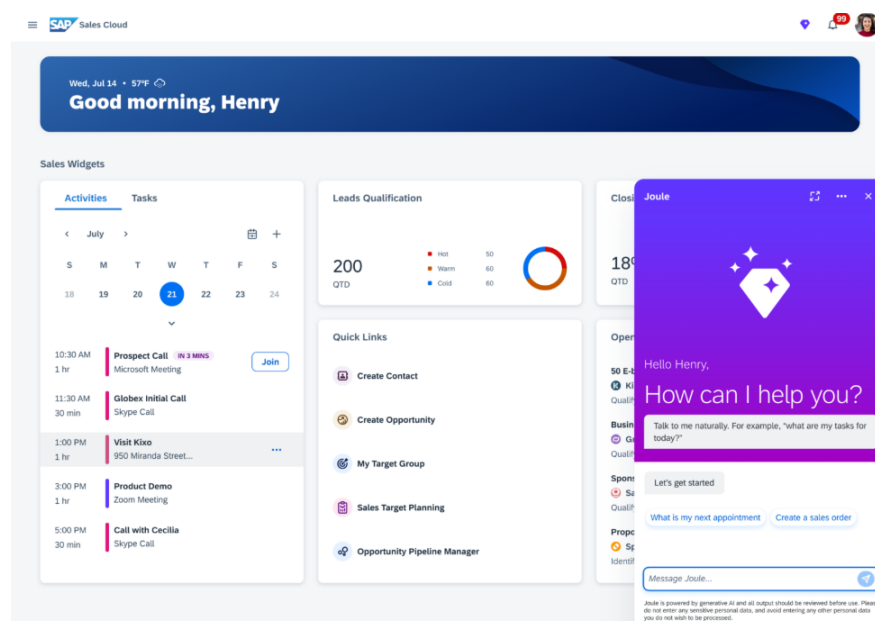


FIGURE 1.4 – SAP Joule au sein de l'écosystème SAP

1.2.3 Critique de l'existant

L'étude de l'existant a montré que le processus de surveillance des environnements SAP demande beaucoup d'efforts de la part des consultants SAP BASIS. La surveillance repose surtout sur des tâches manuelles répétées plusieurs fois par jour via différents T-codes et plusieurs systèmes SAP.

Chez Quadient, les consultants doivent superviser environ dix systèmes répartis sur différents serveurs comme P01, PWD et PW1. Cette diversité des environnements rend le monitoring long et difficile à gérer, surtout lorsque plusieurs incidents surviennent en même temps. Le consultant passe alors beaucoup de temps à naviguer entre les systèmes pour vérifier les informations et analyser les résultats.

Le principal problème de cette approche est le temps nécessaire pour réaliser les contrôles quotidiens. Comme les mêmes opérations se répètent chaque shift, le travail devient rapidement lourd et fatigant. Cette répétition peut entraîner des oublis, des erreurs de vérification ou des retards dans la détection d'anomalies importantes.

De plus, les informations étant dispersées entre plusieurs T-codes et plusieurs systèmes, il devient difficile d'avoir une vue globale et rapide de l'état des environnements SAP. Lorsqu'un problème est détecté, le consultant doit souvent effectuer plusieurs vérifications avant d'identifier l'origine de l'incident et de contacter les équipes concernées, comme les développeurs ABAP ou les consultants fonctionnels.

L'étude a également révélé une autre difficulté liée au niveau d'expérience des consultants SAP BASIS. Un consultant débutant peut avoir besoin de beaucoup plus de temps pour analyser les résultats des T-codes, comprendre certaines anomalies ou identifier les actions à prendre. L'interprétation des informations dépend souvent de l'expérience acquise sur les différents systèmes SAP et des connaissances métier du client. Cela peut ralentir le traitement des incidents et rendre la surveillance moins efficace.

Enfin, les consultants passent une grande partie de leur temps sur des tâches répétitives de contrôle au lieu de se concentrer sur l'analyse et l'anticipation des problèmes. Ces limites montrent donc la nécessité d'une solution intelligente et centralisée pour aider les consultants dans leurs activités de surveillance, réduire les tâches manuelles et faciliter la détection rapide des anomalies.

Comparaison des approches de supervision et d'assistance dans les environnements SAP

Dans le cadre de l'évolution des outils de supervision SAP vers des solutions plus intelligentes et automatisées, il est important d'analyser les approches existantes.

La comparaison entre la supervision manuelle, SAP Leonardo Machine Learning et SAP Joule permet de mettre en évidence les limites des méthodes traditionnelles

ainsi que les avantages offerts par les technologies basées sur l'intelligence artificielle. Le tableau suivant présente une comparaison selon plusieurs critères clés.

TABLE 1.2 – Comparatif entre la supervision manuelle, SAP Leonardo ML et SAP Joule

Critères	Supervision manuelle	SAP Leonardo ML	SAP Joule
Temps de réaction	La détection des problèmes prend du temps et dépend des consultants.	Les anomalies sont détectées rapidement grâce à l'analyse automatique des systèmes SAP.	Les utilisateurs obtiennent des réponses immédiates via une interface de communication.
Coût de gestion	Mobilise beaucoup de ressources humaines et techniques.	Réduit certains coûts grâce à l'automatisation et à l'analyse prédictive.	Diminue le temps de recherche d'informations et améliore l'efficacité des utilisateurs.
Qualité des analyses	Les analyses restent limitées et peuvent contenir des erreurs humaines.	Propose des analyses plus fiables avec détection des anomalies.	Fournit des réponses adaptées au contexte métier et technique.
Niveau d'automatisation	Les tâches sont principalement effectuées manuellement.	Automatise plusieurs opérations de surveillance et d'analyse.	Automatise l'assistance et simplifie les interactions avec les systèmes SAP.
Facilité d'utilisation	Demande une bonne maîtrise technique des outils SAP.	Nécessite des connaissances en configuration et en Machine Learning.	Offre une utilisation simple grâce au langage naturel.
Gestion des données	Les données sont souvent analysées de manière séparée et indépendante.	Centralise et exploite les données des différents modules SAP.	Analyse les données métier et techniques en temps réel.
Capacité d'anticipation	Il est difficile de prévoir les incidents avant leur apparition.	Permet d'anticiper certains problèmes à partir des données historiques.	Peut proposer des recommandations et des suggestions intelligentes.
Impact sur la productivité	Les interventions manuelles ralentissent parfois les opérations et augmentent le risque d'erreurs.	Améliore les processus et réduit le temps d'analyse.	Aide les utilisateurs à travailler plus rapidement et efficacement.

1.2.4 Critique des solutions SAP existantes

Limites des solutions analysées

L'analyse des solutions existantes, notamment SAP Leonardo ML et SAP Joule, met en évidence plusieurs limites liées à leur applicabilité dans un contexte spécifique

tel que celui d'Avaxia Group et du client Quadient.

Dans le cadre des méthodes traditionnelles utilisées par les consultants SAP BASIS, la surveillance repose principalement sur l'utilisation de multiples transactions techniques nécessitant une vérification manuelle régulière. Cette approche engendre un temps important consacré au contrôle manuel des différents indicateurs du système. Le consultant doit parcourir plusieurs transactions afin de vérifier l'état des jobs, des processus, des logs et des performances, ce qui peut rendre le suivi global chronophage. Par ailleurs, la consolidation des informations reste une tâche manuelle, ce qui peut influencer la rapidité d'analyse et la réactivité face aux incidents.

En ce qui concerne les solutions existantes proposées par SAP, telles que SAP Leonardo ML et SAP Joule, elles offrent des capacités avancées basées sur le Machine Learning et l'intelligence artificielle. Toutefois, ces solutions peuvent représenter un investissement important pour les organisations, notamment en termes de coûts d'implémentation et d'exploitation. De plus, elles sont conçues pour répondre à des besoins généraux et standards, ce qui peut limiter leur adaptation à des contextes spécifiques. Dans certains cas, les particularités des données propres aux systèmes et aux clients, comme celles d'un environnement spécifique tel que Quadient, peuvent nécessiter des ajustements ou des traitements complémentaires pour une exploitation optimale.

Ainsi, ces éléments mettent en évidence certaines limites des approches existantes, tant au niveau des méthodes traditionnelles que des solutions avancées, notamment en termes de temps, de coût et d'adaptation aux besoins spécifiques

1.2.5 Solution proposée

La solution proposée repose sur l'utilisation de l'intelligence artificielle pour analyser les événements et les messages générés par les environnements SAP. Grâce à cette approche, le système peut identifier les anomalies, fournir des explications contextualisées et suggérer des pistes de résolution adaptées aux incidents rencontrés.

Pour améliorer la pertinence des réponses proposées, la solution utilise une approche hybride basée sur le modèle RAG (Retrieval-Augmented Generation) et un mécanisme de cache (CAG). Cela permet au système de rechercher des informations utiles à partir de documents fiables et d'utiliser ces connaissances pour générer des réponses plus précises et mieux adaptées au contexte de l'incident détecté.

L'utilisation de la recherche sémantique aide également à reconnaître des incidents similaires même lorsqu'ils sont formulés différemment. Ainsi, le consultant peut rapidement accéder à des explications, des recommandations ou des solutions déjà connues sans avoir à effectuer de longues recherches manuelles.

Cette solution vise également à réduire le temps consacré aux tâches répétitives de surveillance, à améliorer la réactivité face aux incidents et à faciliter le travail

des consultants moins expérimentés en leur fournissant une assistance intelligente et contextualisée.

Enfin, la mise en place de cet assistant intelligent va améliorer le processus de surveillance des environnements SAP en offrant une surveillance plus centralisée, plus rapide et plus efficace. Cela aidera également les consultants SAP BASIS dans leur prise de décision quotidienne.

- **Collecte et traitement de la documentation** : La solution collecte automatiquement la documentation SAP depuis le portail SAP Help ainsi que les livres techniques, en préservant leur structure et leur contenu. Les données collectées sont ensuite nettoyées, organisées et enrichies avec des informations contextuelles pour faciliter leur exploitation ultérieure.
- **Base de connaissances centralisée** : L'ensemble de la documentation traitée est stocké dans une base de connaissances unifiée, permettant aux consultants d'interroger directement le système en langage naturel pour retrouver une information pertinente, sans avoir à naviguer manuellement entre plusieurs sources.
- **Recherche et génération de réponses** : Lorsqu'un consultant pose une question, le système identifie les documents les plus pertinents dans la base de connaissances et génère automatiquement une réponse argumentée et sourcée, adaptée au contexte de la requête.
- **Prévention des erreurs** : Des mécanismes de validation sont intégrés pour s'assurer que les réponses générées sont bien fondées sur la documentation disponible et non sur des informations incorrectes.
- **Interface et déploiement** : La solution est accessible via une interface web intuitive comprenant un chatbot de support, un tableau de bord de surveillance en temps réel et un panneau d'administration. Elle est également connectée à Microsoft Teams afin de notifier automatiquement les équipes lors de la survenue d'incidents critiques.

1.3 Méthodologie de développement

L'adoption d'une méthodologie bien structurée permet d'assurer une meilleure organisation du projet, d'améliorer la qualité des évaluations et de garantir la fiabilité des systèmes développés. Elle facilite aussi la communication avec le client. Elle offre une vision claire des objectifs à atteindre et permet d'anticiper les difficultés. Cela aide à faire les ajustements nécessaires au bon moment. Dans cette section, nous présenterons différentes méthodologies de travail, ainsi que leurs avantages et leurs limites. Nous expliquerons ensuite le choix de l'approche sélectionnée pour ce projet.

1.3.1 Présentation des méthodologies candidates

SCRUM

Scrum est une méthode agile pour gérer des projets. Elle repose sur le travail collaboratif et itératif. Le projet se divise en cycles courts, appelés sprints. Cela permet de livrer rapidement des fonctionnalités et de s'ajuster aux changements des besoins.

Limites de Scrum en Data Science

Malgré sa flexibilité, Scrum a des limites dans les projets de data science.

Il est difficile d'estimer la durée des tâches expérimentales. Les résultats liés aux analyses et aux modèles d'IA sont imprévisibles. De plus, les sprints courts ne conviennent pas toujours aux phases de recherche et d'entraînement des modèles. La préparation et le nettoyage des données prennent souvent plus de temps que prévu. Enfin, il est difficile de mesurer la progression du projet avec des fonctions classiques.

CRISP-DM

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) est une méthodologie éprouvée par l'industrie pour guider les projets de fouille de données. Son cycle de vie se compose de six phases interdépendantes (voir Figure 1.5)

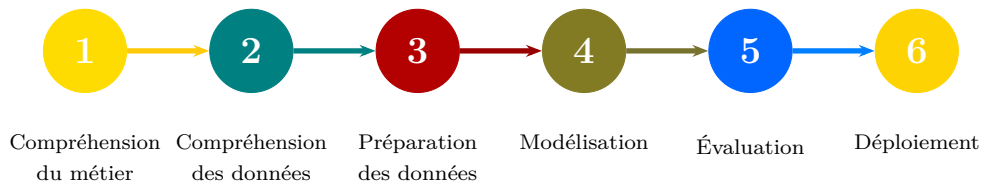


FIGURE 1.5 – Cycle de vie du framework CRISP-DM

1. **Compréhension métier (*Business Understanding*)** : Définir les objectifs du projet et traduire les objectifs métier en problèmes de fouille de données.
2. **Compréhension des données (*Data Understanding*)** : Collecter les données initiales, évaluer leur qualité et découvrir des premières perspectives.
3. **Préparation des données (*Data Preparation*)** : Nettoyer, transformer et mettre en forme les données pour les besoins de la modélisation.
4. **Modélisation (*Modeling*)** : Sélectionner et appliquer des techniques de modélisation avec des paramètres optimisés.
5. **Évaluation (*Evaluation*)** : Évaluer les résultats du modèle par rapport aux objectifs métier et aux critères de succès.
6. **Déploiement (*Deployment*)** : Mettre en production les résultats et établir une stratégie de surveillance continue.

Avantages de CRISP-DM

- **Orienté métier** : Démarre par la compréhension métier, en parfait alignement avec les besoins d'entreprise tels que l'interprétation des anomalies SAP et l'analyse d'impact.
- **Workflow itératif** : Encourage les retours continus et le raffinement du modèle, utile pour améliorer la précision et la pertinence des alertes.
- **Neutre technologiquement** : N'impose pas de contraintes d'outillage, bien adapté à une intégration flexible avec des stacks modernes.
- **Largement adopté** : Éprouvé par l'industrie et soutenu par une documentation et des outils abondants.
- **Accent sur l'évaluation** : Inclut des phases dédiées à l'évaluation de la performance technique et de l'adéquation métier, essentielles pour les applications de surveillance SAP.

Limites de CRISP-DM

- **Absence de MLOps intégré** : Ne traite pas nativement les pratiques de déploiement modernes (CI/CD, versionnage, surveillance continue des modèles).
- **Cadencement peu prescriptif** : CRISP-DM ne définit ni durée de phase ni vélocité cible, ce qui peut rendre le suivi de l'avancement difficile dans un contexte projet avec des échéances formelles.
- **Absence de guidance sur la composition de l'équipe** : Contrairement à Scrum ou TDSP, la méthodologie ne prescrit aucun rôle ni structure d'équipe, laissant cette organisation entièrement à la charge du projet.

KDD – Knowledge Discovery in Databases

Le processus KDD (*Knowledge Discovery in Databases*), introduit par Fayyad et al. en 1996, est l'une des premières méthodologies formelles pour extraire des connaissances actionnables de larges volumes de données. Il met l'accent sur le cycle de vie complet, de la préparation des données jusqu'à l'interprétation des connaissances.

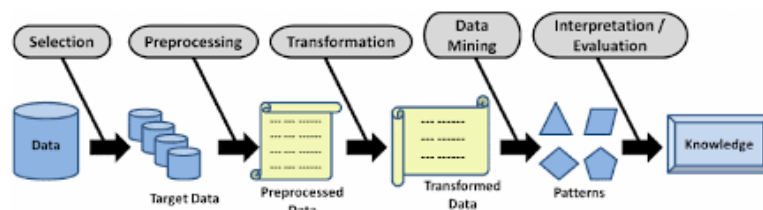


FIGURE 1.6 – Étapes du processus KDD

Avantages

- **Agile** : Insiste sur les livrables incrémentaux.
- **Familier** : Backlog produit, user stories, versionnage Git et sprint planning familiers aux équipes habituées aux pratiques logicielles courantes.
- **Natif data science** : Reconnaît les différences entre data science et ingénierie logicielle.
- **Flexible** : Peut être implémenté seul ou combiné avec d'autres approches.

Inconvénients

- **Sprints fixes** : De nombreux data scientists peinent avec les sprints de durée fixe imposés par TDSP.
- **Quelques incohérences** : La documentation Microsoft n'est pas toujours parfaitement cohérente.

1.3.2 Comparaison des méthodologies

TABLE 1.3 – Comparaison des méthodologies CRISP-DM, TDSP, KDD et Scrum

Critère	CRISP-DM	TDSP	KDD	Scrum
Objectif principal	Processus structuré et itératif pour la fouille de données orientée métier	Cycle de vie agile pour projets ML scalables en équipe	Extraction de patterns et connaissances depuis des données brutes	Livraison incrémentale de produits via des sprints courts et itératifs
Intégration métier	Forte : la compréhension métier est le point de départ du cycle	Forte : phase d'alignement métier intégrée dès le départ	Limitée : centrée sur les données et les patterns	Modérée : assurée par le Product Owner, sans phase dédiée
Gestion itérative	Itérations libres entre les phases, sans cadre temporel imposé	Sprints définis avec livrables et métriques de performance clairs	Processus globalement séquentiel, peu itératif	Sprints fixes (1-4 semaines) avec cérémonies Scrum structurées
Meilleur cas d'usage	Projets data science avec fort contexte métier et phases exploratoires	Flux ML collaboratifs nécessitant reproductibilité et traçabilité	Découverte de connaissances dans de larges volumes de données brutes	Développement logiciel agile avec livraisons fréquentes et besoins évolutifs

La figure 1.8 illustre la fréquence d'utilisation des différentes méthodologies dans les projets de data science.

1.3.3 Méthodologie retenue : CRISP-DM

CRISP-DM a été choisie comme méthodologie principale pour ce projet car elle met l'accent sur les besoins métier, reste indépendante des technologies utilisées et adopte une approche itérative. Cette méthodologie permet de mieux comprendre les exigences

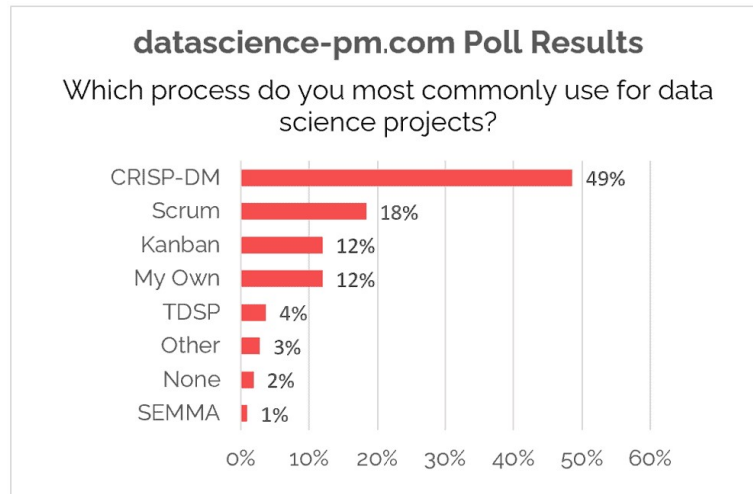


FIGURE 1.8 – Le processus utilisé le plus fréquemment dans les projets de data science [3]

métier liées à la surveillance des systèmes SAP tout en offrant une grande flexibilité dans le choix des outils et des technologies. De plus, son processus d'amélioration continue, ainsi que ses différentes phases d'analyse, de préparation des données, de modélisation et d'évaluation, correspondent parfaitement aux besoins d'un assistant intelligent de surveillance SAP. Dans ce contexte, les modèles et les analyses doivent être régulièrement ajustés et optimisés afin de garantir des résultats fiables capables de s'adapter aux évolutions du système.

Conclusion

Ce chapitre a présenté AVAXIA GROUP et le contexte opérationnel du projet. L'analyse de l'existant a révélé les limites d'une surveillance SAP manuelle et fragmentée, sans solution centralisée ni assistance contextuelle adaptée aux contraintes de l'équipe BASIS.

Ces constats motivent le développement d'un assistant intelligent. Le chapitre suivant en définit les exigences et la méthodologie de réalisation.

Chapitre 2

Planification et spécification des besoins

Introduction

Avant de commencer la phase de réalisation, il est crucial de bien définir les contours du système à mettre en place. Ce chapitre jette les bases du projet en identifiant les attentes des utilisateurs, en précisant les fonctionnalités requises et en établissant les contraintes à respecter. La méthodologie CRISP-DM oriente cette démarche structurée, depuis la compréhension du domaine jusqu'à la conception d'une solution appropriée pour le contexte de surveillance SAP d'Avaxia Group.

2.1 Vision et objectifs du système futur

Dans le cadre de l'optimisation et de la modernisation de l'exploitation des systèmes SAP, le projet vise à développer un assistant intelligent capable d'analyser et d'exploiter de manière efficace les données issues des différentes transactions et opérations. Les principaux objectifs du système futur sont les suivants :

- **Centraliser et organiser les données SAP** provenant des différents modules, afin de disposer d'une source unique et fiable d'informations.
- **Analyser et interpréter les données** grâce à des techniques d'intelligence artificielle et d'analyse avancée, transformant les informations brutes en indicateurs et rapports exploitables.
- **Signaler les anomalies et incidents** en permettant à l'assistant de mettre en évidence les comportements inhabituels ou les erreurs récurrentes identifiées dans les données de monitoring.
- **Intégrer la documentation SAP** pour fournir aux utilisateurs des recommandations, des explications et des solutions adaptées aux problèmes rencontrés.

- **Offrir une interface interactive** sous forme d’assistant conversationnel permettant aux utilisateurs de poser des questions, d’obtenir des analyses et de recevoir des conseils personnalisés.
- **Assurer une accessibilité via une application web** intuitive, permettant de visualiser les analyses, exploiter les données collectées et interagir avec l’assistant de manière ergonomique.

Ainsi, ce système contribuera à améliorer la compréhension et la gestion des données SAP, à faciliter l’analyse des anomalies, et à offrir un outil intelligent d’aide à la décision pour les utilisateurs.

2.2 Spécification des besoins

L’analyse des besoins a pour objectif de comprendre les attentes des utilisateurs et de définir le contexte métier dans lequel s’inscrit le système à développer. Elle permet de décrire les fonctionnalités que le système prendra en charge, ses interactions avec les différents intervenants, ainsi que les règles qui encadrent ces interactions.

2.2.1 Identification des acteurs

Pour déterminer les besoins fonctionnels et non fonctionnels, nous débutons par l’identification des acteurs impliqués dans le système, en précisant le rôle spécifique de chacun. Ainsi, nous identifions trois rôles distincts qui interagissent avec le système :

TABLE 2.1 – Acteurs du système AutoMoni et leurs responsabilités

Acteur	Type	Responsabilités principales
Administrateur système	Acteur principal	Gère la plateforme AutoMoni dans son ensemble : configuration des alertes et des règles métier, gestion des comptes utilisateurs, accès à l'historique global des conversations, supervision des performances du pipeline RAG et administration des paramètres système. Il dispose des droits les plus étendus et cumule toutes les fonctionnalités de l'Administrateur SAP.
Administrateur SAP (consultant BASIS)	Acteur principal	Surveille les environnements SAP à travers les données de monitoring collectées en temps réel. Interagit avec l'assistant intelligent pour poser des questions techniques en langage naturel, consulter les alertes T-code, accéder aux journaux de transactions et visualiser les graphiques d'analyse. Peut également configurer les paramètres de surveillance propres à son périmètre client. Soumet des feedbacks sur les réponses du chatbot (notation et correction) afin d'améliorer la qualité du système RAG.
Système SAP (source externe)	Acteur secondaire	Fournit les données de surveillance opérationnelle via les appels RFC (PyRFC). Ne déclenche pas d'interactions directes avec l'interface, mais alimente en continu la base de données de monitoring qui est interrogée par les deux acteurs humains.

2.2.2 Besoins fonctionnels

Les besoins fonctionnels définissent l'ensemble des fonctionnalités que le système doit fournir afin de permettre aux utilisateurs d'exploiter et d'analyser les données issues de l'environnement SAP à travers un assistant intelligent basé sur l'intelligence artificielle et reposant sur une architecture de génération augmentée par récupération (*RAG – Retrieval-Augmented Generation*).

Le système repose sur deux types d'acteurs principaux : l'**Administrateur SAP** et l'**Utilisateur SAP**. Il convient de préciser que l'administrateur SAP englobe l'ensemble des privilèges de l'utilisateur SAP : en plus de ses responsabilités de supervision et de configuration, il peut également interagir avec l'assistant intelligent et exploiter pleinement les fonctionnalités offertes par le module RAG, au même titre qu'un utilisateur standard.

Administrateur SAP

L'administrateur occupe un rôle central de supervision et de gouvernance du système. En tant qu'acteur disposant des droits les plus étendus, il cumule les fonctionnalités d'administration et celles de l'utilisateur SAP. Ses responsabilités spécifiques sont les suivantes :

- S'authentifier à l'application de manière sécurisée.
- Consulter et analyser les journaux d'activité du système SAP.
- Configurer les paramètres du système ainsi que les règles d'alertes.
- Consulter l'historique global des conversations effectuées avec le chatbot.
- Interagir avec l'assistant intelligent (*RAG*) et formuler des requêtes en langage naturel portant sur les données SAP, au même titre qu'un utilisateur.
- Accéder aux graphiques et aux analyses générés par l'assistant intelligent.

Utilisateur SAP

L'utilisateur SAP constitue l'acteur principal de l'application. Il peut exploiter les fonctionnalités suivantes :

- S'authentifier à l'application.
- Interagir avec l'assistant intelligent à travers une interface conversationnelle.
- Poser des questions en langage naturel concernant les données issues du système SAP.
- Consulter les alertes associées aux transactions SAP (T-Code).
- Accéder aux journaux des transactions SAP.

- Visualiser les graphiques et analyses générés par le système.
- Consulter l'historique des interactions avec le chatbot.

2.2.3 Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les critères de qualité et les contraintes que le système doit respecter afin de garantir son bon fonctionnement. Contrairement aux besoins fonctionnels, ils ne décrivent pas les services offerts aux utilisateurs, mais les caractéristiques attendues en matière de performance, de fiabilité, de sécurité et d'évolutivité.

Performance

Le système doit fournir des réponses dans un délai raisonnable afin d'assurer une expérience utilisateur fluide et efficace. La rapidité d'exécution constitue un critère essentiel pour permettre aux administrateurs et consultants SAP d'obtenir les informations nécessaires sans interruption de leurs activités.

Fiabilité

Les réponses générées doivent être précises, cohérentes et basées sur des sources fiables. Le système doit limiter au maximum les erreurs et garantir la pertinence des informations fournies afin d'assurer un niveau élevé de confiance auprès des utilisateurs.

Confidentialité

La protection des données constitue une exigence fondamentale du projet. Les informations manipulées par le système doivent être sécurisées et traitées de manière à préserver la confidentialité des données de l'entreprise et des environnements SAP supervisés.

Scalabilité

Le système doit être capable d'évoluer en fonction de l'augmentation du volume de données et de l'enrichissement de la base de connaissances. Cette capacité d'adaptation permettra d'intégrer de nouvelles ressources et fonctionnalités sans dégrader les performances globales de la solution.

2.3 Gestion de projet avec la méthode CRISP-DM

La méthodologie CRISP-DM (*Cross-Industry Standard Process for Data Mining*) a été adoptée pour structurer et piloter ce projet. Cette démarche itérative, éprouvée

dans les projets de science des données, organise le travail en six phases successives : *Business Understanding*, *Data Understanding*, *Data Preparation*, *Modeling*, *Evaluation* et *Deployment*. Chaque phase produit des livrables concrets qui servent d'entrée à la phase suivante, garantissant ainsi une progression maîtrisée du projet.

Dans le cadre de ce projet, les six phases se déclinent comme suit :

- **Business Understanding** : identification du contexte du projet, analyse des besoins de l'entreprise, définition des objectifs à atteindre ainsi que des exigences fonctionnelles et non fonctionnelles du système.
“ “
- **Data Understanding** : étude et analyse des différentes sources d'information exploitées pour la constitution de la base de connaissances SAP.
- **Data Preparation** : préparation et structuration des données afin de les rendre exploitables par le système de recherche et de génération de réponses.
- **Modeling** : conception et mise en place de la solution intelligente permettant la recherche d'informations pertinentes et la génération de réponses adaptées aux besoins des utilisateurs.
- **Evaluation** : évaluation des performances de la solution développée afin de vérifier sa capacité à fournir des réponses pertinentes et fiables.
- **Deployment** : intégration de l'assistant intelligent au sein de la plateforme AutoMoni et mise à disposition de la solution dans son environnement d'utilisation.
“ “

2.4 Diagramme de Gantt

Le diagramme de Gantt ci-dessous illustre la planification temporelle du projet selon les phases CRISP-DM. Il met en évidence les différentes tâches, leur durée estimée et les dépendances entre les phases tout au long du cycle de développement.

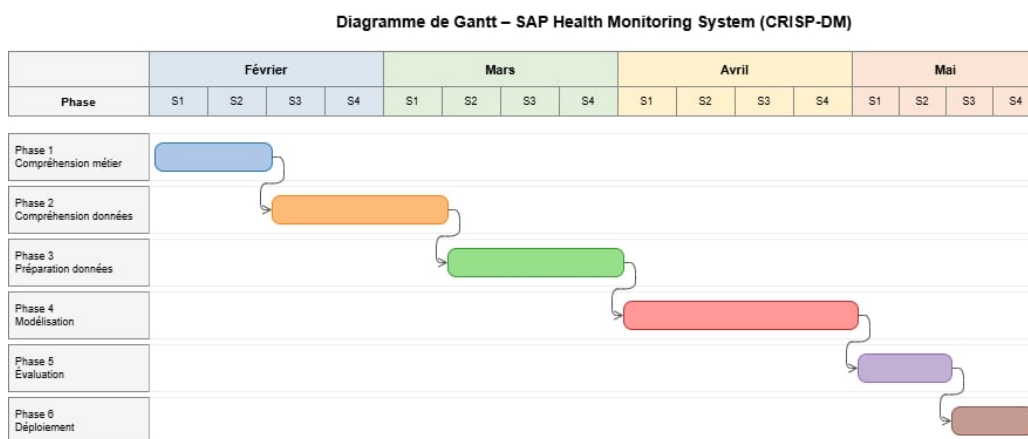


FIGURE 2.1 – Diagramme de Gantt

Conclusion

Ce chapitre a formalisé les besoins fonctionnels et non fonctionnels, identifié les acteurs et structuré la démarche selon la méthodologie CRISP-DM et son calendrier Gantt. Le chapitre suivant présente le contexte métier SAP et les technologies d'IA mobilisées.

Chapitre 3

Compréhension métier

Introduction

Ce chapitre présente le contexte métier et les objectifs du projet d'assistant intelligent de surveillance SAP dans le cadre de la méthodologie CRISP-DM. Il met en évidence l'importance d'aligner les efforts techniques sur les objectifs métier, afin que la solution développée réponde aux défis réels auxquels sont confrontées les équipes SAP.

3.1 Contexte métier SAP

3.1.1 Qu'est-ce qu'un ERP ?

Un système ERP (*Enterprise Resource Planning*) est un logiciel qui intègre et gère les principaux processus métier d'une organisation : finances, ressources humaines, fabrication, chaîne d'approvisionnement, ventes et achats. Il fournit une source unique de vérité, élimine la duplication des données et améliore l'efficacité opérationnelle. Les solutions ERP permettent aux entreprises d'obtenir une visibilité en temps réel sur leurs opérations, d'automatiser les processus et de prendre des décisions basées sur les données.

SAP (*Systems, Applications and Products in Data Processing*) est l'un des principaux fournisseurs de logiciels ERP. Sa suite logicielle couvre de nombreux domaines fonctionnels. Parmi ses modules les plus utilisés :

- **SAP EWM (*Extended Warehouse Management*)** : Module spécialisé dans l'optimisation des entrepôts et la gestion des stocks. Il aide les entreprises à gérer efficacement leurs opérations d'entreposage et à garantir la disponibilité des produits au bon moment.
- **SAP ECC (*Enterprise Central Component*)** : Module central de l'ERP SAP. Il intègre les finances, les ventes, les achats, la fabrication et les ressources humaines

au sein d'une plateforme unifiée.

- **SAP Solution Manager (SOLMAN)** : Outil central de gestion et de surveillance fourni par SAP. Il couvre la gestion du cycle de vie des applications, la gestion de projets, la gestion des services IT et le suivi des processus métier.



FIGURE 3.1 – Logo SAP

3.1.2 Le consultant SAP BASIS

Le consultant SAP BASIS est responsable de l'infrastructure technique de l'environnement SAP : installations, mises à jour et application des correctifs. Il assure la résolution des problèmes de connectivité, les ajustements de composants et l'assistance aux systèmes externes et aux bases de données.

Ses principales responsabilités englobent l'administration des systèmes, la surveillance des performances, la sécurité et la gestion des bases de données, ainsi que la résolution des incidents techniques.

Il prend également en charge la sauvegarde et la récupération des données, la gestion des intégrations et des interfaces, les mises à jour et migrations des systèmes, et veille à maintenir une documentation à jour tout en assurant la formation des équipes concernées.

Dans le cadre du suivi opérationnel, il surveille en continu les performances et le bon fonctionnement des systèmes SAP à l'aide des transactions (*T-Codes*) dédiées, et applique les correctifs nécessaires via les *SAP Notes* afin de garantir la stabilité et la fiabilité de l'environnement.

3.1.3 Les codes de transaction SAP (T-codes)

Dans SAP, les codes de transaction (T-codes) sont des commandes alphanumériques courtes permettant d'accéder rapidement à des fonctions spécifiques, des rapports ou des diagnostics système. Ils servent de points d'entrée dans les modules fonctionnels de SAP, permettant aux utilisateurs de contourner les menus profonds et d'exécuter directement des tâches telles que la surveillance des jobs, la gestion des files d'attente ou l'analyse des performances.

Chaque T-code correspond à une fonction ou un programme backend — généralement implémenté en ABAP — qui effectue un ensemble défini d'opérations. Lors de l'exécution d'un T-code, SAP vérifie les autorisations de l'utilisateur, exécute la logique associée et présente une interface interactive.

Dans le cadre de ce projet, un ensemble de T-codes à haute criticité a été sélectionné pour assurer une couverture complète du comportement des systèmes SAP :

- ST22 – Analyse des erreurs d’exécution ABAP (*short dumps*).
- DB02 – Surveillance des statistiques de base de données (croissance, espace table, index manquants).
- SM37 – Surveillance des jobs en arrière-plan.
- SP01 – Vérification des requêtes d’impression et de leurs statuts.
- SM66 – Vue globale des processus de travail actifs.
- SM58 – Analyse des erreurs RFC transactionnelles (traitement asynchrone).
- SMQ1, SMQ2 – Surveillance des files d’attente entrantes et sortantes (qRFC/tRFC).
- SM12, SM13 – Révision des entrées verrouillées et des enregistrements de mise à jour.
- ST03N – Analyse des statistiques de charge de travail et de performance.
- SOST – Surveillance des messages SAPconnect (e-mails, fax, etc.).
- ST13 – Outils BASIS pour les diagnostics approfondis et les journaux.

Ces T-codes ont été sélectionnés en raison de leur criticité pour les opérations en temps réel, de leur pertinence pour les équipes BASIS et de leur fréquence d’utilisation lors de la résolution des incidents. En surveillant en continu ces transactions et en interprétant leurs sorties par le biais d’un raisonnement assisté par IA, le système permet une détection précoce des anomalies et soutient une analyse plus rapide des causes profondes.

SM37 : Job Overview

SM37 surveille les *jobs batch* ce sont des tâches automatiques qui tournent en arrière-plan (traitements de données, interfaces, rapports). L’administrateur vérifie qu’aucun job n’est en erreur ou bloqué.

Job	Job Created	Status	Start date	Start time	Duration(sec.)	Delay (sec.)
/BDL/TASK_PROCESSOR	ITELLIUSER	Released			0	0
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	00:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	01:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	02:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	03:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	04:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	05:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	06:27:36	0	15
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	07:27:30	0	9
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	08:27:29	0	8
/BDL/TASK_PROCESSOR	ITELLIUSER	Finished	24.04.2026	09:27:21	0	0
/SDF/MON_SCHEDULER	ABDELMOU	Finished	24.04.2026	00:00:01	3	1
/SDF/MON_WATCHDOG	ABDELMOU	Released			0	0
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	00:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	01:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	02:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	03:50:25	1	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	04:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	05:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	06:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	07:50:25	0	11
/SDF/MON_WATCHDOG	ABDELMOU	Finished	24.04.2026	08:50:14	0	0
0000000094636939	NFPA-211010	Finished	24.04.2026	06:02:05	1	0
0000000094636940	NFPA-211010	Finished	24.04.2026	06:02:06	0	1
0000000094636941	NFPA-211010	Finished	24.04.2026	06:02:05	1	0
0000000094636942	NFPA-211010	Finished	24.04.2026	06:02:05	1	0
0000000094636943	NFPA-211010	Finished	24.04.2026	06:02:09	3	0
0000000094636944	NFPA-211010	Finished	24.04.2026	06:02:09	1	0
0000000094636945	NFPA-211010	Finished	24.04.2026	06:02:09	1	0
0000000094636946	NFPA-211010	Finished	24.04.2026	06:02:09	1	0

FIGURE 3.2 – SM37 — Vue d'ensemble des jobs batch

La figure 3.2 présente l'interface du T-code **SM37**, qui affiche la liste des jobs planifiés avec leur statut (*Finished*, *Released*, *Active*), leur date et heure de démarrage, leur durée d'exécution ainsi que le délai éventuel. Cette vue permet à l'administrateur BASIS d'identifier en un coup d'œil tout job bloqué, annulé ou présentant un délai anormal, et d'intervenir rapidement avant que cela n'impacte les processus métier.

DB02 : Space Overview

DB02 surveille la base de données l'administrateur contrôle l'espace disque disponible pour éviter que la base de données sature, ce qui provoquerait un arrêt complet du système.

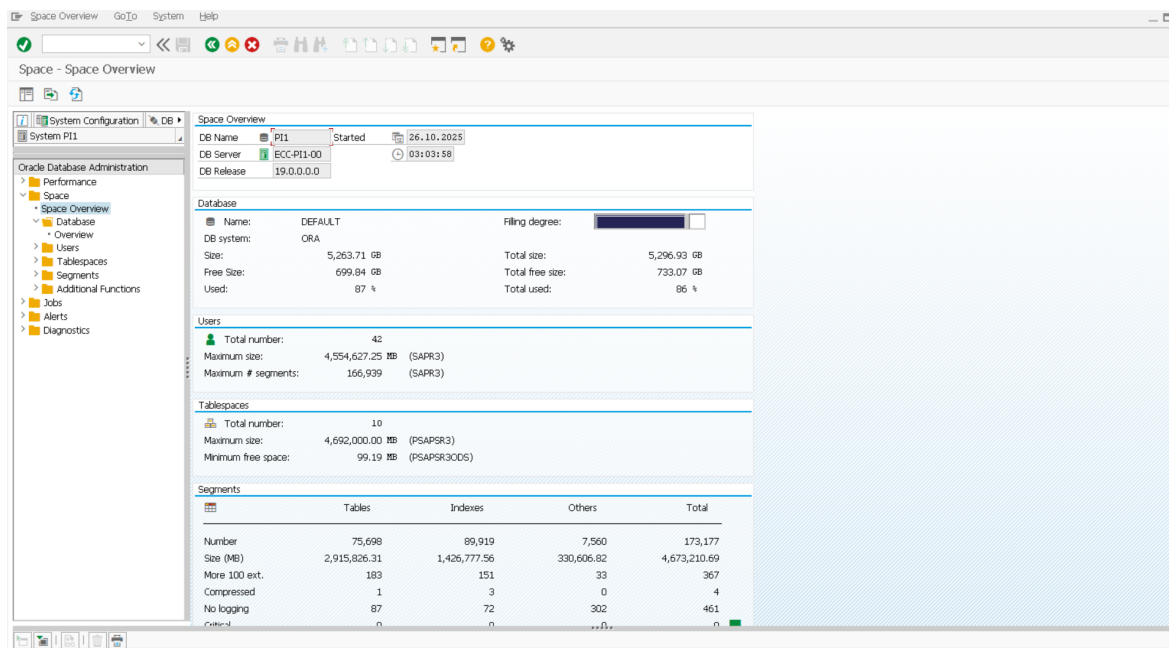


FIGURE 3.3 – DB02 — Vue d'ensemble de l'espace base de données

La figure 3.3 illustre le T-code DB02, qui fournit une vue synthétique de l'état de la base de données Oracle. On y retrouve le taux de remplissage global, la taille totale et l'espace libre, ainsi que le détail par tablespace et par segment (tables, index, autres). Un taux d'utilisation élevé comme les 87% visibles ici — constitue un signal d'alerte critique nécessitant une action immédiate d'extension ou de purge pour prévenir toute saturation du système.

ST22 : Runtime Errors

ST22 liste les erreurs critiques ABAP — l'administrateur analyse les *dumps* pour identifier les programmes qui plantent et corriger les problèmes avant qu'ils n'impactent les utilisateurs.

Current Date Time	App. Server	User Name	CL - KeName of runtime error	Exception	Appl. component
24.04.2026 09:19:16	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 09:18:50	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 09:13:51	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 09:00:46	ECC-P11-01_P11...	WF-BATCH	620 C SAPSQL_ARRAY_INSERT_DURREC	CK_SY_OPEN_SQL_DB	
24.04.2026 08:43:05	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 08:42:40	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 08:36:46	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 08:24:29	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 08:10:16	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 07:57:02	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 07:56:35	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 07:43:34	ECC-P11-00_P11...	CONFAUTO	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 07:36:39	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 06:55:42	ECC-P11-00_P11...	WF-BATCH	620 C SAPSQL_ARRAY_INSERT_DURREC	CK_SY_OPEN_SQL_DB	
24.04.2026 06:36:43	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 05:36:39	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 05:02:16	ECC-P11-00_P11...	WF-BATCH	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 05:01:23	ECC-P11-00_P11...	WF-BATCH	620 C TSV_TNEW_PAGE_ALLOC_FAILED		
24.04.2026 04:36:46	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 03:36:41	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 02:36:44	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 01:36:47	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
24.04.2026 00:36:43	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
23.04.2026 23:36:42	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
23.04.2026 23:00:27	ECC-P11-01_P11...	WF-BATCH	000 C DDIC_TYPES_INCONSISTENT		
23.04.2026 23:00:24	ECC-P11-01_P11...	WF-BATCH	000 C DDIC_TYPES_INCONSISTENT		
23.04.2026 23:00:22	ECC-P11-01_P11...	WF-BATCH	000 C DDIC_TYPES_INCONSISTENT		
23.04.2026 23:00:19	ECC-P11-01_P11...	WF-BATCH	000 C DDIC_TYPES_INCONSISTENT		
23.04.2026 22:36:48	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
23.04.2026 21:36:47	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
23.04.2026 20:36:58	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		
23.04.2026 20:00:54	ECC-P11-00_P11...	SM_PM1	000 C TABLE_ILLEGAL_STATEMENT		

FIGURE 3.4 – ST22 — Liste des erreurs d’exécution ABAP

La figure 3.4 présente l’interface du T-code ST22, qui recense l’ensemble des *short dumps* ABAP survenus sur le système. Chaque entrée indique la date, l’heure, le serveur applicatif, l’utilisateur concerné et le nom de l’erreur. Les erreurs récurrentes telles que TSV_TNEW_PAGE_ALLOC_FAILED (manque de mémoire) ou TABLE_ILLEGAL_STATEMENT (instruction SQL non autorisée) sont des indicateurs d’anomalies systémiques qui doivent être analysées et résolues en priorité par l’équipe BASIS.

3.2 État de l’art

3.2.1 Les Fondements Technologiques de l’IA Générative

L’intelligence artificielle générative repose sur un ensemble de technologies complémentaires qui, combinées, permettent de concevoir des systèmes capables d’analyser, de raisonner et de produire du contenu à partir de données complexes. Cette section présente les principaux fondements technologiques qui soutiennent la compréhension et la génération du langage naturel.

Elle introduit tout d’abord les concepts essentiels de l’apprentissage automatique (*Machine Learning*) et de l’intelligence artificielle, avant d’explorer l’apprentissage profond (*Deep Learning*) ainsi que les modèles basés sur l’architecture *Transformer*. Elle aborde ensuite le traitement automatique du langage naturel (*NLP – Natural Language Processing*) et met en lumière l’émergence des modèles de langage de grande taille (*LLMs – Large Language Models*), qui jouent un rôle central dans la génération de texte. Enfin, l’architecture *RAG (Retrieval-Augmented Generation)* est présentée

comme une approche permettant d'enrichir ces modèles en leur fournissant un accès à des connaissances actualisées et vérifiables. L'ensemble de ces technologies constitue le socle sur lequel repose la solution proposée dans ce projet.

3.2.2 Machine Learning et Intelligence Artificielle

Le *Machine Learning*, ou apprentissage automatique, constitue l'un des piliers fondamentaux de l'intelligence artificielle. Il permet à un système d'apprendre à partir de données sans être explicitement programmé pour chaque tâche. En analysant des ensembles de données, le modèle identifie des régularités statistiques qu'il utilise pour effectuer des prédictions sur de nouvelles données. Cette capacité d'apprentissage automatique est à la base des avancées récentes en intelligence artificielle générative.

3.2.3 Deep Learning

L'apprentissage profond (*Deep Learning*) est une sous-discipline du *Machine Learning* qui repose sur des réseaux de neurones artificiels composés de plusieurs couches. Ces réseaux permettent d'extraire progressivement des représentations de plus en plus abstraites des données. Grâce à cette structure, ils sont particulièrement efficaces pour traiter des données non structurées telles que le texte, les images ou encore les signaux audio.

3.2.4 Architecture Transformer

Proposée en 2017 dans l'article « *Attention is All You Need* » [10], l'architecture *Transformer* s'est imposée comme une référence dans le domaine du traitement du langage naturel. Elle repose sur le mécanisme d'**attention** (*self-attention*), qui permet au modèle d'évaluer l'importance de chaque mot par rapport à tous les autres mots d'une séquence, indépendamment de leur position.

Contrairement aux architectures séquentielles traditionnelles telles que les réseaux récurrents (*RNN*, *LSTM*), le Transformer traite les données en parallèle, ce qui améliore considérablement les performances et permet de capturer efficacement les dépendances à long terme dans le texte.

3.2.5 Traitement Automatique du Langage Naturel (NLP)

Le traitement automatique du langage naturel (*NLP*) est un domaine de l'intelligence artificielle dédié à l'interaction entre les machines et le langage humain. Il regroupe un ensemble de techniques permettant de comprendre, analyser et générer du texte [6].

Parmi les principales applications du NLP figurent la classification de textes, l'analyse de sentiments, la traduction automatique, la reconnaissance d'entités nommées ainsi que la génération automatique de contenu.

3.2.6 Les Modèles de Langage de Grande Taille : Capacités et Limites

Les modèles de langage de grande taille (*LLMs*) sont des modèles avancés d'apprentissage profond entraînés sur des volumes massifs de données textuelles (livres, articles, sites web, code, etc.). Leur objectif est de comprendre et de générer un langage proche de celui des humains [1].

Comme illustré dans la figure 3.5, ces modèles se situent à l'intersection du *Deep Learning* et du *NLP*, combinant ainsi la puissance des réseaux de neurones profonds avec les techniques d'analyse linguistique.

En pratique, un LLM prédit le mot (ou jeton) le plus probable à la suite d'une séquence donnée. En répétant ce processus, il est capable de produire des textes cohérents, de répondre à des questions, de résumer des documents ou encore de traduire des contenus. Plus le modèle contient de paramètres, plus il est capable de capturer les nuances complexes du langage.

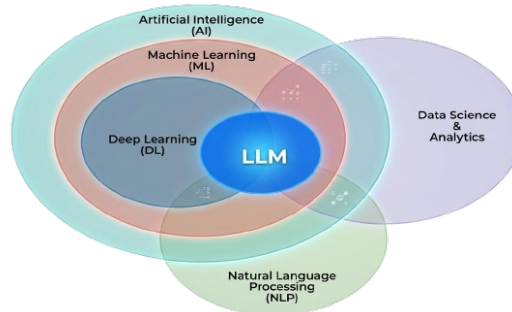


FIGURE 3.5 – Les LLMs à l'intersection du *Deep Learning* et du *NLP*

Malgré leurs performances impressionnantes, les LLMs présentent certaines limites. Leur connaissance est limitée aux données utilisées lors de leur entraînement, ce qui les empêche de traiter des informations récentes. De plus, ils peuvent générer des **hallucinations**, c'est-à-dire produire des réponses plausibles mais incorrectes. Enfin, ils n'ont pas accès aux données internes ou confidentielles d'une organisation.

Parmi les modèles les plus connus figurent *GPT-4*, *Claude* et *Gemini*.

3.2.7 Architecture de RAG (Retrieval-Augmented Generation)

Bien que les LLMs possèdent une connaissance générale acquise lors de leur entraînement, ils ne disposent pas d'un accès direct aux données spécifiques d'une organisation

ni aux informations les plus récentes. L'architecture *RAG* (*Retrieval-Augmented Generation*) [7] a été conçue pour répondre à cette limitation.

Son fonctionnement repose sur plusieurs étapes. Tout d'abord, les documents sont découpés en fragments appelés *chunks*. Chaque fragment est ensuite transformé en un vecteur numérique (*embedding*) représentant son contenu sémantique [8]. Ces vecteurs sont stockés dans une base de données vectorielle.

Lorsqu'une requête utilisateur est soumise, le système identifie les fragments les plus pertinents à l'aide d'une mesure de similarité (généralement la similarité cosinus). Ces informations sont ensuite injectées dans le contexte du modèle, qui génère une réponse basée sur des données réelles et vérifiables.

Ainsi, le RAG peut être comparé à un assistant intelligent capable de consulter une base documentaire avant de formuler une réponse, garantissant ainsi une meilleure fiabilité.

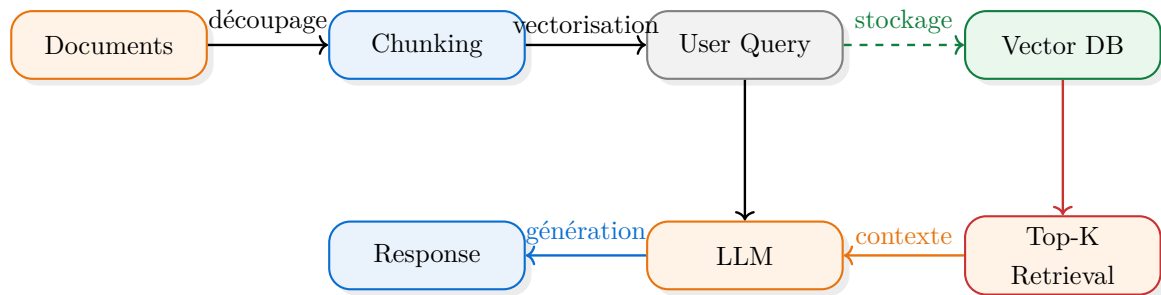


FIGURE 3.6 – Architecture du système RAG

3.2.8 L'architecture de CAG (Cache-Augmented Generation) : une amélioration du RAG

Bien que l'architecture RAG améliore significativement la génération de réponses en intégrant des documents externes, elle présente encore certaines limites, notamment en termes de latence et de coût de calcul, dues aux requêtes répétées vers la base vectorielle.

Pour répondre à ces contraintes, une évolution appelée *CAG* (*Cache-Augmented Generation*) [2] a été proposée. Le principe de cette approche repose sur l'introduction d'un mécanisme de cache intelligent permettant de stocker les résultats des requêtes précédentes ainsi que les documents fréquemment récupérés.

Ainsi, lorsqu'une requête similaire est posée, le système peut réutiliser directement les informations déjà calculées sans solliciter à nouveau la base vectorielle ou les modèles d'embedding. Cela permet de réduire le temps de réponse et d'améliorer l'efficacité globale du système.

En comparaison avec le RAG classique, le CAG optimise le processus de génération en réduisant les redondances de calcul tout en conservant la pertinence des informations fournies. Cette approche est particulièrement utile dans les systèmes à forte charge ou

dans les applications nécessitant une réponse en temps réel.

3.2.9 L'architecture de Fine-Tuning : personnalisation des modèles IA

En plus des approches RAG et CAG, le *Fine-Tuning* est une autre méthode souvent utilisée pour améliorer les performances des modèles d'intelligence artificielle [4]. Cette technique consiste à ré-entraîner un modèle de langage préexistant sur des données propres à un domaine ou à une organisation.

Le Fine-Tuning permet au modèle d'acquérir le vocabulaire métier, les structures de données et les connaissances spécifiques à une entreprise. Le Fine-Tuning, à la différence du RAG qui récupère des informations externes dynamiquement au moment de la requête, intègre directement ces connaissances dans les paramètres internes du modèle.

Cette approche a plusieurs avantages, notamment une meilleure spécialisation du modèle, une génération de réponses plus cohérente et une réduction de la dépendance aux bases documentaires externes. Mais, elle demande beaucoup de ressources lors de l'entraînement, des mises à jour fréquentes des données et un coût de calcul plus lourd.

3.2.10 Comparaison des approches Fine-Tuning, RAG et CAG

Afin de mieux positionner ces trois approches les unes par rapport aux autres, il convient de les comparer selon plusieurs critères déterminants. Chacune d'elles présente un profil distinct en termes de flexibilité, de coût et de précision. Le Fine-Tuning offre une très haute spécialisation métier, mais au prix d'un entraînement coûteux et d'une mise à jour manuelle des connaissances. Le RAG classique permet une adaptation dynamique aux nouvelles informations grâce à la base documentaire, mais peut souffrir de latences liées aux requêtes vectorielles répétées. Le RAG enrichi du mécanisme de cache (CAG) combine la pertinence documentaire du RAG avec une réduction significative des délais de calcul, ce qui le rend particulièrement adapté aux environnements à forte charge comme les systèmes SAP en production. Le tableau 3.1 synthétise cette comparaison selon les principaux critères d'évaluation.

Critère	Fine-Tuning	RAG	RAG + CAG (RAG avancé)
Principe	Réentraîner le modèle sur des données spécifiques	Récupération dynamique de documents pertinents	RAG avec mécanisme de cache intelligent
Accès aux données récentes	Limité après l'entraînement	Oui	Oui avec optimisation du cache
Temps de réponse	Rapide	Moyen	Très rapide
Coût de calcul	Élevé lors de l'entraînement	Modéré	Optimisé grâce au cache
Mise à jour des connaissances	Nécessite un nouvel entraînement	Automatique via la base documentaire	Automatique avec réutilisation des résultats
Précision métier	Très élevée	Élevée	Très élevée
Dépendance à une base vectorielle	Non	Oui	Oui
Cas d'utilisation	Modèles spécialisés métier	Assistants documentaires intelligents	Applications temps réel et systèmes à forte charge

TABLE 3.1 – Comparaison entre Fine-Tuning, RAG et RAG+CAG

3.3 Le choix du RAG pour ce projet

Dans le cadre de ce projet, l'architecture RAG avec le CAG a été retenue pour plusieurs raisons essentielles :

- **Adaptation à un domaine évolutif** : Dans des environnements tels que les systèmes ERP et SAP, les informations évoluent fréquemment. Le RAG permet d'intégrer des données actualisées au moment de la génération des réponses.
- **Amélioration de la fiabilité** : En s'appuyant sur des documents externes, le RAG réduit significativement les hallucinations et améliore la précision des réponses.
- **Optimisation des coûts** : Contrairement à d'autres approches, le RAG et spécifiquement le CAG ne nécessite pas de réentraînement du modèle, ce qui permet de limiter les coûts tout en conservant ses capacités en utilisant le cache.

En outre, en l'absence de jeux de données annotés par des experts et compte tenu de la complexité des *SAP Notes*, l'utilisation d'approches telles que la distillation de

connaissances n'est pas adaptée. Le RAG constitue ainsi un compromis efficace entre flexibilité, précision et scalabilité, particulièrement pertinent pour les applications en entreprise.

Conclusion

Ce chapitre met en évidence l'importance stratégique de la mise en place d'un système intelligent dédié à la détection des anomalies dans les transactions SAP, en s'appuyant sur une démarche structurée inspirée de la méthodologie CRISP-DM.

En alignant les solutions technologiques avec les besoins métiers, ce projet vise à améliorer l'efficacité opérationnelle des équipes BASIS, à faciliter la prise de décision et à réduire le temps nécessaire à la résolution des incidents. Il s'inscrit ainsi dans une dynamique d'innovation où l'exploitation des données devient un levier essentiel pour optimiser la performance des systèmes d'information.

Chapitre 4

Compréhension des données

Introduction

Dans la méthode CRISP-DM, la phase d'analyse des données est essentielle pour garantir le succès d'un projet d'analyse de données ou de machine learning. Cette étape permet de se familiariser avec les ensembles de données à disposition, de repérer d'éventuels problèmes et de définir les actions nécessaires pour préparer les données en vue de leur utilisation dans les étapes ultérieures du projet. Dans le cadre de ce projet, l'exploration des données implique l'examen des différentes sources.

4.1 Collecte des Données

4.1.1 Données de Surveillance SAP : Extraction via PyRFC

Les données de surveillance opérationnelle sont collectées automatiquement à partir des systèmes SAP de production en utilisant PyRFC, le connecteur Python pour les appels de fonctions distantes SAP. Cette bibliothèque permet d'interagir directement avec les modules de fonction ABAP(le langage de programmation), exposés par le serveur d'application, sans avoir besoin de l'interface SAP GUI. Cette méthode facilite l'automatisation, la planification et la répétabilité du processus d'extraction des données.

Chaque enregistrement extrait contient deux attributs essentiels pour garantir la traçabilité des informations collectées.

Chaque Tcode a une structure bien définie, mais il y a des champs existants comme le champ `Extraction_Stamp` enregistre la date et l'heure de la collecte des données, ce qui permet de garder un historique des événements et des métriques observées. L'extraction comprend plusieurs transactions SAP pour la surveillance opérationnelle, notamment les journaux d'erreurs, les traitements en arrière-plan, les verrouillages, les processus actifs et la messagerie système.

```
_id: ObjectId('6914a49da720a433b96b57db')
Current_Date : "2025-11-12"
Time : "09:25:36"
Application_Server : "ECC-PI1-00_PI1_00"
User_Name : "CONFAUTO"
Client : "620"
Runtime_Error_Name : "TSV_TNEW_PAGE_ALLOC_FAILED"
SID : "PI1"
Extraction_Timestamp : "2025-11-12 15:15:41"
```

FIGURE 4.1 – Structure des données extraites depuis la transaction ST22

```
_id: ObjectId('6914a49da720a433b96b57b1')
Job : "Z3_DE_SD_CUST_MASTER_SIEB_UPDATE"
Job_Created_By : "A.SENG"
Status : "Cancelled"
Start_Date : "2025-11-12"
Start_Time : "15:32:01"
End_Date : "2025-11-12"
End_Time : "15:32:01"
SID : "PI1"
Extraction_Timestamp : "2025-11-12 15:15:41"
```

FIGURE 4.2 – Structure des données extraites depuis la transaction SM37

4.1.2 Les données documentaires pour l'architecture de RAG

Les données de notre projet se basent sur deux sources documentaires.

La première source se compose de plus de 300 documents de formation en SAP sous format pdf. Ceux-ci traitent de divers sujets allant de l'administration BASIS, à l'écriture de code en ABAP jusqu'à la formation fonctionnelle. Ils constituent une documentation complète concernant les concepts et architectures SAP.

La deuxième source consiste dans le site Web d'aide SAP (help.sap.com). Certaines de ses parties ont été extraites via le processus de web scraping. Cette documentation contient des informations techniques relatives à la configuration système SAP, à l'inter-

prétation des codes d'erreurs SAP, à la configuration des paramètres de SAP ainsi que des notes de corrections.

4.1.2.1 Extraction des livres PDF

La tâche d'extraction du contenu de + 300 livres SAP est coordonnée par un script qui lance un agent d'extraction par livre, traitant donc tous ces processus simultanément. Un tel agent agit indépendamment : ouvre le PDF en utilisant la librairie **pdfplumber** et procède au recensement des propriétés typographiques du document (taille et style des polices), ce qui lui permet de générer automatiquement la structure du document (unité, leçon, sections et contenu principal).

Ensuite, le système effectue l'extraction des différents fragments textuels, détection des figures et leur légende, des tableaux et identification des éléments SAP pertinents, notamment des transactions (*T-codes*) et des notes SAP. Toute information structurée est ensuite sauvegardée dans un fichier JSON.

En raison d'une telle approche à la résolution de la tâche d'information extraction, la durée globale de traitement de l'ensemble du corpus a été réduite à plusieurs dizaines de minutes par rapport à plusieurs heures.

4.1.2.2 Extraction des données depuis le portail SAP Help

La récupération des données du portail **help.sap.com** sera réalisée à l'aide d'un processus de scraping permettant d'extraire automatiquement les pages de documentation SAP. Deux liens principaux ont été utilisés : le premier fournit la liste des pages disponibles sur le portail, tandis que le second permet d'accéder au contenu détaillé de chaque page.

Lors des premières tentatives de collecte, certaines requêtes ont été bloquées par le portail SAP afin de limiter les accès automatisés. Pour résoudre ce problème, plusieurs ajustements ont été mis en place.

La première solution consiste à utiliser différents identifiants de navigateurs web lors des requêtes. Cette approche permet au script de se comporter comme un utilisateur naviguant depuis différents navigateurs classiques, ce qui réduit les risques de blocage.

La seconde solution repose sur une collecte parallèle modérée des pages du portail. Plusieurs requêtes sont exécutées simultanément afin d'accélérer l'extraction des données, tout en conservant un rythme raisonnable pour éviter les limitations imposées par le serveur SAP. Grâce à ces ajustements, l'accès aux ressources documentaires a pu être réalisé de manière stable et efficace.

4.1.2.3 Cadre juridique et éthique de la collecte

La collecte automatisée du portail `help.sap.com` a été réalisée dans un cadre académique, à des fins de recherche et de développement d'un prototype non commercial. Avant le lancement du scraper, le fichier `robots.txt` du portail a été consulté afin d'identifier les répertoires explicitement exclus de toute indexation automatique ; ces sections ont été écartées du périmètre de collecte. Le débit des requêtes a été volontairement limité afin de ne pas surcharger les serveurs SAP. La collecte a porté exclusivement sur des pages de documentation publiquement accessibles, sans contournement d'aucune mesure d'authentification ou de contrôle d'accès. Les données collectées sont utilisées uniquement dans le cadre interne du projet, conformément aux conditions générales d'utilisation du portail SAP.

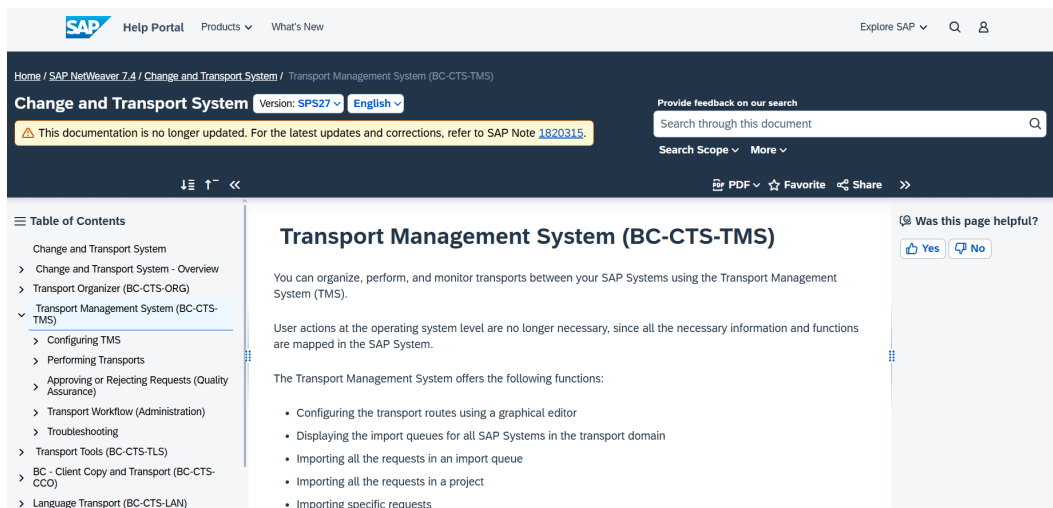


FIGURE 4.3 – les sites de SAP Help

4.1.3 Ingestion des données

L'ingestion de données provenant de diverses sources est cruciale pour appréhender et préparer les données en vue de leur utilisation dans un système RAG. Les données extraites des livres au format PDF, du portail d'aide SAP et de la surveillance en temps réel des codes de transaction sont organisées de manière hiérarchique. Cette structure hiérarchique vise à représenter fidèlement l'organisation logique des documents SAP.

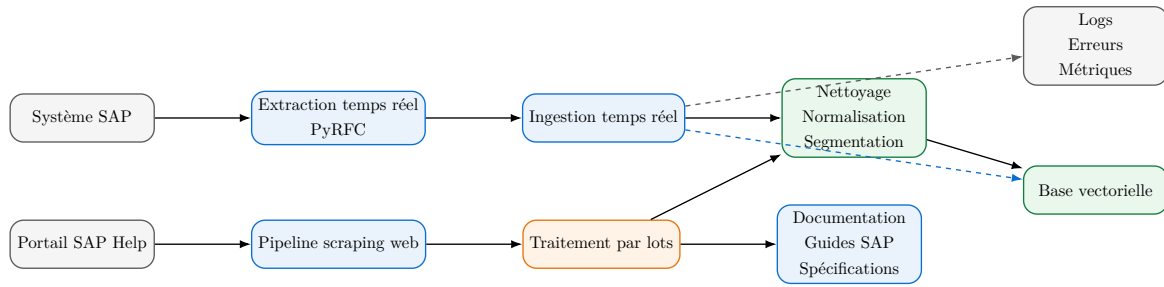


FIGURE 4.4 – Pipeline d’ingestion et de traitement des données SAP

4.2 Description des données à exploiter

Le corpus documentaire issu des livres de formation et du portail SAP Help est composé de segments textuels structurés. Chaque segment représente une unité d’information cohérente, généralement constituée de plusieurs paragraphes, centrés sur un thème précis lié aux différents domaines SAP.

Ces segments permettent d’organiser le contenu de manière logique et exploitable, en regroupant les informations par sujet afin de faciliter leur traitement et leur utilisation dans des tâches d’analyse ou de recherche.

4.2.1 Nature des données résultantes de l’extraction

Les données varient selon les documents, qu’il s’agisse de pages Web (crawlé) ou de PDF (livres). Cependant, la plupart des données sont textuelles et fournissent des informations techniques sur SAP, notamment des descriptions de processus, des configurations système, des analyses de codes d’erreur et des notes de correction. Ces informations sont cruciales pour comprendre en profondeur les systèmes SAP et résoudre les problèmes qui peuvent survenir.

4.2.1.1 Nature des données de livres PDF résultantes de l’extraction

Les données utilisées dans cette étape proviennent de documents techniques SAP au format PDF. Ces documents sont considérés comme le guide principal pour la certification de système SAP. Ils incluent des supports de formation tels que BC100, BC400, ainsi que d’autres modules SAP comme ABAP et d’autres.

Ces documents contiennent des données non structurées, qui comprennent du texte technique, des éléments graphiques, des tableaux et des fragments de code ABAP. L’objectif principal du processus d’extraction est de transformer ces documents en une représentation structurée. Cette représentation structurée doit être exploitable dans un système RAG géré par des vecteurs, également appelés embeddings. Le but est de permettre au modèle LLM de comprendre ces données de manière efficace.

4.2.1.2 Nature des données de SAP Help résultantes de l'extraction

Les données utilisées dans ce projet viennent d'un processus automatisé de collecte sur le portail SAP Help (<https://help.sap.com>). Elles sont non structurées et semi-structurées, et concernent la documentation technique de SAP NetWeaver 7.03 et autre comme NetWeaver 7.4.

Les informations recueillies incluent :

- Des pages de documentation avec des descriptions des composants SAP et de conception des systèmes,
- Des guides techniques qui détaillent les procédures à suivre,
- Des informations sur la configuration du système,
- Des renseignements sur les erreurs et les journaux SAP,
- Des éléments qui structurent hiérarchiquement le contenu, comme les titres et les sections.

4.2.1.3 Nature des données de surveillance SAP en temps réel

les données de surveillance SAP en temps réel sont collectées à partir des systèmes de production SAP des clients Quadient en utilisant PyRFC. Ces données sont structurées et comprennent des informations sur les transactions SAP, selon les codes de transaction (T-codes) utilisés, avec des structures bien définies qui varient en fonction des T-codes.

Chaque enregistrement contient des attributs tels que le code de transaction, la description de l'événement, la date et l'heure d'extraction, ainsi que d'autres détails pertinents pour la surveillance opérationnelle.

Ces données sont très importantes pour analyser les performances du système et identifier les problèmes potentiels.

4.2.2 Exploration des données résultantes de l'extraction

Les données extraites des livres PDF et du portail SAP Help sont organisées selon une structure hiérarchique qui reproduit fidèlement l'architecture logique des documents SAP. Chaque document PDF est ainsi converti en un ensemble de sections imbriquées suivant les niveaux suivants : Book (document source), Unit, Lesson, Section et Body (contenu détaillé).

L'ensemble de ces informations est ensuite enregistré dans des fichiers JSON. Chaque section est représentée sous la forme d'un objet JSON structuré contenant les différents champs nécessaires à sa description et à sa position dans la hiérarchie du document. Les champs principaux de chaque enregistrement comprennent :

- `doc_id` : identifiant unique généré (UUID)

- `source_book` : nom du document PDF
- `title` : titre de la section
- `content` : texte extrait et nettoyé
- `heading_level` : niveau hiérarchique (0 à 4)
- `parent_id` : relation hiérarchique avec la section parente
- `position` : ordre d'apparition dans le document
- `page_start` / `page_end` : localisation dans le PDF
- `section_type` : type de contenu (unit, lesson, section, body)

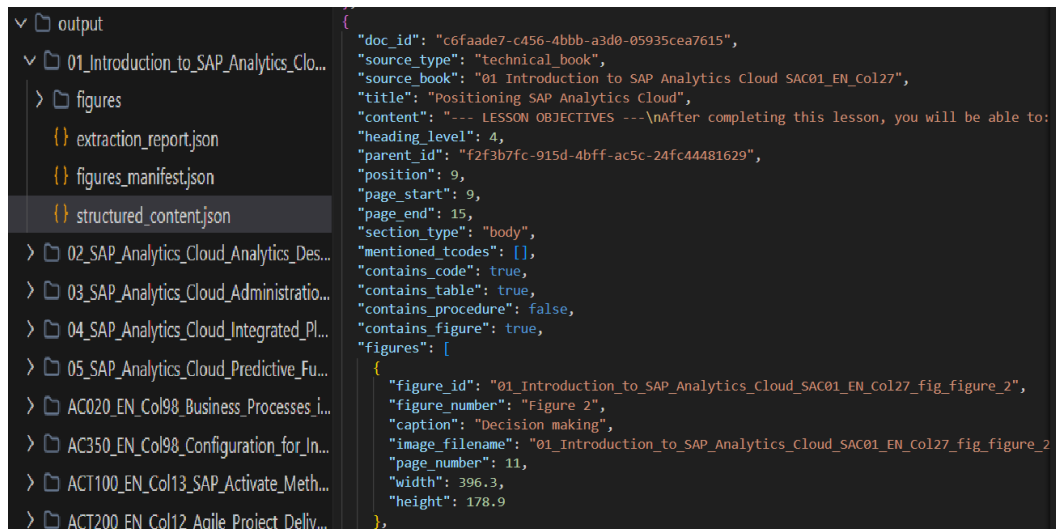


FIGURE 4.5 – Données des livres PDF SAP

En outre, les pages extraites par le scraper sont transformées en un objet structuré au format JSON. La structure d'un enregistrement est définie comme suit :

- `id` : identifiant de la page
- `doc_id` : identifiant du document SAP (extrait de l'URL)
- `title` : titre de la page
- `url` : lien source de la page
- `depth` : profondeur de navigation dans le crawler
- `root_title` : titre du document racine
- `breadcrumb` : chemin hiérarchique de la page dans la documentation
- `content` : contenu textuel principal nettoyé
- `sections` : structure hiérarchique des sections du document
- `scraped_at` : horodatage de l'extraction

```

{
  "id": "0433db3fd3d44bc990182b72ea376727",
  "doc_id": "0433db3fd3d44bc990182b72ea376727",
  "title": "Making Forums Available in the Portal",
  "url": "https://help.sap.com/doc/0433db3fd3d44bc990182b72ea376727/7.03.30/en-US/index.html?forSiteMap=true",
  "depth": 1,
  "root_title": "Making Forums Available in the Portal",
  "root_id": "0433db3fd3d44bc990182b72ea376727",
  "breadcrumb": "Making Forums Available in the Portal",
  "hierarchy": [],
  "content": "Making Forums Available in the Portal",
  "sections": [
    {
      "level": 1,
      "title": "Making Forums Available in the Portal",
      "content": "Making Forums Available in the Portal ContentMaking Forums Available in the PortalPlanning ForumsForum Con
    }
  ],
  "toc": [],
  "tables": [],
  "code_blocks": [],
  "scraped_at": "2026-02-03T11:15:54.348207"
},

```

FIGURE 4.6 – Données de SAP Help

Cette structuration permet de transformer des pages HTML non structurées en documents exploitables pour un système RAG.

D'autre part, les données de surveillance SAP en temps réel sont collectées directement depuis les systèmes de production des clients Quadient à l'aide de PyRFC. Ces données sont structurées et contiennent des informations relatives aux transactions SAP, identifiées par les codes de transaction (T-codes). Leur organisation varie en fonction du type de T-code, selon des structures prédéfinies adaptées à chaque cas d'usage. Chaque enregistrement contient des attributs tels que :

- le code de transaction,
- la description de l'événement,
- la date et l'heure d'extraction,
- ainsi que d'autres détails pertinents pour la surveillance opérationnelle.

▼ Monitoring	
CPU ...	<code>_id: ObjectId('691b14cd1b95eb497f5f4598')</code>
DB02	<code>TYPE : 101</code>
FSYS	<code>SUBTYPE : 2</code>
MEMORY	<code>SERIALNR : 12</code>
MSS_DB	<code>INT_HOUR : 5724060</code>
SM12	<code>SYSC_HOUR : 716812920</code>
SM13	<code>CS_HOUR : 284212200</code>
SM37	<code>LD_AVG_MAX : 758342961</code>
SM51	<code>LD_AVG_MIN : 945907257</code>
SM58	<code>SID : "SMP"</code>
	<code>Extraction_Timestamp : "2025-11-17 13:27:48"</code>
	<code>HOURL_EXTRACTED : 12</code>
	<code>user_cpu_percent : 32</code>
	<code>system_cpu_percent : 14</code>
	<code>idle_cpu_percent : 54</code>
	<code>idle_true_percent : 55</code>
	<code>wait_true_percent : 0</code>

FIGURE 4.7 – Données de surveillance SAP

Ces données sont essentielles pour analyser les performances du système et identifier les problèmes potentiels.

4.3 Qualité des données

L'évaluation de la qualité des données est une étape centrale de la phase *Data Understanding* dans la méthodologie CRISP-DM. Elle conditionne directement la pertinence des réponses produites par le système RAG et la fiabilité de la détection d'anomalies. Cette section présente une analyse structurée selon quatre axes : volumétrie, valeurs manquantes, doublons et bruit textuel, ainsi qu'une observation du déséquilibre thématique du corpus.

4.3.1 Volumétrie du corpus

Le tableau 4.1 résume les volumes de données collectés par source, avant et après nettoyage.

Source	Volume brut	Volume après nettoyage	Taux de rejet
Livres PDF SAP	297 documents / ~118 400 segments JSON	~110 200 segments	~7 %
Portail SAP Help	~28 500 pages crawlées	~22 800 pages retenues	~20 %
Surveillance PyRFC	~5 200 enregistrements	~5 150 enregistrements	< 1 %

TABLE 4.1 – Volumétrie du corpus par source

La longueur moyenne d'un segment textuel après découpage (*chunking*) est de ~312 tokens (écart-type : ~143 tokens). Les segments inférieurs à 50 tokens ont été systématiquement écartés, car trop courts pour constituer une unité sémantique exploitable par le modèle d'embeddings.

4.3.2 Valeurs manquantes

Les champs obligatoires pour l'ingestion dans le système RAG sont `doc_id`, `title`, `content` et `source`. Une vérification systématique a révélé que ~11 % des segments extraits des livres PDF présentaient un champ `title` vide, en raison de l'absence de titre explicite dans certains blocs de corps (*body*). Ces cas ont été traités en attribuant automatiquement le titre de la section parente. Aucune valeur manquante n'a été détectée sur le champ `content`, qui est le champ le plus critique pour la vectorisation.

Pour les données de surveillance SAP, tous les attributs sont renseignés directement par PyRFC ; aucune valeur manquante n'a été observée.

4.3.3 Doublons et redondances

Deux catégories de doublons ont été identifiées :

- **Doublons exacts** : segments dont le contenu textuel est identique après normalisation (casse, espaces, caractères spéciaux). Pour les pages SAP Help, ~18 % des pages collectées étaient des doublons exacts, dus à des liens redondants dans l'arborescence du portail ; elles ont été éliminées par comparaison de hachage MD5 sur le contenu brut.
- **Doublons partiels** : sections d'introduction reproduites à l'identique dans plusieurs livres PDF (chapitres d'entrée en matière communs à plusieurs certifications). Ces redondances représentent ~7 % du corpus PDF ; elles ont été détectées par similarité cosinus sur les embeddings et dédupliquées avant ingestion.

Au total, ~15 800 segments ont été supprimés pour cause de duplication, soit ~11,8 % du corpus brut.

4.3.4 Bruit textuel

L'extraction automatique a introduit plusieurs catégories de bruit :

- **Artefacts typographiques** : caractères de remplacement (???), tirets erronés et numéros de page intégrés dans le corps du texte des PDF.
- **Balises HTML résiduelles** : fragments de balises non nettoyées (<div>,) présents dans certaines pages du portail SAP Help après le parsing.
- **Fragments de code hors contexte** : extraits ABAP isolés, sans le texte explicatif environnant, générant des segments peu utiles à la recherche sémantique.

Un pipeline de nettoyage fondé sur des expressions régulières et des règles de filtrage a traité ces artefacts. Au total, ~34 % des segments ont subi une correction et ~42 % ont été écartés pour bruit excessif.

Conclusion

Ce chapitre aborde la phase d'analyse des données dans la méthodologie CRISP-DM. Cette étape vise à identifier, évaluer et décrire les diverses sources de données utilisées dans le projet afin de préparer au mieux les étapes suivantes, notamment la préparation et la modélisation des données.

Chapitre 5

Préparation des données

Introduction

Ce chapitre décrit les étapes de préparation des données pour le pipeline RAG. Les documents SAP collectés ont été nettoyés, structurés et transformés en embeddings pour alimenter le moteur de recherche sémantique.

5.1 Vue d'ensemble du pipeline de préparation

Le pipeline de préparation des données se compose de cinq étapes. Il démarre avec les données brutes et se termine par l'intégration finale dans la base vectorielle. Chaque étape utilise les résultats de la précédente pour produire des données intermédiaires bien définies. Cela assure un processus de transformation structuré, traçable et reproductible.



FIGURE 5.1 – Pipeline complet de préparation des données

5.2 Nettoyage, normalisation et déduplication

Cette section couvre les trois premières étapes du pipeline de préparation. Dans un premier temps, chaque source est nettoyée indépendamment selon ses anomalies propres (§ 5.2.1 et § 5.2.2). Les fichiers sont ensuite fusionnés et soumis à une normalisation textuelle unifiée (§ 5.2.3). Enfin, les doublons inter-sources et le bruit sémantique résiduel sont supprimés sur le corpus fusionné (§ 5.2.4).

5.2.1 Nettoyage des données du portail SAP Help

Les pages extraites du portail `help.sap.com` présentent des anomalies spécifiques aux données issues du web. Les scripts appliquent donc un traitement dédié à ces fichiers sources de manière indépendante avant toute étape de fusion.

Problèmes identifiés

- **HTML** **Balises et entités HTML résiduelles** : malgré le traitement initial, certaines pages conservent des fragments tels que `<b>`, ` ` ou `&` dans le texte extrait.
- **NAVIGATION** **Éléments de navigation** : les pages incluent des phrases standards comme « *Accepter les cookies* », « *Évaluez cette page* », « *Voir aussi* », qui n'offrent aucune information substantielle.
- **TOKENS** **Mots concaténés** : L'extraction HTML peut parfois générer des tokens combinés tels que `contentadministration` au lieu de `content administration`.
- **DOUBLONS** **Doublons par URL** : Une page donnée peut avoir été récupérée au cours de plusieurs sessions de scraping ; l'URL agit comme clé de déduplication principale.
- **QUALITÉ** **Pages insuffisamment longues** : certaines entrées se réfèrent à des pages de navigation vides ou comportant moins de 80 caractères, dépourvues de valeur informative.

Traitement appliqué Pour chaque fichier JSON issu du scraping, le script identifie les doublons en utilisant l'URL comme clé d'unicité, puis écarte les entrées dont la longueur est inférieure au seuil minimal (`min_chars = 80`).

La fonction `normalize_text` assure ensuite la normalisation du contenu textuel : elle supprime les balises HTML, les caractères invisibles, les sections non informatives de type « *for more information* », ainsi que les éléments de navigation bruités.

À l'issue du nettoyage, des métadonnées sont extraites automatiquement — T-codes,

SAP Notes, module SAP et fil d'Ariane — et associées à chaque document. Le résultat est sauvegardé dans un fichier `.cleaned.json` propre à la source.

5.2.2 Nettoyage des données des livres techniques PDF

Les livres SAP en format PDF sont soumis à une extraction structurée préalable, générant des fichiers `structured_content.json` pour chaque livre. Le fichier `clean_scraped_content.py` gère ensuite ces documents sans tenir compte du portail SAP Help.

Problèmes identifiés

- **CID** **Caractères CID** : lorsqu'une police n'est pas embarquée dans le PDF, `pdfplumber` ne peut pas décoder les glyphes et produit des séquences du type `(cid:123)`, sans aucune valeur sémantique.
- **TOC** **Lignes de table des matières** : les tables des matières produisent des lignes du format `Chapitre 3 . . . 42` qui viennent perturber les segments contenant des séries de points.
- **FIGURES** **Légendes et textes de figures** : les légendes précises extraites des figures (« *Figure 3.2 — overview* ») ne représentent pas un contenu documentaire exploitable.
- **DOUBLONS** **Contenu répété à la fin du document** : quelques PDFs contiennent des sommaires ou des synthèses qui reprennent textuellement des parties précédentes.

Traitement appliqué : Le script reconstitue le contenu brut en concaténant le titre, le contenu principal, le fil d'Ariane (*breadcrumb*) et les sections imbriquées de chaque entrée JSON. La fonction `remove_duplicate_content` détecte les répétitions en comparant la première et la seconde moitié du texte : si leur chevauchement lexical dépasse 80 %, seule la première moitié est conservée. Les entrées dont la longueur après nettoyage reste inférieure à 80 caractères sont éliminées. Chaque document retenu est enrichi des mêmes métadonnées que pour les données web (T-codes, modules, hiérarchie), puis sauvegardé dans `scraped_content.cleaned.json`.

5.2.3 Normalisation commune des deux sources

Une fois nettoyées *séparément*, les données des deux sources sont **fusionnées** dans un corpus unique, puis soumises à un pipeline de normalisation textuelle unifié implémenté dans `text_utils.py`. Contrairement au nettoyage source-spécifique des sections précédentes — qui traitait les anomalies propres à chaque format (balises HTML

pour le web, caractères CID pour les PDF) — cette étape applique des transformations *identiques* à tous les documents, indépendamment de leur origine, afin de garantir un schéma de sortie homogène.

TABLE 5.1 – Les opérations de normalisation appliquées à chaque document

N°	Opération	Description	Exemple
1	Décodage HTML	Convertit les entités HTML en caractères réels	 → espace
2	Suppression bruit	Élimine les motifs de navigation (« <i>See also</i> », « <i>Rate this page</i> », etc.)	supprimés
3	Segmentation lexicale	Redécoupe les mots concaténés issus de l'extraction web ou PDF	contentadmin → content admin
4	Normalisation T-codes	Fusionne les codes transactionnels mal formatés	"SM 37" → "SM37"
5	Normalisation Unicode	Applique NFC, supprime les caractères invisibles, met en minuscules	(cid:34) → espace

Expansion sémantique des termes SAP Les modèles d'embedding n'ont pas été entraînés sur de la documentation SAP et ne connaissent pas la signification des T-codes comme SM37 ou PFCG. Pour compenser ce manque, chaque document contenant des T-codes reconnus reçoit une extension textuelle à la fin. Cette extension associe chaque T-code à sa description en anglais courant.

¹ # Texte original

```
2 "La transaction SM37 permet de surveiller les jobs en arriere
   -plan."
3
4 # Texte apres expansion
5 "La transaction SM37 permet de surveiller les jobs en arriere
   -plan.
6 [SAP context: SM37=job monitor background job overview
   scheduled
7 jobs batch processing]"
```

Listing 5.1 – Exemple d’expansion sémantique pour SM37

Cette technique permet à un utilisateur posant la question « *comment vérifier mes jobs de nuit ?* » de retrouver les documents qui mentionnent SM37, même si sa question ne contient pas le code lui-même.

5.2.4 Suppression des doublons et du bruit

5.2.4.1 Stratégie de déduplication en deux passes

Le processus de suppression des doublons se fait après la fusion des différentes sources de données pour éviter d’avoir des documents répétés dans la base de connaissances. Cette étape se déroule en deux phases.

La première phase consiste à identifier les documents web identiques en utilisant le titre et l’URL de chaque page. Cette méthode aide à repérer les pages du portail SAP qui ont été récupérées plusieurs fois lors de différentes sessions de collecte.

La seconde phase s’appuie sur l’examen du contenu des documents. Une empreinte numérique est créée à partir du texte pour repérer les contenus similaires ou totalement identiques, même s’ils viennent de sources différentes. Cette méthode aide à identifier les passages du portail SAP qui apparaissent aussi dans certains supports de formation PDF.

La figure 5.2 illustre l’algorithme de déduplication en deux passes appliqué après la normalisation du corpus.

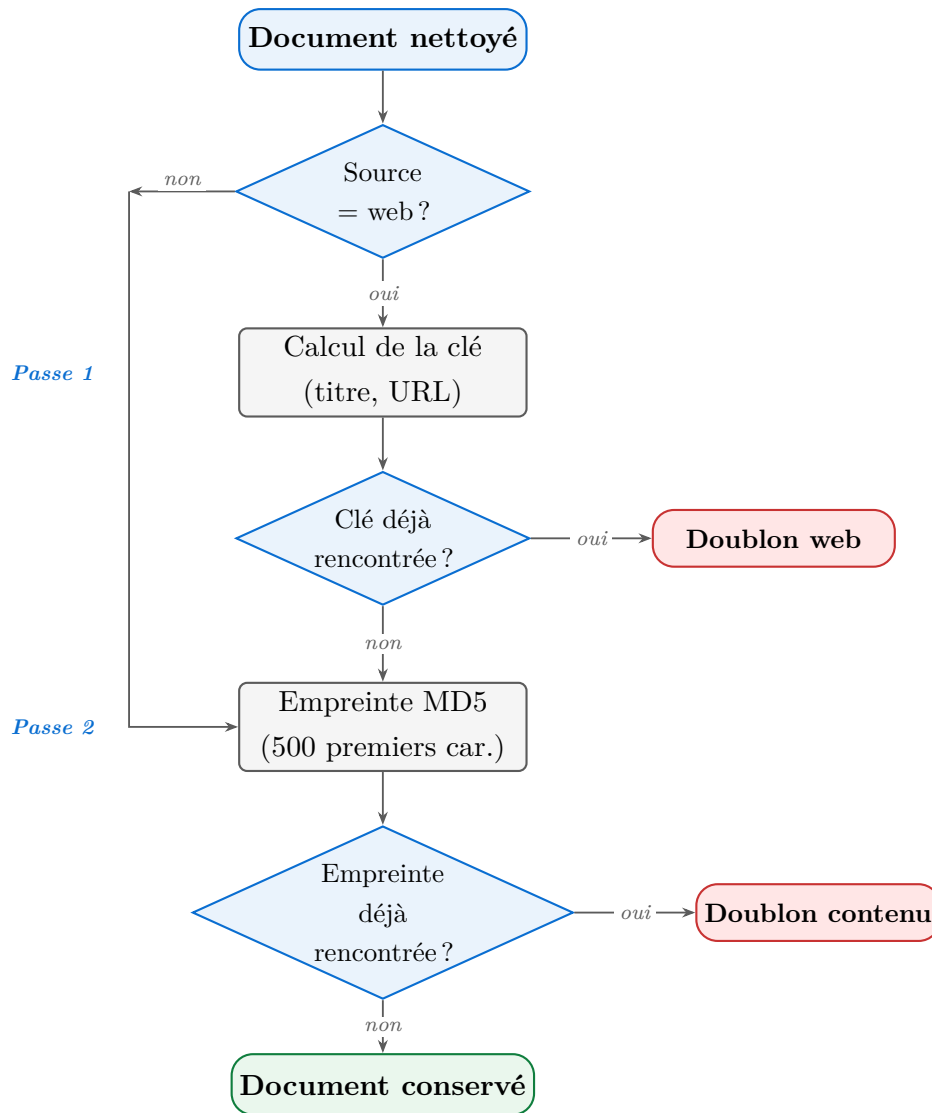


FIGURE 5.2 – Algorithme de déduplication en deux passes appliqué après l’étape de normalisation.

La première passe, restreinte aux documents issus du *scraping* web, élimine les doublons stricts identifiés par le couple (titre, URL). La seconde passe, appliquée à l’ensemble du corpus, détecte les contenus quasi-identiques au moyen d’une empreinte MD5 calculée sur les 500 premiers caractères, indépendamment de la source du document.

5.2.4.2 Filtrage du bruit sémantique

Certains documents sont techniquement valides sans doublons et avec un texte soigné mais ne sont pas utiles pour un système RAG de support technique. C’est notamment le cas des pages de quiz et d’auto-évaluation sur le portail SAP Help, qui contiennent des questions à choix multiples telles que « *Déterminez si cette affirmation est vraie ou fausse* ». Ces pages nuisent à la base de connaissances, car le moteur de recherche pourrait les afficher en réponse à des requêtes légitimes, fournissant ainsi des réponses de type examen plutôt que des réponses techniques. Un filtre basé sur des expressions régulières permet d’identifier et d’éliminer ces pages en repérant des motifs

tels que `learning assessment, determine whether this statement` ou `choose the correct answer` dans le titre ou dans les 200 premiers caractères du contenu.

TABLE 5.2 – Filtres de qualité appliqués après fusion des sources

Type de filtre	Méthode	Portée
Doublon exact (web)	Empreinte (titre + URL)	Documents web uniquement
Doublon contenu	MD5 des 500 premiers caractères	Toutes sources
Pages quiz	Regex sur titre et début de contenu	Toutes sources
Contenu trop court	Longueur < 80 caractères	Toutes sources

5.3 Découpage des documents (Chunking)

5.3.1 Qu'est-ce que le découpage

Le découpage consiste à découper de grands ensembles de texte ou de données en segments plus petits et plus faciles à gérer, appelés « chunks ». Cette technique est utile lorsqu'on travaille avec des modèles d'apprentissage automatique qui ont des limites de taille d'entrée. Elle est également efficace pour traiter de grands volumes de données transactionnelles, où chaque entrée doit être analysée séparément tout en contribuant à des analyses globales.

5.3.2 Pourquoi découper les documents ?

Le modèle d'embedding peut traiter un maximum de 512 tokens en entrée. Toutefois, la documentation SAP contient souvent des textes bien plus longs que cette limite, atteignant parfois plusieurs milliers de tokens. Dans ce cas, soumettre un document complet au modèle entraînerait une troncature automatique, ce qui provoquerait une perte d'informations, en particulier dans les sections finales du document.

D'un autre côté, une segmentation simpliste ou non régulée des textes pourrait affecter la continuité sémantique des informations et réduire la qualité des représentations vectorielles générées. Il est donc nécessaire que le processus de découpage remplisse deux buts : garantir que les morceaux sont compatibles avec la fenêtre de contexte du modèle et préserver la cohérence sémantique indispensable pour une représentation exacte du contenu.

5.3.3 Stratégie hiérarchique parent-enfant

L'application de la base vectorielle s'appuie sur une méthode de découpage hiérarchique qui garantit la conservation de la structure des documents tout en améliorant l'efficacité de recherche et de récupération d'information. Cette approche a été spécifiquement élaborée pour gérer la complexité et l'imbrication de la documentation SAP.

- **Extraction sensible à la structure et au contenu :** Le système applique une méthode d'extraction basée sur la structure pour maintenir l'organisation logique et sémantique des documents SAP. Cette méthode permet de garder la hiérarchie des sections et les liens entre les divers composants du document, ce qui améliore la qualité de l'indexation et de la recherche d'informations.
- **Extraction basée sur la mise en page :** Le processus d'extraction prend en compte la structure visuelle et organisationnelle des documents à travers plusieurs mécanismes :
 - Conservation de la hiérarchie des titres (H1, H2, H3, etc.) ;
 - Préservation des relations parent-enfant entre les sections et sous-sections ;
 - Identification et traitement des tableaux sous une représentation structurée de type HTML ;
 - Reconnaissance des listes et maintien de leur niveau d'imbrication.
- **Traitement enrichi par les métadonnées :** Le système extrait et conserve différentes catégories de métadonnées afin d'améliorer l'organisation documentaire et la précision de la récupération :
 - Métadonnées générales du document (titre, date de création, version) ;
 - Métadonnées structurelles liées à l'organisation du contenu ;
 - Métadonnées techniques telles que les associations aux modules SAP ou les références aux codes de transaction.
- **Traitement multi-niveaux du contenu :** Le contenu documentaire est analysé à plusieurs niveaux afin de capturer à la fois la structure globale du document et les caractéristiques spécifiques de chaque élément.
- **Analyse au niveau des blocs :** Cette étape comprend :
 - La détection automatique des titres à partir de leur position, formatage et propriétés textuelles ;
 - La classification des types de contenu afin de distinguer les paragraphes, listes, tableaux et images ;
 - Une analyse spatiale exploitant les coordonnées des éléments pour reconstruire la disposition du document.

- **Traitement spécifique selon le type de contenu** : Chaque type de contenu bénéficie d'un traitement adapté :
 - Les tableaux complexes sont convertis vers un format HTML structuré afin de préserver leur organisation ;
 - Les éléments de listes associés sont regroupés dans des fragments cohérents ;
 - Les images font l'objet d'un processus d'extraction sémantique basé sur le modèle `microsoft/Florence-2-large`. Ce choix a été retenu après une évaluation comparative avec `GLM-OCR` ainsi qu'une approche reposant uniquement sur `Tesseract OCR`. Contrairement à une simple reconnaissance optique de caractères, le modèle `Florence-2-large` permet de mieux préserver le contexte et la signification des informations visuelles présentes dans les schémas, captures d'écran et illustrations techniques de la documentation SAP ;
 - Les légendes et descriptions associées aux images sont reliées automatiquement au contenu visuel correspondant.

5.3.4 Algorithme de découpage et chevauchement

L'algorithme de découpage interne cherche toujours à opérer aux frontières les plus naturelles du texte. Il tente d'abord de sectionner entre les **paragraphes** (délimités par deux sauts de ligne), ensuite entre les **phrases** (séparées par « . », « ! » ou « ? »), et finalement entre les **mots** en dernier recours.

Pour éviter toute perte de contexte aux frontières entre deux fragments successifs, chaque nouveau fragment débute avec les **50 derniers tokens** du fragment précédent. Ce chevauchement (*overlap*) d'environ 10 % assure que les phrases qui traversent une limite de découpage restent compréhensibles dans les deux fragments.

TABLE 5.3 – Niveaux de découpage par ordre de priorité

Priorité	Niveau	Déclencheur	Séparateur
1	Paragraphe	Paragraphe seul > 512 tokens	\n\n
2	Phrase	Phrase seule > 512 tokens	. ! ?
3	Mot	Phrase extrêmement longue	espace

5.3.5 Résultats du découpage

Après application du chunker sur l'ensemble du corpus normalisé, les statistiques obtenues sont les suivantes :

TABLE 5.4 – Statistiques de découpage du corpus SAP

Type d'enregistrement	Nombre	Description
Singles	73 137	Documents courts, un seul enregistrement, vectorisé
Parents	8 795	Documents longs, texte complet, stocké, non vectorisé
Enfants	32 194	Fragments de documents longs, vectorisés et indexés
Total enregistrements	114 126	Tous types confondus
<i>dont vectorisés (Singles + Enfants)</i>	<i>105 331</i>	<i>Éligibles à l'embedding</i>

Les 8 795 enregistrements de type *Parent* sont conservés en base pour l'enrichissement de contexte lors de la récupération, mais ne portent pas de vecteur propre. Après application des filtres finaux de longueur et de qualité lors de l'ingestion dans PostgreSQL, **76 463 vecteurs** sont effectivement indexés — ce chiffre constitue la taille opérationnelle de la base vectorielle utilisée dans toutes les évaluations ultérieures du rapport.

5.4 Transformation des données en embeddings vectoriels

5.4.1 Choix du modèle d'embedding

Le modèle retenu pour la génération des vecteurs est **BAAI/bge-large-en-v1.5**, un modèle de la famille BGE (*BAAI General Embedding*) publié par l'Institut de l'Intelligence Artificielle de Pékin. Ce modèle produit des vecteurs denses de 1 024 dimensions et a été entraîné pour la recherche sémantique dense en anglais, ce qui correspond à la langue principale de la documentation SAP technique.

TABLE 5.5 – Comparaison des modèles d’embedding évalués

Modèle	Dim.	Langue	Points forts	Limitation
bge-large-en-v1.5	1 024	Anglais	Très performant sur benchmarks BEIR	Monolingue
bge-m3	1 024	Multilingue	Supporte 100+ langues	Légèrement moins précis en anglais
all-MiniLM-L6-v2	384	Anglais	Rapide et léger	Moins précis
text-embedding-ada-002	1 536	Multilingue	API OpenAI, très utilisé	Payant, dépendance externe

Dim. = Dimensionnalité du vecteur d’embedding produit par le modèle.

Le modèle `bge-large-en-v1.5` a été retenu pour ses performances supérieures sur des tâches de recherche documentaire technique, avec une fenêtre de 512 tokens alignée exactement sur la taille maximale de nos chunks.

5.4.2 Processus de génération des vecteurs

Le script `embedder.py` prend en entrée le fichier `chunked_for_embedding.json` et produit deux fichiers indexés de manière alignée : `embeddings.npy` (tableau NumPy de forme $(N, 1024)$) et `metadata.json` (liste de N dictionnaires). L’alignement par indice garantit que le vecteur `embeddings[i]` correspond exactement au document `metadata[i]`.

Le modèle BGE requiert un préfixe spécifique ajouté à chaque texte avant l’encodage pour signaler qu’il s’agit d’un document (et non d’une requête) :

Les vecteurs produits sont normalisés en L2, ce qui signifie que la similarité cosinus entre deux vecteurs est équivalente à leur produit scalaire. Cette propriété accélère significativement la recherche dans pgvector.

5.5 Structuration des métadonnées

5.5.1 Rôle des métadonnées dans un système RAG

Dans un système RAG, les métadonnées jouent un rôle aussi important que les vecteurs eux-mêmes. Elles permettent d’appliquer des filtres *avant* la recherche vec-

torielle, réduisant l'espace de recherche et améliorant la précision des résultats. Par exemple, une question portant sur la gestion des background jobs peut être filtrée sur le champ `module = "BASIS"` avant même de comparer les vecteurs, excluant d'emblée les documents d'autres modules SAP.

5.5.2 Schéma des métadonnées

Chaque enregistrement porte quatre catégories de métadonnées définies dans et transportées jusqu'en base de données .

TABLE 5.6 – Champs d'identification et de source

Champ	Type	Description
<code>doc_id</code>	string	Identifiant unique du document d'origine
<code>source_type</code>	string	"sap_help" ou "technical_book"
<code>source_book</code>	string	Nom du livre PDF source (si applicable)
<code>title</code>	string	Titre de la section ou du document
<code>module</code>	string	Module SAP concerné (BASIS, ABAP, FI, etc.)

TABLE 5.7 – Champs de structure hiérarchique et d'entités SAP

Champ	Type	Description
<code>heading_level</code>	entier	Niveau de titre (H1=1, H2=2, H3=3...)
<code>section_type</code>	string	Type de section (procedure, code, note, table)
<code>breadcrumb</code>	liste	Chemin hiérarchique dans le document
<code>mentioned_tcodes</code>	liste	T-codes cités dans le document, ex. ["SM37", "ST22"]
<code>mentioned_sap_notes</code>	liste	Numéros de SAP Notes référencées
<code>contains_code</code>	bool	Le document contient du code ABAP
<code>contains_table</code>	bool	Le document contient un tableau
<code>contains_procedure</code>	bool	Le document décrit une procédure étape par étape

TABLE 5.8 – Champs de liaison parent-enfant

Champ	Type	Description
<code>is_parent</code>	bool	True si enregistrement parent (non embedded)
<code>parent_chunk_id</code>	string	UUID du parent (vide si single ou parent)
<code>chunk_index</code>	entier	Position du fragment dans le document (−1 pour parent)
<code>chunk_total</code>	entier	Nombre total de fragments du document

5.5.3 Stockage et indexation dans PostgreSQL

Les chunks extraits sont persistés dans une base de données PostgreSQL enrichie de l’extension **pgvector**, qui ajoute un type de données natif pour les vecteurs denses et les opérateurs de similarité associés. L’ensemble des données est stocké dans une table unique **sap_chunks**, dont le schéma est défini comme suit :

Chaque enregistrement correspond à un chunk textuel identifié par un UUID, accompagné de son contenu brut, de ses métadonnées structurées et de son vecteur d’embedding de dimension 1024. La colonne **metadata** adopte le type **JSONB** (JSON Binaire), format natif de PostgreSQL permettant des requêtes conditionnelles efficaces sur des champs imbriqués arbitraires.

Afin d’assurer des performances de recherche adaptées à un usage en production, deux index complémentaires sont définis sur cette table :

- **Index HNSW** (*Hierarchical Navigable Small World*) : appliqué sur la colonne **embedding**, cet index permet la recherche approximative des plus proches voisins par similarité cosinus. Il est configuré avec les paramètres **m=16** et **ef_construction=64**, qui contrôlent respectivement le nombre de liens par nœud dans le graphe et la précision du processus de construction.
- **Index GIN** (*Generalized Inverted Index*) : appliqué sur la colonne **metadata**, il accélère les filtres JSONB du type **metadata @> '{"module": "BASIS"}'**, permettant de restreindre l’espace de recherche vectorielle avant l’exécution de la requête HNSW.

L’articulation de ces deux mécanismes d’indexation constitue le cœur de la stratégie de recherche hybride adoptée dans ce travail. Comme l’illustre la figure 5.3, la requête utilisateur est d’abord filtrée par les métadonnées via l’index GIN, réduisant l’espace candidat avant la recherche vectorielle approximative effectuée par l’index HNSW. Les chunks retenus sont ensuite enrichis par remontée au chunk parent le cas échéant, avant d’être transmis au modèle de langage comme contexte de génération.

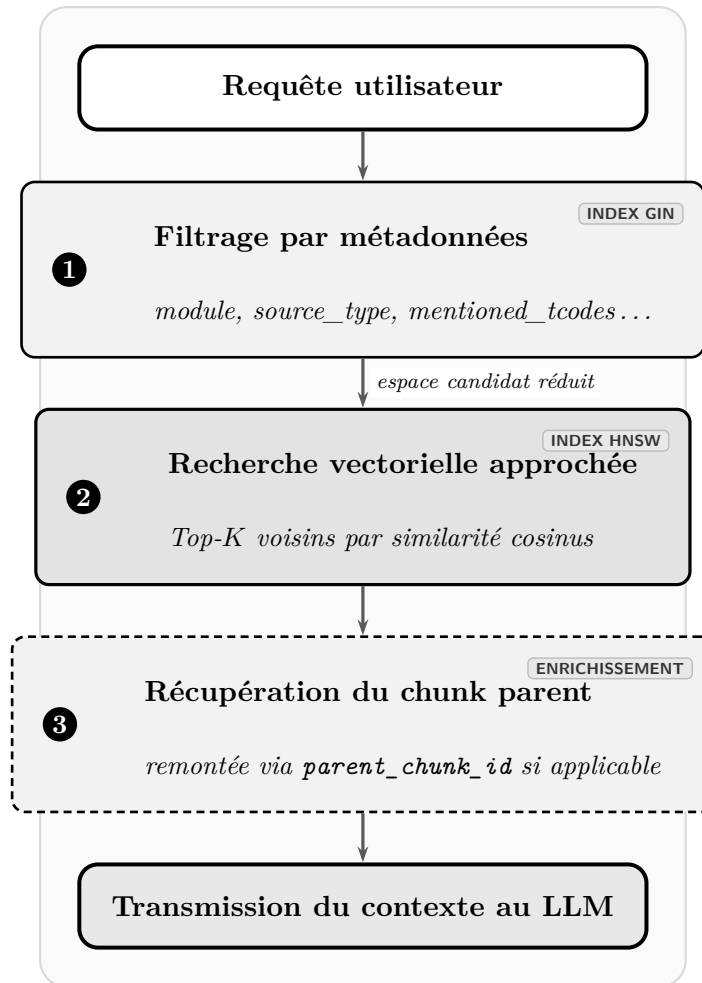


FIGURE 5.3 – Pipeline de recherche hybride exploitant les index GIN et HNSW lors d’une requête RAG

Conclusion

Ce chapitre a décrit l’ensemble du pipeline de préparation des données, depuis les sources brutes jusqu’à la base vectorielle opérationnelle. Les documents SAP ont été nettoyés, dédoublés et normalisés à travers un processus rigoureux adapté à chaque source. Le découpage hiérarchique parent-enfant a permis de produire **76 463 vecteurs** indexés, générés par le modèle **BAAI/bge-large-en-v1.5** et enrichis de métadonnées structurées. Stockés dans PostgreSQL avec pgvector, ces données sont indexées via les mécanismes HNSW et GIN, constituant ainsi le socle de la recherche hybride du système RAG. Le chapitre suivant présente la phase de modélisation, qui exploite cette base pour construire le pipeline de génération augmentée par récupération.

Chapitre 6

Modélisation

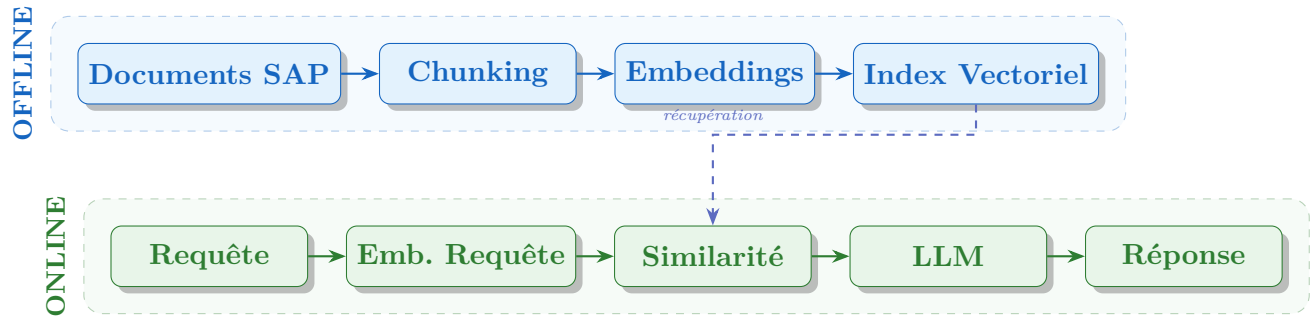
Introduction

La phase de modélisation constitue l'étape clé de la méthodologie CRISP-DM, où les données préparées sont utilisées pour créer des solutions répondant aux objectifs commerciaux. Dans le contexte de ce projet, cette étape implique la mise en place d'un système de Retrieval-Augmented Generation (RAG) spécifiquement destiné au secteur SAP, afin d'offrir des réponses précises, fiables et contextualisées aux questions techniques.

6.1 Architecture globale du système RAG

6.1.1 Vue d'ensemble

Le système **RAG** (*Retrieval-Augmented Generation*) conçu s'articule autour de deux étapes majeures et complémentaires : une **étape *offline***, dédiée à l'ingestion et à l'indexation des données, et une **étape *online***, responsable du traitement dynamique des requêtes utilisateurs. L'étape *offline* convertit les documents SAP bruts en une base vectorielle interrogeable ; l'étape *online* gère la requête d'un utilisateur en l'analysant sur le plan sémantique, en extrayant les passages les plus pertinents et en produisant une réponse structurée à l'aide d'un LLM.



6.1.2 Flux de données

Lorsqu'un utilisateur pose une question dans le chatbot SAP, celle-ci traverse plusieurs étapes avant qu'une réponse lui soit renvoyée. Ce flux peut se décomposer en six grandes phases.

1. Compréhension de la question La question brute est d'abord analysée par un module NLP (*Natural Language Processing*) qui identifie l'intention de l'utilisateur (cherche-t-il à résoudre un problème, à comprendre un concept, à suivre une procédure?), extrait les entités SAP présentes (T-codes, numéros de notes), et reformule la question de manière enrichie pour améliorer la recherche.

2. Recherche hybride dans la base documentaire La question enrichie est convertie en vecteur numérique, puis soumise simultanément à deux types de recherche complémentaires : une **recherche sémantique** (qui compare le sens de la question avec celui des documents) et une **recherche par mots-clés** (qui repère les documents contenant exactement les termes SAP de la question). Cette double recherche permet de ne manquer ni les documents conceptuellement proches ni ceux qui contiennent des identifiants exacts.

3. Fusion et classement des résultats Les résultats des deux recherches sont fusionnés par un algorithme appelé **RRF** (*Reciprocal Rank Fusion*), qui combine les classements en favorisant les documents bien placés dans les deux listes à la fois. Un second modèle, le **cross-encoder**, affine ensuite ce classement en lisant conjointement la question et chaque document candidat pour produire un score de pertinence plus précis.

4. Construction de la réponse. Les documents les mieux classés sont injectés dans un prompt structuré transmis au modèle de langage (LLM), qui génère une réponse synthétique en citant ses sources.

5. Contrôle anti-hallucination Avant d’être renvoyée à l’utilisateur, la réponse est vérifiée automatiquement : tout T-code ou numéro de note mentionné dans la réponse mais absent des documents sources est détecté et supprimé.

6.2 Pipeline RAG : Description détaillée

6.2.1 Vue séquentielle du pipeline

Le pipeline RAG, implémenté principalement dans `rag_pipeline.py`, orchestre l’ensemble du traitement d’une requête utilisateur à travers la fonction centrale `ask()`. Celle-ci coordonne successivement : la compréhension de la requête, l’enrichissement sémantique, la recherche hybride, la fusion RRF, le reranking par cross-encoder, la construction du prompt et la génération par le LLM, avant de valider la réponse par un contrôle anti-hallucination.

6.2.2 Normalisation de la requête

La première opération appliquée à toute requête entrante est une normalisation SAP-spécifique réalisée par la fonction `normalize_query()`. Cette étape corrige les espaces erronés dans les codes de transaction (ex. "SM 37" → "SM37"), et rétablit la casse correcte des termes SAP ("abap" → "ABAP", "idoc" → "IDoc"). Cette harmonisation est essentielle pour garantir que les entités extraites correspondront exactement aux valeurs indexées dans la base de données.

6.2.3 Compréhension sémantique de la requête (NLP)

La fonction `understand_query()` constitue l’un des éléments clés du pipeline RAG. Cette fonction s’appuie sur une méthode totalement *rule-based* et ne fait appel à aucun modèle de langage (LLM). L’analyse est ainsi entièrement déterministe, rapide et stable, avec un temps d’exécution inférieur à une milliseconde.

Sa principale mission est de convertir une requête utilisateur brute en une forme structurée et enrichie, qui peut être directement utilisée par le moteur de recherche hybride du système.

La fonction renvoie un objet qui inclut plusieurs attributs essentiels :

- **intent** : intention détectée parmi `lookup`, `how-to`, `troubleshoot` ou `explain`.
- **clean_query** : version nettoyée de la requête, sans mots inutiles ni expressions de remplissage (*filler words*).
- **expanded_query** : requête enrichie par des termes SAP associés afin d’améliorer le rappel documentaire.

- **tcodes** : liste des codes de transaction SAP détectés dans la requête.
- **sap_notes** : liste des numéros de notes SAP OSS éventuellement identifiés.
- **module** : module SAP détecté (BASIS, ABAP, MM, SD, FI, SECURITY, etc.).
- **min_score** : seuil minimal de similarité cosinus utilisé lors de la recherche vectorielle. Ce seuil est adaptatif : il est fixé à 0.05 lorsqu'un T-code est détecté dans une requête de type **lookup**, et à 0.30 pour les autres cas.

Détection de l'intention. La détection de l'intention repose sur un ensemble de règles linguistiques appliquées par ordre de priorité : la première règle qui correspond détermine immédiatement la catégorie. Quatre catégories ont été définies en fonction des besoins réels des utilisateurs SAP :

- **troubleshoot** : l'utilisateur signale une erreur ou un dysfonctionnement et cherche à le résoudre ;
- **how-to** : l'utilisateur demande comment effectuer une procédure ou une action ;
- **lookup** : l'utilisateur cherche la définition ou le rôle d'un élément précis ; cette catégorie est également assignée automatiquement lorsqu'un T-code est présent dans la question, car une question contenant un T-code est presque toujours une demande d'information ciblée ;
- **explain** : catégorie par défaut pour toute question conceptuelle ne correspondant pas aux trois cas précédents.

Identification du module SAP. En parallèle de l'intention, le système identifie le module SAP concerné (BASIS, ABAP, SECURITY, etc.) à partir des mots-clés présents dans la question ou du T-code détecté. Cette information est utilisée comme filtre lors de la recherche documentaire, afin de limiter les résultats aux documents du module pertinent et d'améliorer la précision des réponses.

Continuité conversationnelle. Dans un échange multi-tours, l'utilisateur peut poser une question de suivi sans répéter le sujet : par exemple, après avoir demandé des informations sur **SM37**, il peut enchaîner avec « Qu'est-ce qu'il fait ? ». Le système résout automatiquement ce type de pronom en réinjectant le dernier T-code mentionné dans l'historique de conversation, maintenant ainsi la cohérence entre les tours.

Cette étape améliore fortement la qualité des embeddings pour les transactions SAP spécialisées. Par exemple, le T-code **SMQ1** peut être enrichi par des termes tels que :

- qRFC outbound queue ;
- asynchronous communication ;
- queue monitoring ;

- outbound scheduler ;
- RFC queue administration.

Grâce à cet enrichissement sémantique, le moteur de recherche vectorielle peut retrouver des documents pertinents même lorsque les contenus documentaires ne contiennent pas explicitement le T-code recherché.

6.2.4 Vectorisation de la requête

Le modèle BAAI BAAI/bge-large-en-v1.5 transforme la requête améliorée en un vecteur dense, étant chargé de manière paresseuse via la fonction `get_model()` dans un cadre de singleton. La bibliothèque SentenceTransformers assure que l'encodage inclut une normalisation L2. Selon les recommandations du modèle BGE pour la recherche asymétrique (requête brève contre passages longs), un préfixe spécifique "**Represent this sentence for searching relevant passages:** " est toujours ajouté avant la requête. La taille du vecteur produit est de 1024.

Ce modèle a été sélectionné pour garantir la cohérence entre les embeddings créés lors de l'interrogation et ceux qui sont déjà présents dans la base vectorielle, lesquels ont également été générés avec le modèle BAAI/bge-large-en-v1.5. Cette uniformité assure la compatibilité des représentations vectorielles et renforce la pertinence de la recherche sémantique.

6.2.5 Recherche hybride parallèle

La fonction `_do_search()` orchestre trois recherches simultanées via un `ThreadPoolExecutor` à trois workers :

1. **Recherche vectorielle** : requête SQL utilisant l'opérateur de distance cosinus `<=>` de pgvector, avec filtrage par `min_score`. Les 15 meilleurs candidats (`FETCH_K=15`) sont récupérés.
2. **BM25 sur expanded_query** : recherche plein texte via `plainto_tsquery('english', ...)` et `ts_rank()` sur la colonne `text_search` (tsvector précalculé).
3. **BM25 sur clean_query** : même recherche mais sur la requête nettoyée, pour capturer les documents correspondant exactement aux termes SAP.

Les résultats BM25 des deux variantes sont fusionnés par score maximum (`max`) — et non par somme — pour éviter la surpondération artificielle d'un document présent dans les deux listes .

6.2.6 Fusion par Reciprocal Rank Fusion (RRF)

La recherche hybride produit deux listes de résultats indépendantes : une issue de la recherche vectorielle, l'autre de la recherche BM25. Le problème est de les combiner intelligemment : un document peut être en position 3 dans la première liste et en position 1 dans la seconde. Quelle liste croire ?

L'algorithme **RRF** (*Reciprocal Rank Fusion*) résout ce problème en attribuant à chaque document un score basé sur sa *position* dans chaque liste, et non sur son score brut. Plus un document est bien classé dans *plusieurs* listes à la fois, plus son score RRF est élevé. Un document médiocre dans les deux listes ne peut pas compenser en étant excellent dans une seule.

Dans ce projet, la pondération est délibérément asymétrique : la recherche BM25 est favorisée ($W_{\text{BM25}} = 0.6$) par rapport à la recherche vectorielle ($W_{\text{vec}} = 0.4$). Ce choix est justifié par la nature du corpus SAP : les T-codes, noms de modules et numéros de notes sont des identifiants exacts que la recherche par mots-clés repère mieux que la similarité sémantique.

La fonction `rrf_fuse()` implémente cet algorithme selon la formule suivante :

$$\text{score_RRF}(d) = \frac{W_{\text{vec}}}{k + r_{\text{vec}}(d)} + \frac{W_{\text{BM25}}}{k + r_{\text{BM25}}(d)} \quad (6.1)$$

où $W_{\text{vec}} = 0.4$, $W_{\text{BM25}} = 0.6$, $k = 60$, et $r(d)$ désigne le rang du document d dans chaque liste. Le choix de favoriser BM25 ($W_{\text{BM25}} > W_{\text{vec}}$) est motivé par la nature terminologique du corpus SAP : les codes de transaction, les noms de modules et les numéros de notes sont des correspondances exactes que BM25 capture mieux que la sémantique vectorielle. Les 10 meilleurs candidats fusionnés (`RERANK_K=10`) sont transmis au cross-encoder.

6.2.7 Injection CAG (Context-Augmented Generation)

Pour certaines questions très ciblées, la recherche dynamique n'est pas toujours nécessaire. Lorsqu'un utilisateur demande directement des informations sur un T-code connu (par exemple `SM37` ou `ST22`), les documents les plus pertinents sont prévisibles et peuvent être pré-chargés depuis la base de données, plutôt que recalculés à chaque fois.

C'est le rôle du mécanisme **CAG** (*Context-Augmented Generation*) : pour les requêtes de type `lookup` portant sur un T-code identifié, des chunks spécifiques à ce T-code sont récupérés en base et injectés directement en tête du contexte, *avant* les résultats de la recherche hybride. Cela garantit que le contenu le plus autoritaire sur ce T-code est toujours visible par le LLM, même si la recherche dynamique l'aurait classé en dehors des cinq premiers résultats. Ces chunks sont mis en cache après le premier accès, de sorte que chaque T-code ne déclenche qu'une seule requête base de données,

quelle que soit la fréquence des questions posées.

Après la fusion RRF et avant le reranking, ce mécanisme injecte des chunks pré-chargés pour les requêtes de type `lookup` portant sur un T-code connu. La fonction `get_cag_context(tcode, conn_str)` interroge la base de données pour récupérer les chunks les plus ciblés sur ce T-code (triés par nombre croissant de T-codes mentionnés, pour favoriser les chunks les plus spécifiques) :

```
1 SELECT ... FROM sap_chunks
2 WHERE %s = ANY(mentioned_tcodes)
3     AND is_parent IS NOT TRUE
4 ORDER BY array_length(mentioned_tcodes, 1) ASC
5 LIMIT 6;
```

Listing 6.1 – Requête CAG : chunks les plus ciblés pour un T-code

Ces chunks sont mis en cache process-lifetime dans le dictionnaire `_CAG_DB_CACHE` (protégé par un verrou thread-safe), de sorte que chaque T-code ne déclenche qu'un seul accès DB quelle que soit la fréquence des requêtes. Lors de l'injection, les chunks CAG sont placés **en tête** du contexte (avant les chunks RAG dynamiques), garantissant que le contenu le plus autoritaire sur ce T-code est toujours visible par le LLM :

- **CAG_MAX_CHUNKS = 3** : nombre maximum de chunks pré-chargés injectés.
- Les chunks RAG dont l'identifiant est déjà présent dans les chunks CAG sont dédupliqués.
- Le total des chunks injectés au LLM reste plafonné à **TOP_K** (5 par défaut).

6.3 Modélisation des données

6.3.1 Structure de la table `sap_chunks`

La base de données PostgreSQL conserve la documentation SAP sous forme de chunks dans la table `sap_chunks`. Chaque enregistrement correspond à un fragment autonome de documentation, accompagné de métadonnées structurées permettant d'effectuer des recherches ciblées et un filtrage précis des résultats.

TABLE 6.1 – Colonnes de la table `sap_chunks`

Colonne	Type	Description
<code>chunk_id</code>	UUID	Identifiant unique du chunk
<code>text</code>	TEXT	Contenu textuel du chunk
<code>title</code>	TEXT	Titre de la section ou du document
<code>source_type</code>	VARCHAR	Type de source : <code>sap_help</code> ou <code>technical_book</code>
<code>source_book</code>	TEXT	Nom du livre ou de la documentation source
<code>module</code>	VARCHAR	Module SAP (BASIS, ABAP, MM, SD, FI, SECURITY)
<code>section_type</code>	VARCHAR	Type de section (chapter, section, sub-section)
<code>breadcrumb</code>	TEXT[]	Fil d'Ariane hiérarchique
<code>mentioned_tcodes</code>	TEXT[]	T-codes SAP mentionnés dans le chunk
<code>mentioned_sap_notes</code>	TEXT[]	Numéros des notes SAP mentionnées
<code>embedding</code>	VECTOR(1024)	Vecteur d'embedding dense (BGE-large)
<code>text_search</code>	TSVECTOR	Index de recherche plein texte (BM25)
<code>is_parent</code>	BOOLEAN	Indique si le chunk est un chunk parent
<code>parent_chunk_id</code>	UUID	Référence au chunk parent (si enfant)

6.3.2 Hiérarchie des chunks et stratégie Small-to-Big

Un problème classique dans les systèmes RAG est le suivant : pour bien retrouver un document, il vaut mieux des *petits* fragments très précis ; mais pour bien répondre, le LLM a besoin de *grands* blocs de texte avec suffisamment de contexte. Ces deux besoins sont contradictoires si l'on n'utilise qu'un seul niveau de découpage.

La stratégie **Small-to-Big Retrieval** résout ce problème avec une hiérarchie à deux niveaux :

- Les **chunks enfants** (petits, ciblés) sont utilisés pour la recherche : leur précision maximise la pertinence des résultats trouvés.

- Les **chunks parents** (plus larges, plus contextuels) sont transmis au LLM pour la génération : leur taille fournit suffisamment de contexte pour construire une réponse cohérente.

Les colonnes `is_parent` et `parent_chunk_id` dans la base de données matérialisent cette hiérarchie. Les requêtes de recherche filtrent systématiquement les chunks parents (`is_parent IS NOT TRUE`) pour ne travailler qu'avec les chunks feuilles lors du retrieval.

6.4 Techniques d'Embeddings

6.4.1 Modèle retenu : BAAI/bge-large-en-v1.5

Le modèle d'embedding retenu dans le cadre de ce projet est Beijing Academy of Artificial Intelligence BAAI/bge-large-en-v1.5. Développé selon une architecture bi-encodeur, ce modèle génère des représentations vectorielles denses de dimension 1024. Les embeddings produits sont normalisés selon la norme L2, permettant ainsi d'exploiter directement le produit scalaire pour calculer la similarité cosinus entre les vecteurs. Ce choix est motivé par plusieurs facteurs :

- **Performance sur les benchmarks de retrieval** : BGE-large-en-v1.5 figure parmi les meilleurs modèles sur le benchmark MTEB (*Massive Text Embedding Benchmark*), notamment pour les tâches de recherche d'information en anglais.
- **Optimisation asymétrique** : le modèle supporte l'ajout d'un préfixe spécifique pour les requêtes ("**Represent this sentence for searching relevant passages:** "), permettant une séparation sémantique entre la représentation d'une requête courte et celle d'un passage long.
- **Exécution locale** : le modèle peut être chargé localement via `SentenceTransformers`, sans dépendance à une API externe payante.
- **Richesse sémantique** : avec 1024 dimensions, le modèle capture des nuances sémantiques fines, essentielles pour distinguer des concepts SAP proches (ex. `SM37` vs `SM36`, `qRFC` vs `trFC`).

6.4.2 Méthode de vectorisation

L'embedding des chunks à l'indexation est réalisé sans préfixe (représentation du document), tandis que l'embedding des requêtes utilise le préfixe dédié. La fonction `embed_query()` encapsule cette logique :

```
1 EMBED_MODEL    = "BAAI/bge-large-en-v1.5"
2 QUERY_PREFIX   = "Represent this sentence for searching
   relevant passages: "
```

```

3
4 def embed_query(text: str) -> list[float]:
5     return get_model().encode(
6         QUERY_PREFIX + text,
7         normalize_embeddings=True,
8         convert_to_numpy=True
9     ).tolist()

```

Listing 6.2 – Fonction d’embedding de la requête

Le chargement du modèle est réalisé en singleton paresseux (*lazy singleton*) via `get_model()`, évitant ainsi de recharger le modèle en mémoire à chaque requête.

6.5 Base Vectorielle

6.5.1 PostgreSQL avec l’extension pgvector

Le système n’utilise pas un vector store dédié (tel que ChromaDB, Pinecone ou Weaviate), mais s’appuie sur **PostgreSQL** enrichi de l’extension **pgvector**. Ce choix architectural présente plusieurs avantages décisifs pour le contexte SAP :

- **Unification des données** : texte, métadonnées structurées (module, T-code, source) et vecteurs cohabitent dans la même table, permettant des filtres combinés en une seule requête SQL.
- **Recherche hybride native** : PostgreSQL supporte nativement les index TSVECTOR (BM25) et les index vectoriels HNSW via pgvector, éliminant le besoin d’orchestrer deux systèmes distincts.
- **ACID et fiabilité** : les garanties transactionnelles de PostgreSQL assurent la cohérence des données lors des mises à jour de la base de connaissances.

6.5.2 Indexation vectorielle HNSW

L’extension pgvector crée un index HNSW (*Hierarchical Navigable Small World*) sur la colonne `embedding`, permettant une recherche approximative du plus proche voisin (*Approximate Nearest Neighbor*) en temps sous-linéaire. La recherche vectorielle est exprimée via l’opérateur `<=>` qui calcule la distance cosinus :

```

1 SELECT chunk_id::text, text, title, source_type, source_book,
2        module, section_type, breadcrumb,
3        mentioned_tcodes, mentioned_sap_notes,
4        is_parent, parent_chunk_id::text,
5        1 - (embedding <=> '[0.012,...,0.034] '::vector) AS
        score

```



```
6 FROM sap_chunks
7 WHERE is_parent IS NOT TRUE
8     AND module = 'BASIS'
9 ORDER BY embedding <=> '[0.012,...,0.034] '::vector
10 LIMIT 15;
```

Listing 6.3 – Requête de recherche vectorielle avec pgvector

Le score de similarité cosinus est calculé comme $1 - \text{distance_cosinus}$, ce qui donne une valeur entre -1 et 1 , les chunks les plus proches du vecteur requête obtenant les scores les plus élevés.

6.6 Mécanisme de Retrieval

6.6.1 Recherche sémantique vectorielle

La recherche vectorielle représente le premier axe du retrieval hybride. Elle repose sur la mesure de similarité cosinus entre le vecteur de la requête et ceux des chunks indexés. L'opérateur pgvector `<=>` effectue cette évaluation de manière optimale grâce à l'index HNSW déjà construit. La fonction `vector_search()` élimine les résultats dont le score est inférieur au seuil `min_score`, qui oscille entre 0.05 (requêtes *lookup* avec T-code) et 0.30 (requêtes *explain* générales). Cette adaptation dynamique du seuil permet d'ajuster la précision du retrieval selon le type de requête : une question concernant un T-code spécifique doit permettre de récupérer tous les chunks faisant mention de ce code, même avec une similarité globale faible.

6.7 Mécanisme de Reranking par Cross-Encoder

6.7.1 Modèle cross-encoder ms-marco-MiniLM-L-6-v2

Le reranking constitue la troisième couche de filtrage du pipeline. Contrairement au bi-encodeur BGE qui encode requête et document séparément, le cross-encoder `cross-encoder/ms-marco-MiniLM-L-6-v2` analyse conjointement la paire (requête, chunk) pour produire un score de pertinence fine. Ce modèle, entraîné sur le dataset MS-MARCO de recherche de passages, produit des scores sur une échelle approximative de -10 à $+10$.

La fonction `rerank()` applique le cross-encoder sur toutes les paires (requête, chunk candidat) en une seule inférence vectorisée :

```

1 def rerank(query, chunks, top_k=TOP_K):
2     scores = get_reranker().predict(
3         [[query, c["content"]] for c in chunks],
4         show_progress_bar=False
5     )
6     for chunk, score in zip(chunks, scores):
7         chunk["rerank_score"] = round(float(score), 4)
8     # Filtrage par seuil MIN_RERANK_SCORE = 2.0
9     ranked = [c for c in
10                sorted(chunks, key=lambda c: c["rerank_score"],
11                      reverse=True)
12                   if c["rerank_score"] >= MIN_RERANK_SCORE]
13     # Deduplication : max 3 chunks par source_book
14     source_counts = {}
15     deduped = []
16     for c in ranked:
17         src = (c["metadata"].get("source_book")
18               or c["metadata"].get("source_type", ""))
19         if source_counts.get(src, 0) < MAX_CHUNKS_PER_SOURCE:
20             deduped.append(c)
21             source_counts[src] = source_counts.get(src, 0) +
22                 1
23     return deduped[:top_k]
```

Listing 6.4 – Fonction de reranking par cross-encoder

6.7.2 Seuillage et déduplication

Le paramètre `MIN_RERANK_SCORE = 2.0` [F2] filtre les chunks dont le score du cross-encoder est inférieur à ce seuil, éliminant les passages hors-sujet. Ce seuil, relevé de 0.5 à 2.0 par rapport à la version précédente, permet un filtrage efficace tout en conservant

les chunks borderline potentiellement utiles. Un mécanisme de déduplication limite à `MAX_CHUNKS_PER_SOURCE = 3` le nombre de chunks provenant d'une même source, évitant qu'un seul ouvrage ne domine le contexte injecté au LLM.

Un filet de sécurité (*safety net*) est implémenté : si le reranker filtre tous les candidats, les meilleurs candidats RRF bruts sont utilisés à la place, garantissant qu'une réponse est toujours générée.

6.8 Large Language Model

6.8.1 Modèle et infrastructure d'exécution locale

Le modèle de langage utilisé pour générer les réponses est **llama3.2 :3b**, déployé localement à travers **Ollama** (<http://localhost:11434/api/generate>). Cette architecture *on-premise* offre plusieurs bénéfices dans le cadre d'un système d'information SAP d'entreprise : sécurité des données (aucune demande ne quitte l'infrastructure interne), contrôle des coûts (pas de facturation à l'utilisation), et autonomie par rapport à un fournisseur de cloud. Un modèle alternatif, **qwen2.5:7b**, est également pris en charge via la variable d'environnement `OLLAMA_MODEL_QWEN`, permettant de choisir entre rapidité (llama3.2 :3b) et qualité supérieure (qwen2.5 :7b).

6.8.2 Mode streaming et intégration API

Le LLM supporte deux modes de génération : le mode *streaming* (tokens renvoyés au fil de la génération, affiché en temps réel dans le terminal) et le mode *non-streaming* (réponse complète renvoyée en une fois). L'API FastAPI désactive systématiquement le streaming (`stream=False`) pour renvoyer une réponse JSON complète au frontend Fiori.

6.9 Prompt Engineering

6.9.1 Structure du prompt système

Le prompt système (`SYSTEM_PROMPT`) définit le rôle du LLM et ses contraintes de **synthèse**. Il positionne le modèle comme un *expert SAP technical consultant* et lui demande explicitement de *synthétiser* une réponse à partir des blocs d'évidence pré-sélectionnés (déjà filtrés par `synthesize_size_chunks`), en appliquant quatre règles de synthèse :

1. **Fusionner** les faits connexes provenant de plusieurs blocs en une réponse cohérente ; ne pas répéter le même fait par source.

2. **Convergence** : si deux blocs s'accordent sur un fait, l'énoncer une seule fois avec double citation : "as described in [1] [3]".
3. **Conflit** : si deux blocs se contredisent, présenter explicitement les deux versions : "[1] states X, while [2] states Y".
4. Citer les sources **en ligne** uniquement via la notation [N] ; ne jamais inventer de citations absentes des en-têtes d'evidence.

Des règles strictes (*STRICT RULES*) interdisent explicitement l'invention de T-codes, de titres de livres, d'URLs SAP Help ou d'étapes non présentes dans l'evidence. Le LLM doit déclarer explicitement : "The available documentation does not cover this." si le contexte est insuffisant. Cette combinaison de *grounding* strict et de *synthesis rules* constitue la principale défense contre les hallucinations tout en améliorant la cohérence des réponses multi-sources.

6.9.2 Construction dynamique du prompt

Le prompt final soumis au LLM est assemblé dynamiquement par la fonction `build_prompt()` à partir de trois composants : le prompt système fixe, les blocs d'evidence récupérés, et l'historique conversationnel. Chaque chunk est présenté au LLM sous forme d'un bloc numéroté avec un en-tête structuré :

```
1 [1] ADM325_Software_Logistics -- Setting Up RFC Connections (
   BASIS > RFC)
2 The RFC destination SM59 allows configuring connections
   between SAP
3 systems. Enter the target hostname, system number, and logon
   data...
```

Listing 6.5 – Exemple de bloc d'evidence injecté dans le prompt

L'en-tête contient : le numéro de citation [N], le nom du livre source, le titre de la section, et le *breadcrumb* hiérarchique (BASIS > RFC). Cette structuration permet au LLM de citer précisément ses sources via la notation [1] [3] et d'éviter toute confusion entre documents de même thématique.

6.9.3 Pré-synthèse des chunks (`synthesis_text`)

Un document récupéré par le moteur de recherche peut contenir dix phrases : certaines directement utiles à la question posée, d'autres hors-sujet. Si l'on injecte le document entier dans le prompt du LLM, les phrases inutiles consomment de l'espace et peuvent distraire le modèle. La fenêtre de contexte du LLM étant limitée, chaque phrase compte.

La **pré-synthèse** résout ce problème : avant d’injecter un document dans le prompt, on filtre ses phrases pour ne garder que celles qui sont réellement utiles à la question posée. Ce filtrage se fait automatiquement par similarité sémantique : chaque phrase du document est comparée à la question, et seules les phrases dont le score de pertinence dépasse un seuil sont conservées.

Concrètement, la fonction `synthesize_chunks()` applique les étapes suivantes :

1. Les phrases du chunk sont découpées et vectorisées individuellement avec BGE-large.
2. Chaque phrase est scorée par similarité cosinus avec le vecteur de la requête.
3. Seules les phrases dépassant un seuil de pertinence sont conservées dans `synthesis_text`.
4. Les chunks quasi-identiques (similarité inter-chunks > 0.92) sont éliminés comme doublons.

```

1 # Pre-synthesized sentences if available, fallback to raw
  content
2 content = c.get("synthesis_text") or c["content"]
3 content = content[:500] # Hard-cap to keep total prompt
  under 2048 tokens

```

Listing 6.6 – Utilisation de `synthesis_text` dans `build_prompt()`

Cette pré-synthèse réduit le bruit injecté dans le prompt : le LLM reçoit une sélection de phrases directement utiles plutôt que des blocs de texte bruts pouvant contenir des informations hors-sujet. Elle améliore également la fidélité de la réponse en concentrant l’attention du modèle sur les passages les plus pertinents.

6.9.4 Injection de l’historique conversationnel

Pour supporter les échanges multi-tours, `build_prompt()` intègre les derniers tours de la conversation sous forme d’un bloc `RECENT CONVERSATION HISTORY` :

```

1 === RECENT CONVERSATION HISTORY ===
2 User: How do I check background jobs in SAP?
3 Assistant: To monitor background jobs, use transaction SM37
4   ...
5 === QUESTION ===
6 Which statuses indicate a failed job?

```

Listing 6.7 – Injection de l’historique conversationnel

Chaque message est tronqué à 150 caractères afin de contenir la taille totale du prompt dans la fenêtre de contexte du LLM (2048 tokens pour llama3.2 :3b). Cette

troncature préserve la référence contextuelle tout en évitant de saturer le budget de tokens alloué aux chunks d'evidence.

6.9.5 Contrôle anti-hallucination post-génération

Un modèle de langage peut parfois inventer des informations qui semblent plausibles mais sont factuellement incorrectes : on parle d'**hallucination**. Dans le contexte SAP, ce risque est particulièrement problématique : un T-code inventé ou un numéro de note inexistant peut induire un technicien en erreur directement dans son environnement de production.

Pour limiter ce risque, un mécanisme de vérification automatique est appliqué après chaque génération. Son principe est simple : tout T-code ou numéro de note SAP mentionné dans la réponse du LLM est comparé à ceux présents dans les documents sources récupérés. Si le LLM cite un identifiant qui n'apparaît dans aucun des documents fournis, cela constitue une hallucination probable.

Ce contrôle opère en deux étapes complémentaires :

- **Étape 1. Détection par `check_hallucinations()`**. La fonction compare les T-codes et numéros de notes SAP présents dans la réponse générée avec ceux effectivement contenus dans les chunks récupérés. Tout T-code connu (`KNOWN_TCODES`) mentionné dans la réponse sans apparaître dans le contexte est signalé comme hallucination potentielle :

$$\text{T-codes halluc.} = (\text{T-codes}(\text{réponse}) \setminus \text{T-codes}(\text{contexte})) \cap \text{KNOWN_TCODES}$$

- **Étape 2. Sanitisation par `sanitize_answer()`**. Les phrases de la réponse dont tous les T-codes mentionnés sont hallucinés (non présents dans le contexte) sont supprimées de la réponse finale avant renvoi à l'utilisateur. Les phrases mixtes (T-codes halluc. + T-codes légitimes) sont conservées et signalées en **warnings** dans la réponse API.

6.10 Workflow global d'une requête utilisateur

Le traitement d'une requête utilisateur suit plusieurs étapes depuis l'interface SAP Fiori jusqu'au retour de la réponse finale. Après la soumission de la question dans le chatbot, la requête est envoyée au backend Python via l'endpoint `POST /query` de l'API FastAPI. Une session identifiée par UUID est ensuite récupérée ou créée avant l'appel de la fonction principale `ask()`.

La première étape du pipeline consiste en une analyse NLP **rule-based** avec `understand_query()`, permettant d'identifier l'intention, les entités SAP et une reformulation optimisée de la requête, sans appel au LLM. La requête enrichie est ensuite vectorisée avec BGE-large-en-v1.5 puis comparée au **cache sémantique**. En cas de similarité suffisante, la réponse est immédiatement renvoyée sans exécuter de retrieval ni de génération.

En cas de cache miss, la fonction `retrieve()` lance en parallèle une recherche vectorielle et deux recherches BM25. Les résultats sont fusionnés par RRF, avec relâchement progressif des filtres si nécessaire. Pour les requêtes contenant un T-code connu, les chunks du **cache CAG** sont injectés dans le contexte. Les meilleurs résultats sont ensuite rerankés par un cross-encoder ; un *safety net* restaure les meilleurs chunks RRF si tous les candidats sont rejetés.

Les chunks retenus sont synthétisés afin d'extraire les informations essentielles et supprimer les redondances. Le prompt final est construit avec l'historique conversationnel puis envoyé au modèle Ollama pour la génération de la réponse. Après vérification via `check_hallucinations()`, la réponse est stockée dans le cache sémantique et retournée sous forme JSON avec les sources, scores de pertinence, avertissements éventuels et identifiant de session. Enfin, la session est sauvegardée de manière asynchrone dans PostgreSQL.

6.11 Algorithme général du pipeline

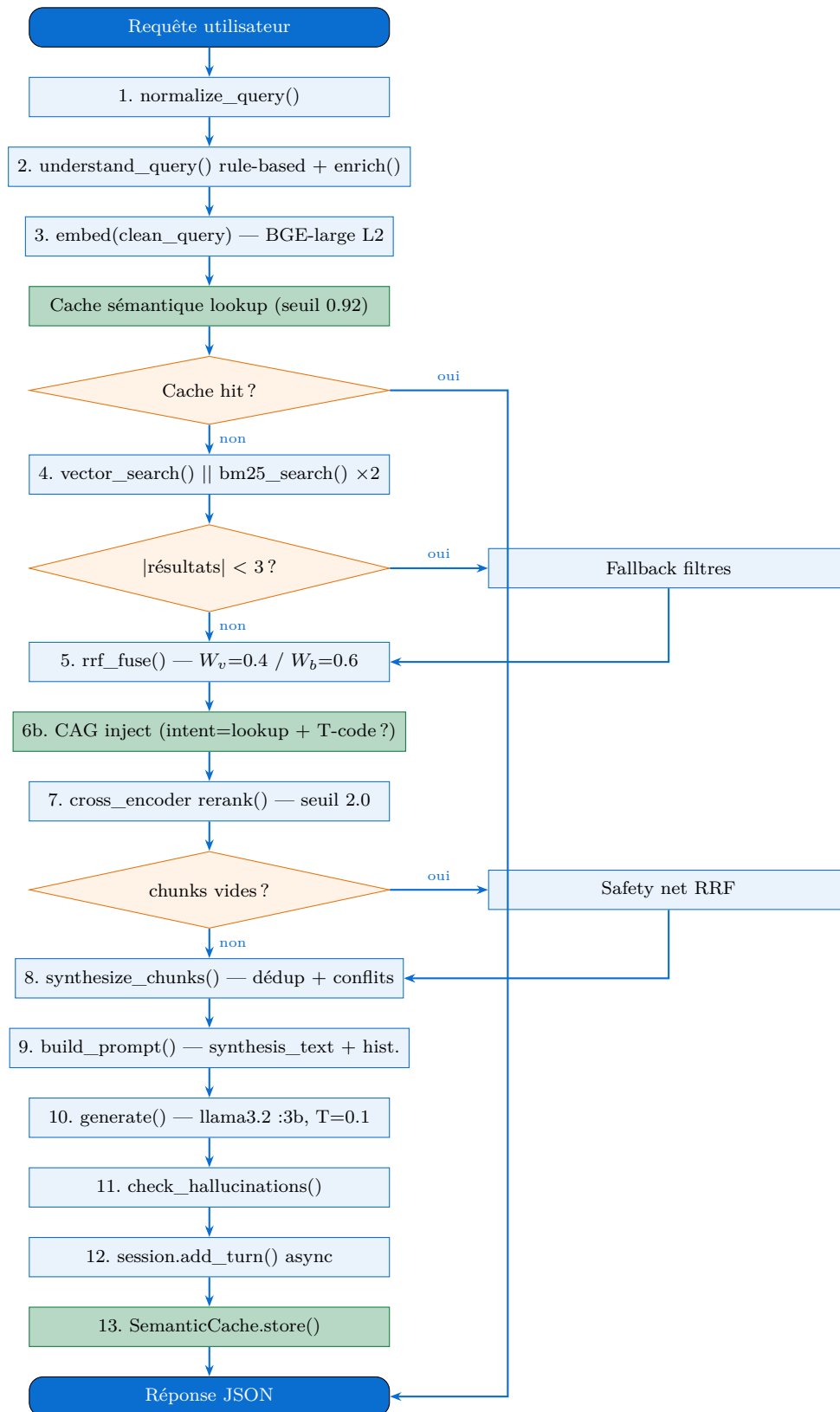


FIGURE 6.1 – Flowchart du pipeline RAG SAP V2 Fixed (rag_pipeline_v2_fixed.py)

6.11.1 Limites identifiées

Malgré ses nombreux atouts, le système comporte certaines limitations. La génération locale avec `llama3.2:3b` reste plus lente et moins performante qu'un grand modèle hébergé dans le cloud, tel que GPT-4o ou Claude Sonnet. De plus, la fenêtre de contexte limitée à 8192 tokens, bien qu'adaptée à l'utilisation de cinq chunks, peut devenir insuffisante pour des requêtes nécessitant un volume documentaire important.

La détection des hallucinations repose essentiellement sur des heuristiques basées sur des expressions régulières identifiant des T-codes et des numéros de notes. Cette approche ne permet pas de détecter toutes les formes d'hallucination, notamment les affirmations factuellement erronées ne contenant aucun identifiant explicite. Par ailleurs, le **cache sémantique fonctionne uniquement en mémoire** : son contenu est perdu à chaque redémarrage de l'application et il ne peut pas être mutualisé entre plusieurs instances, ce qui réduit son intérêt dans une architecture distribuée ou multi-processus.

L'analyse **NLP fondée sur des règles**, bien qu'efficace en termes de rapidité et de déterminisme, montre également ses limites face aux requêtes ambiguës, complexes ou multilingues. Les formulations en français ou les tournures inhabituelles peuvent ainsi conduire à une mauvaise classification des intentions utilisateur. Enfin, les performances du mécanisme de retrieval dépendent fortement de la qualité du découpage initial des documents ainsi que de la pertinence des métadonnées associées, rendant indispensable un important travail de préparation et de structuration des données en amont.

Conclusion

Ce chapitre a présenté la phase de modélisation du système RAG hybride développé pour la documentation SAP. L'architecture combine plusieurs composants complémentaires, notamment le NLP rule-based, le cache sémantique, le retrieval hybride BM25 + vecteurs, le reranking par cross-encoder et la génération via Llama3.2 avec Ollama. Les améliorations intégrées.

Chapitre 7

Évaluation

Introduction

L'évaluation d'un système RAG est une étape essentielle pour vérifier la qualité des réponses fournies aux utilisateurs. Étant donné que ce type de système repose à la fois sur la recherche d'informations et sur la génération de réponses, il est nécessaire d'évaluer ces deux aspects afin de mesurer son efficacité globale.

Dans ce chapitre, nous présentons l'approche adoptée pour évaluer la performance de la solution développée. Cette évaluation porte à la fois sur la pertinence des informations retrouvées et sur la qualité des réponses générées, afin de garantir une assistance fiable et adaptée aux besoins des utilisateurs.

7.1 Protocole d'évaluation

7.1.1 Dataset de test

Pour évaluer un système RAG, il faut disposer d'un ensemble de questions de test représentatives (appelé *dataset de test*), accompagnées d'informations permettant de juger la qualité des réponses produites. Ce dataset joue le rôle d'un examen : il soumet au système des questions variées et mesure si ses réponses sont correctes et bien fondées.

Quelles questions tester ? Le pipeline développé reconnaît quatre grandes catégories de questions SAP, appelées *types d'intention* :

- **troubleshooting** : l'utilisateur rencontre une erreur et cherche à la résoudre (ex. : « *Why does job SM37 fail with status cancelled ?* ») ;
- **configuration** : l'utilisateur veut paramétrer un composant SAP (ex. : « *How to configure SSL certificates in SAP BASIS ?* ») ;

- **how-to** : l'utilisateur cherche une procédure à suivre étape par étape (ex. : « *How to transport objects using STMS ?* »);
- **explanation** : l'utilisateur veut comprendre un concept ou un mécanisme SAP (ex. : « *What is the role of the ABAP dispatcher ?* »).

Pour que l'évaluation soit équilibrée, le dataset doit contenir des questions de chacune de ces catégories, et couvrir plusieurs modules SAP (ABAP, BASIS, SECURITY, HANA, FIORI, BW).

Pourquoi 18 requêtes ? Le nombre de 18 questions résulte de quatre contraintes pratiques cumulées :

1. **Couverture minimale.** Avec quatre types d'intention et six modules SAP à couvrir, il faut au minimum trois questions par catégorie pour obtenir des résultats représentatifs ; on arrive ainsi naturellement à un total compris entre 15 et 20 questions.
2. **Effort d'annotation.** Chaque question nécessite qu'un expert SAP définisse manuellement ses *signaux de pertinence*, c'est-à-dire les mots-clés et T-codes qui doivent figurer dans un bon document de réponse. C'est ce travail d'expertise, et non le simple volume de questions, qui limite la taille du dataset.
3. **Contrainte matérielle.** L'évaluation de la génération (présentée en section 7.3) mobilise simultanément le modèle d'embedding et le modèle de langage, saturant la mémoire disponible de la machine dès trois requêtes traitées en parallèle. Un dataset plus grand n'aurait pu être évalué à ce niveau sans ressources supplémentaires.
4. **Objectif de validation préliminaire.** Ce dataset vise à détecter d'éventuels dysfonctionnements majeurs du pipeline avant déploiement, et non à établir des conclusions statistiquement définitives. Cette limitation est discutée à la section 7.2.6.

La répartition finale des 18 requêtes est présentée dans le tableau 7.1.

TABLE 7.1 – Distribution du dataset de test par type d'intention et module SAP

Type d'intention	Nombre de requêtes	Modules couverts
troubleshooting	5	ABAP, BASIS, SECURITY, HANA
configuration	3	BASIS, SECURITY
how-to	5	BASIS, FIORI, ABAP
explanation	5	BASIS, HANA, ABAP, SECURITY, BW
Total	18	6 modules

Chaque requête est accompagnée d'une liste de *signaux*, c'est-à-dire des termes ou T-codes qui doivent impérativement apparaître dans les documents récupérés pour que ceux-ci soient considérés pertinents. Ces signaux servent de référence pour l'évaluation automatique, présentée dans la section suivante.

7.1.2 Méthode d'auto-labellisation

Pour évaluer si le système retrouve les bons documents, il faut savoir à l'avance quels documents sont « corrects » pour chaque question. Dans un projet classique, cette annotation est réalisée manuellement par des experts : pour chaque question, un spécialiste lit chaque document récupéré et décide s'il est pertinent ou non. Avec 18 questions et 10 documents récupérés par question, cela représenterait 180 jugements humains, ce qui représente un travail fastidieux et difficile à reproduire.

Pour éviter ce coût, on utilise une **méthode d'auto-labellisation par signaux**. Le principe est simple : pour chaque requête, on définit à l'avance une liste de *signaux*, c'est-à-dire les mots-clés ou codes SAP (T-codes) qui *doivent* obligatoirement figurer dans un document pour qu'il soit considéré comme pertinent. Ces signaux sont définis par un expert SAP au moment de la rédaction des questions.

Exemple concret. Pour la question « *How to analyze ABAP short dumps in ST22 ?* », les signaux définis sont : ST22, short dump, ABAP runtime error, dump analysis. Un document récupéré par le système est ensuite comparé automatiquement à cette liste :

- s'il contient **3 signaux ou plus**, il est jugé **très pertinent** (score 2) ;
- s'il contient **1 ou 2 signaux**, il est jugé **partiellement pertinent** (score 1) ;
- s'il ne contient **aucun signal**, il est jugé **non pertinent** (score 0).

Cette comparaison est entièrement automatique et ne nécessite aucune intervention humaine après la définition initiale des signaux. Elle produit un score gradué (0, 1

ou 2) qui reflète le degré de pertinence, et non un simple oui/non, ce qui permet aux métriques comme le nDCG de tenir compte de la qualité du classement, pas seulement de la présence ou l'absence d'un résultat pertinent.

Cette approche est reconnue dans la littérature comme un substitut acceptable au jugement humain lorsque les signaux sont définis avec soin par des experts du domaine [9]. Sa principale limite (la circularité entre les signaux utilisés pour annoter et les termes exploités par le retrieval) est discutée à la section 7.2.6.

7.2 Évaluation du Retrieval

7.2.1 Métriques implémentées

Une fois le dataset de test constitué et les labels de pertinence calculés automatiquement, il faut des *métriques* pour mesurer objectivement si le système retrouve les bons documents. Une métrique est simplement une formule mathématique qui transforme les résultats du système en un score entre 0 et 1 : plus le score est proche de 1, meilleures sont les performances.

Trois familles de métriques complémentaires ont été retenues, chacune répondant à une question différente sur la qualité du retrieval.

Recall@k : « Combien de bons documents sont retrouvés ? » Le Recall@k mesure la proportion de documents pertinents que le système a réussi à retrouver parmi les k premiers résultats. Par exemple, si pour une question donnée il existe 4 documents pertinents dans les 10 résultats récupérés, et que le système en place les 3 en tête de liste, alors le $\text{Recall}@3 = 3/4 = 0.75$.

$$\text{Recall}@k = \frac{\text{nombre de documents pertinents dans les } k \text{ premiers}}{\text{nombre total de documents pertinents dans les 10 résultats récupérés}}$$

Remarque importante. Le dénominateur ne compte que les documents pertinents parmi les 10 résultats récupérés, et non parmi les 76 463 documents du corpus entier. Un $\text{Recall}@10 = 1.0$ signifie donc que *tous les documents jugés pertinents dans cette fenêtre de 10 résultats ont été retrouvés*, pas qu'aucun document utile n'existe ailleurs dans le corpus. Cette convention par *pool* est une pratique standard dans les évaluations à grande échelle [9].

MRR : « Le premier bon document arrive-t-il en tête ? » Le Mean Reciprocal Rank (MRR) mesure la position du *premier* document pertinent dans la liste de résultats, moyennée sur toutes les questions. Si le premier document pertinent est en position 1,

la contribution est $1/1 = 1.0$; s'il est en position 3, la contribution est $1/3 \approx 0.33$. Un $\text{MRR} = 1.0$ signifie que pour chaque question, un document pertinent apparaît *systématiquement* en première position, ce qui est la situation idéale pour un assistant conversationnel, où l'utilisateur ne lit généralement que la première source citée.

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rang du premier document pertinent pour la question } q}$$

nDCG@k : « Les meilleurs documents sont-ils bien classés en premier ? » Le nDCG@k (Normalized Discounted Cumulative Gain) est la métrique la plus complète des trois : elle tient compte à la fois du *degré de pertinence* (un document très pertinent vaut plus qu'un document partiellement pertinent) et de la *position dans le classement* (un bon document en première position vaut plus qu'en cinquième position). C'est cette pénalité de position, appelée « discount », qui rend le nDCG particulièrement adapté aux systèmes de recherche, où l'ordre des résultats est aussi important que leur présence.

$$\text{nDCG@k} = \frac{\sum_{i=1}^k \frac{\text{score de pertinence du document en position } i}{\log_2(i+1)}}{\text{score idéal théorique si tous les meilleurs documents étaient en tête}}$$

Un $\text{nDCG@k} = 1.0$ correspond au classement parfait : les documents les plus pertinents sont tous placés en première position. Un nDCG@k proche de 0 indique que les documents retrouvés sont peu pertinents ou mal ordonnés.

7.2.2 Configuration d'évaluation

Avant de présenter les résultats, il est nécessaire de préciser dans quelles conditions l'évaluation a été réalisée. Ces paramètres permettent au lecteur de reproduire l'expérience et d'interpréter correctement les scores obtenus.

Nombre de résultats récupérés (top_k = 10). Pour chaque question du dataset, le système récupère les **10 documents les plus pertinents** du corpus. Ce paramètre, noté $\text{top_k} = 10$, définit la fenêtre d'évaluation : seuls ces 10 documents sont analysés et notés. Augmenter ce nombre améliorerait mécaniquement le Recall, mais au prix d'un contexte plus bruité transmis au modèle de langage.

Activation du reranker. Le reranker est un second modèle (`cross-encoder/ms-marco-MiniLM-L-` qui intervient *après* la recherche hybride pour réordonner les 10 résultats récupérés.

Contrairement au moteur de recherche initial qui compare chaque document à la question de manière indépendante, le reranker analyse la paire (question, document) conjointement, ce qui lui permet d’affiner le classement avec une meilleure compréhension du contexte. C’est son impact sur les métriques de classement (MRR, nDCG) qui est mesuré ici.

Comparaison avec une baseline. Pour mesurer l’apport réel du système hybride, les résultats sont comparés à une **baseline** : une recherche vectorielle pure, sans BM25 ni fusion de scores RRF, appliquée sur le même corpus de 76 463 vecteurs indexés dans PostgreSQL. Cette baseline représente ce que le système produirait si l’on retirait toutes les améliorations hybrides ; tout gain observé dans le tableau de résultats est donc directement attribuable à ces améliorations.

7.2.3 Résultats globaux

Le tableau 7.2 compare deux versions du système sur les mêmes 18 questions.

La colonne « **Hybride + Reranking** » représente le système complet tel qu’il a été développé : il combine la recherche sémantique par vecteurs, la recherche par mots-clés (BM25), la fusion des deux scores (RRF), puis un reranker qui affine le classement final.

La colonne « **Baseline (vecteur pur)** » représente une version simplifiée du système, utilisée comme point de référence. Elle repose uniquement sur la recherche sémantique par vecteurs, sans aucune des améliorations ajoutées. Concrètement, c’est ce que le système aurait produit si l’on n’avait utilisé qu’un moteur de recherche standard basé sur la similarité de sens entre la question et les documents. La comparer au système complet permet de mesurer précisément le gain apporté par chaque amélioration.

TABLE 7.2 – Résultats du retrieval : Système hybride vs baseline vectorielle pure

Métrique	Hybride + Reranking	Baseline (vecteur pur)
Recall@1	0.122	0.103
Recall@3	0.367	0.330
Recall@5	0.574	0.526
Recall@10	1.000	1.000
MRR	1.000	0.972
nDCG@3	0.959	0.878
nDCG@5	0.959	0.891
nDCG@10	0.983	0.947

Sur cet échantillon de 18 requêtes, le $MRR = 1.0$ indique qu’au moins un chunk pertinent apparaît en première position pour chaque requête, et le $Recall@10 = 1.0$ signifie que tous les chunks jugés pertinents dans le pool évalué sont récupérés dans les 10 premiers résultats. Ces valeurs sont prometteuses, mais doivent être interprétées avec prudence : l’échantillon est restreint ($n = 18$), les labels sont auto-générés par des signaux lexicaux alignés sur le corpus de retrieval (voir § 7.2.6), et l’évaluation devra être confirmée sur un jeu de test plus large et indépendant.

7.2.4 Résultats par type d’intention

Le tableau précédent donnait une moyenne globale sur l’ensemble des 18 questions. Ce tableau va plus loin : il décompose les performances selon les quatre catégories de questions définies dans le dataset, afin de vérifier si le système se comporte de manière homogène ou si certains types de questions lui posent plus de difficultés que d’autres.

- Les questions de **troubleshooting** contiennent souvent des T-codes explicites (par exemple **SM37**, **ST22**, **SU53**). Ces codes agissent comme des marqueurs précis : le système peut les repérer directement dans les documents et les utiliser comme filtre. C’est pourquoi ces questions obtiennent le meilleur Recall@5.
- Les questions de **configuration** et **how-to** portent sur des procédures bien documentées dans le corpus SAP. Les documents pertinents sont en général peu nombreux mais ciblés, ce qui favorise un classement précis ($nDCG@3 = 1.000$ pour ces deux types).

- Les questions d'**explanation** sont les plus difficiles : elles demandent de comprendre un concept (par exemple « What is the role of the ABAP dispatcher ? ») sans contenir d'entité SAP précise. Le système ne peut pas s'appuyer sur un T-code pour filtrer les résultats, ce qui explique le Recall@5 et le nDCG@3 légèrement inférieurs.

TABLE 7.3 – Performance du retrieval décomposée par type d'intention

Type	Recall@5	MRR	nDCG@3	nDCG@10
troubleshooting	0.678	1.000	0.953	0.978
configuration	0.590	1.000	1.000	0.996
how-to	0.525	1.000	1.000	0.997
explanation	0.511	1.000	0.900	0.968
Moyenne	0.574	1.000	0.959	0.983

Le type *troubleshooting* obtient le Recall@5 le plus élevé (0.678) en raison de la présence de T-codes explicites dans ces requêtes (ex. SM37, ST22, SU53) qui permettent un filtre précis lors du retrieval. Les requêtes de type *explanation* sont naturellement plus difficiles, car elles ne contiennent pas d'entités SAP directement adressables.

Note sur la valeur du Recall@5. Le Recall@5 = 0.574 s'explique par la structure du corpus : le dénominateur de la formule comptabilise les chunks pertinents dans le pool de 10 résultats récupérés, et non dans l'intégralité des 76 463 vecteurs indexés. Pour une requête sur SM37, le corpus peut contenir 8 à 10 chunks pertinents répartis sur plusieurs sections de la documentation SAP Help ; seuls 5 peuvent figurer dans les 5 premiers résultats, même si le système les détecte tous dans le top 10 (Recall@10 = 1.000). Ce comportement est structurel : il traduit la densité du corpus et non une défaillance du retrieval. Pour un assistant conversationnel, le **MRR = 1.000** est la métrique opérationnelle pertinente : il garantit qu'un chunk hautement pertinent apparaît systématiquement en première position, ce qui suffit pour construire une réponse de qualité.

7.2.5 Couverture des signaux

La couverture des signaux mesure combien de termes de référence ont été retrouvés dans les chunks récupérés, indépendamment du classement. Sur les 18 requêtes, 16 atteignent une couverture de 100 %, et 2 manquent d'un signal sur 5 (Q02 et Q04 — à distinguer des anomalies de classement Q04 et Q10 discutées à la section 7.5).

7.2.6 Limites de validité du protocole d'évaluation

7.2.6.1 Circularité de l'auto-labellisation

La méthode d'évaluation présente un risque de circularité : les labels de pertinence sont définis par la présence de signaux (T-codes, termes-clés), et le système de retrieval exploite précisément ces mêmes T-codes comme filtres de métadonnées et comme termes pondérés dans BM25. En conséquence, les scores élevés obtenus — notamment le $MRR = 1.0$ — mesurent en grande partie le *recouvrement lexical* entre la requête et les documents, et non une pertinence sémantique plus profonde. Les requêtes de type *troubleshooting*, qui contiennent des T-codes explicites, bénéficient particulièrement de cet alignement, ce qui explique leurs scores supérieurs. Pour lever cette circularité, un sous-ensemble de jugements humains sur des requêtes dépourvues de T-codes explicites serait nécessaire.

7.2.6.2 Taille de l'échantillon

L'évaluation du retrieval repose sur $n = 18$ requêtes, et l'évaluation de génération sur $n = 3$ requêtes seulement. Ces tailles sont insuffisantes pour tirer des conclusions statistiquement robustes. L'intervalle de confiance des métriques agrégées est large ; les résultats constituent une indication préliminaire à confirmer sur un jeu de test étendu (idéalement $n \geq 100$ pour le retrieval).

7.3 Niveau 2 : Évaluation LLM-as-Judge avec RAGAS

7.3.1 Présentation et métriques RAGAS

Le Niveau 1 mesurait si les *bons documents* étaient retrouvés, en comparant les résultats du moteur de recherche à une liste de signaux attendus. Mais il ne disait rien sur la *réponse finale* : le modèle de langage a-t-il bien utilisé ces documents ? A-t-il inventé des informations ? Sa réponse répond-elle vraiment à la question posée ?

Pour répondre à ces questions, on utilise une approche différente appelée **LLM-as-Judge** : un second modèle de langage joue le rôle d'un correcteur automatique. On lui soumet la question, la réponse générée, et les documents sources récupérés, et il attribue un score entre 0 et 1 selon des critères précis. Cette approche est automatisée grâce au framework **RAGAS** [5], un outil open-source conçu spécifiquement pour évaluer les pipelines RAG.

Dans ce projet, le modèle-juge utilisé est **llama3.2:3b**, exécuté localement via Ollama, sans appel à une API externe, ce qui garantit la reproductibilité complète de

l'évaluation et reste cohérent avec l'architecture hors-ligne du système.

Quatre métriques complémentaires ont été retenues, chacune évaluant un aspect distinct :

TABLE 7.4 – Description des métriques RAGAS utilisées

Métrique	Entrées requises	Ce qu'elle mesure
Faithfulness	Réponse + contextes	Chaque affirmation de la réponse est-elle ancrée dans le contexte fourni ? Détecte les hallucinations.
Answer Relevancy	Question + réponse	La réponse adresse-t-elle bien la question posée ? Pénalise les réponses hors-sujet.
Context Recall	Question + contextes + référence	Le contexte récupéré contient-il les informations nécessaires pour répondre correctement ?
Context Precision	Question + contextes + référence	Les contextes récupérés sont-ils précis et peu bruités ? Mesure le rapport signal/bruit.

7.3.2 Configuration de l'évaluation RAGAS

Contrairement au Niveau 1 où l'on évaluait 18 questions, l'évaluation RAGAS ne porte que sur **3 questions représentatives** : une question de troubleshooting ABAP (Q01), une question de sécurité et autorisations (Q03), et une question de monitoring BASIS (Q14). Cette réduction s'explique par une contrainte matérielle concrète : faire tourner simultanément le modèle d'embedding (1,5 Go en mémoire) et le modèle-juge (2 Go via Ollama) sature la RAM disponible sur la machine locale. Évaluer davantage de questions aurait nécessité une infrastructure plus puissante.

Pour chacune de ces trois questions, une **réponse de référence** a été rédigée manuellement par un expert SAP. C'est cette réponse idéale qui sert de repère au modèle-juge pour noter la réponse générée par le système.

Les autres paramètres utilisés sont les suivants : le système récupère les 5 documents les plus pertinents (`top_k = 5`), et chaque appel au modèle-juge dispose d'un délai maximum de 600 secondes avant d'être interrompu, soit un timeout nécessaire car le modèle tourne entièrement sur CPU, sans carte graphique dédiée.

7.3.3 Résultats RAGAS

TABLE 7.5 – Résultats RAGAS — Évaluation LLM-as-Judge (llama3.2:3b) sur 3 requêtes SAP

Métrique	Score moyen	Calculable	Ce que ça signifie concrètement
Answer Relevancy	0.920	Oui	Le système répond bien à ce qui est demandé : les réponses restent centrées sur la question, sans partir sur des informations hors-sujet.
Context Recall	0.889	Oui	Les documents récupérés contiennent, dans la grande majorité des cas, les informations nécessaires pour construire une réponse correcte.
Faithfulness	N/A	Non	Non calculé : le modèle-juge llama3.2:3b est trop petit pour produire la réponse structurée requise par RAGAS. Tous les appels ont échoué (voir § 7.8.2).
Context Precision	N/A	Non	Non calculé pour la même raison. Un modèle-juge plus puissant est nécessaire pour évaluer ces deux métriques (voir § 7.8.2).

Le tableau suivant détaille les scores question par question, afin de voir si certaines questions ont posé plus de difficultés que d'autres au système.

TABLE 7.6 – Résultats RAGAS détaillés par requête

ID	Question posée	Answer Relevancy
Q01	Comment analyser les erreurs ABAP avec ST22 ?	—

La valeur Answer Relevancy de Q01 est absente en raison d'un *TimeoutError* causé par la saturation CPU lors de l'exécution simultanée du modèle d'embedding et du juge LLM. Le Context Recall de Q14 (0.667) reflète une requête multi-entités : les chunks récupérés couvrent principalement SM50 et ne contiennent qu'une couverture partielle des métriques OS de ST06.

7.3.4 Analyse des résultats

Sur les deux métriques évaluables ($n = 3$ requêtes), les résultats sont encourageants mais insuffisants pour conclure de façon générale. L'Answer Relevancy de **0.92** suggère que les réponses adressent les questions posées sans dériver hors-sujet ; le Context Recall de **0.89** indique que le contexte récupéré couvre en grande partie les informations de référence, ce qui est cohérent avec le $MRR = 1.0$ et le $nDCG@3 = 0.959$ observés au Niveau 1. Ces chiffres doivent être lus comme des indicateurs préliminaires sur un sous-ensemble très réduit : avec $n = 3$, un seul outlier modifie la moyenne de 0.33 point, et deux métriques sur quatre (Faithfulness, Context Precision) sont non calculables à cause des échecs de parsing JSON du juge.

Le Context Recall de Q14 (0.667) est inférieur aux deux autres requêtes (1.0) : la requête portant sur ST06 et SM50 requiert des informations sur les métriques OS (ST06) et sur le moniteur de processus (SM50), mais les chunks récupérés couvrent principalement SM50. Ce résultat illustre la limite des requêtes multi-entités, qui nécessitent plusieurs chunks complémentaires pour une couverture complète.

7.4 Comparaison Hybride vs Baseline

La comparaison entre le système hybride complet (vectoriel + BM25 + RRF + cross-encoder) et la baseline (recherche vectorielle pure) met en évidence plusieurs apports concrets. Le tableau suivant résume les gains obtenus :

TABLE 7.7 – Gain apporté par le système hybride par rapport à la baseline

Métrique	Baseline	Hybride	Gain
MRR	0.972	1.000	+2.9%
nDCG@3	0.878	0.959	+9.2%
nDCG@5	0.891	0.959	+7.6%
nDCG@10	0.947	0.983	+3.8%

Le reranking par cross-encoder garantit qu'un chunk pertinent apparaît systéma-

tiquement en première position (MRR : 1.000 vs 0.972). Le gain le plus significatif concerne le nDCG@3 (+9.2%), ce qui est particulièrement important pour un système de chat : l'utilisateur ne consulte généralement que les 3 à 5 premières sources citées dans la réponse. L'amélioration se maintient sur toute la fenêtre de 10 résultats (nDCG@10 : +3.8%).

7.5 Analyse visuelle des performances

La figure 7.1 synthétise l'ensemble des résultats d'évaluation sous quatre angles complémentaires. Le premier graphique confirme l'apport mesurable du reranker : le système avec reranker dépasse systématiquement la baseline sur les quatre métriques clés, avec la différence la plus marquée sur le nDCG@3 (+9,2%). Le deuxième graphique présente la distribution des scores sur les 18 requêtes sous forme de boîtes à moustaches : le nDCG@10 est très concentré près de 1.0, tandis que le Recall@1 montre une dispersion plus importante, confirmant que la première position brute est moins fiable que les positions agrégées. Le troisième graphique trace l'évolution du nDCG@10 requête par requête, coloré par type d'intention ; les deux anomalies de *classement* (Q04 et Q10) y sont clairement identifiables (à distinguer des requêtes Q02 et Q04 présentant des lacunes de *couverture de signaux*, cf. § 7.3.5) : Q10 correspond à une requête de configuration STMS dont les chunks pertinents étaient mal positionnés avant reranking, et Q04 à une requête HANA backup pour laquelle le corpus contient peu de documentation spécialisée. Enfin, la matrice de corrélation révèle deux clusters distincts : les métriques de Recall (R@1, R@3, R@5) sont fortement corrélées entre elles mais quasi indépendantes des métriques nDCG, illustrant que la couverture brute et la qualité du classement mesurent deux propriétés orthogonales du retrieval.

SAP RAG System — Evaluation Performance Analysis

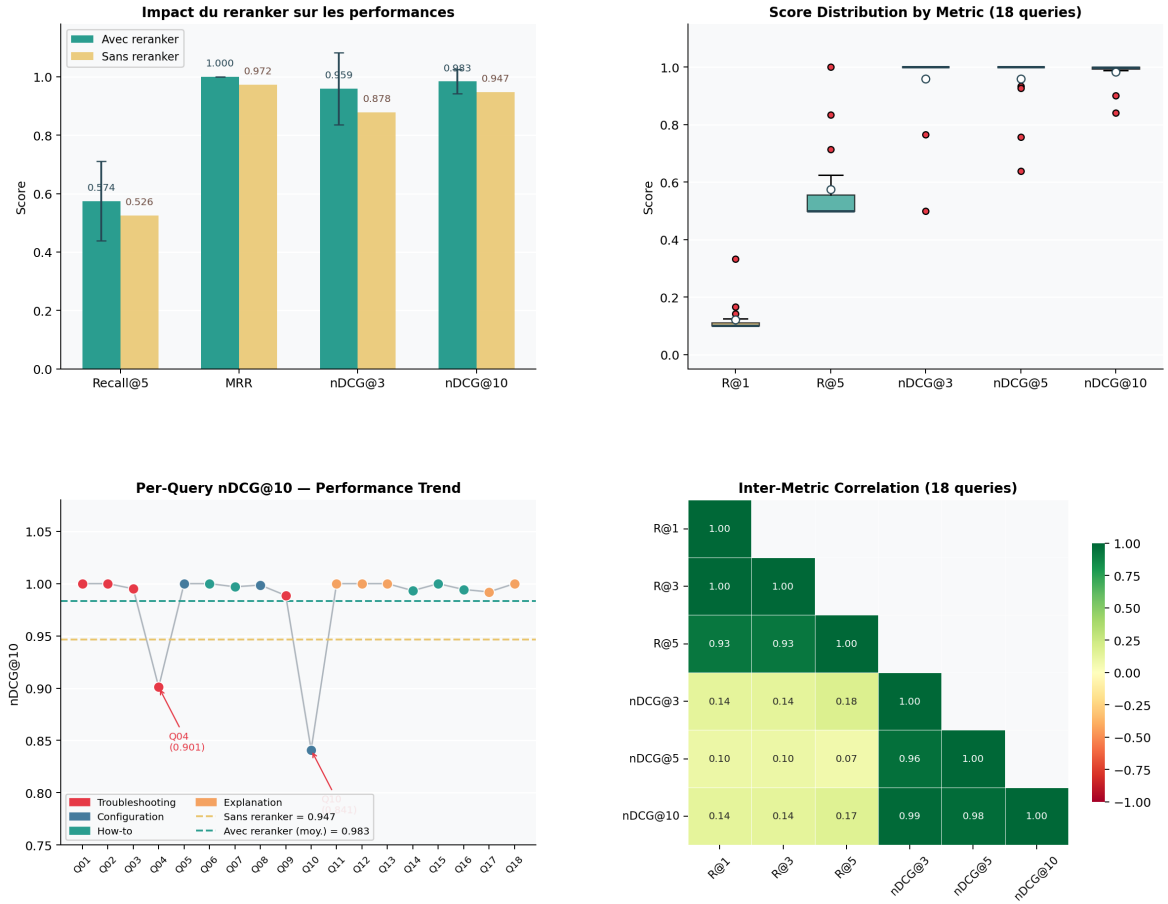


FIGURE 7.1 – Analyse des performances du système RAG SAP : (a) impact du reranker, (b) distributions des scores par métrique, (c) tendance nDCG@10 par requête colorée par type d'intention, (d) matrice de corrélation inter-métriques.

7.6 Justification de la double approche d'évaluation

Les métriques de retrieval et les métriques RAGAS sont complémentaires et non redondantes. Le Recall@k mesure si les bons chunks sont récupérés, indépendamment de ce que le LLM en fait. Le Context Recall RAGAS mesure si les informations nécessaires à la réponse sont présentes dans le contexte fourni au LLM, une notion sémantique plus riche. Le MRR et le nDCG mesurent la qualité du classement : un retrieval bien classé réduit le bruit injecté dans le prompt. La Faithfulness RAGAS mesure si le LLM a bien utilisé le contexte fourni sans inventer d'informations, ce qu'aucune métrique de retrieval ne peut capturer.

Cette double évaluation permet d'isoler la source d'un éventuel problème : si le Recall@k est bon mais la Faithfulness est faible, le problème vient de la génération ; si le Recall@k est faible, le problème vient du retrieval, indépendamment du LLM.

7.7 Détection d'hallucinations

En complément des métriques RAGAS, le pipeline intègre un mécanisme de détection d'hallucinations déterministe dans la fonction `check_hallucinations()`. Celui-ci extrait les T-codes et numéros de notes SAP mentionnés dans la réponse du LLM, et les compare à ceux présents dans les chunks récupérés :

$$\text{T-codes halluc.} = \left(\text{T-codes}(\text{réponse}) \setminus \text{T-codes}(\text{contexte}) \right) \cap \text{KNOWN_TCODES}$$

Toute réponse contenant un T-code connu non présent dans les chunks récupérés est identifiée comme potentiellement hallucinée, et les phrases incriminées sont supprimées par la fonction `sanitize_answer()` avant renvoi à l'utilisateur. Ce mécanisme déterministe est complémentaire à la métrique RAGAS Faithfulness : il opère en moins d'une milliseconde sans appel LLM supplémentaire, avec un comportement prévisible et explicable, tandis que RAGAS évalue la fidélité sémantique globale.

7.8 Synthèse et discussion

7.8.1 Points forts du système

Sur le jeu de test disponible ($n = 18$ pour le retrieval, $n = 3$ pour la génération), plusieurs résultats sont notables. L'approche hybride obtient un $\text{MRR} = 1.0$ et un $\text{nDCG}@3 = 0.959$, indiquant qu'un chunk pertinent apparaît systématiquement en première position et que le classement est de haute qualité dans les cas testés. Le gain de +9,2 % sur le $\text{nDCG}@3$ par rapport à la baseline valide l'apport du reranking cross-encoder. Sur le plan de la génération, l'Answer Relevancy de 0,92 ($n = 3$) suggère que les réponses restent centrées sur les questions posées. Ces résultats sont prometteurs, mais conditionnés par les limites de validité du protocole (taille de l'échantillon, circularité des labels, juge LLM identique au générateur) discutées à la section 7.2.6.

7.8.2 Limites identifiées

Le Recall@1 de 0.122 indique que le premier résultat brut n'est pas toujours le plus pertinent ; ce comportement est attendu sur une base de 76 463 vecteurs et est corrigé par le reranking ($\text{MRR} = 1.0$ sur $n = 18$). L'évaluation porte uniquement sur des requêtes en anglais, le modèle d'embedding BAAI/bge-large-en-v1.5 n'étant pas optimisé pour le français.

Non-calculabilité de Faithfulness et Context Precision. Le modèle-juge `llama3.2:3b` présente deux limites majeures qui rendent les métriques Faithfulness et Context Precision entièrement non calculables dans ce contexte.

- **Défaut de format de sortie.** RAGAS requiert que le juge LLM produise une réponse au format JSON strict (`{"score": 0.x, "statements": [...]}`). Le modèle `llama3.2:3b` — un modèle à 3 milliards de paramètres — échoue systématiquement à respecter ce format : 100 % des appels concernant ces deux métriques se terminent par une `JSONDecodeError`. Ce comportement est documenté dans la littérature pour les modèles de petite taille [5].
- **Biais d’auto-évaluation.** Le modèle `llama3.2:3b` est à la fois le générateur des réponses évaluées et le juge. Cette identité introduit un biais systématique : un modèle tend à juger favorablement des formulations proches des siennes, gonflant artificiellement les scores de fidélité.

Ces deux métriques restent donc volontairement déclarées *N/A* plutôt que de présenter des scores non fiables. Pour une évaluation RAGAS complète, l’utilisation d’un modèle-juge indépendant et de taille suffisante (par exemple `llama3.1:70b` ou un service API externe tel que GPT-4) est indispensable ; cette amélioration est identifiée comme une priorité dans les perspectives du projet.

Enfin, la circularité de l’auto-labellisation par signaux (détaillée au § 7.2.6) constitue la limite de validité la plus structurelle : elle favorise mécaniquement les requêtes contenant des T-codes explicites et produit des scores de retrieval optimistes sur ce type de corpus.

Conclusion

La phase d’évaluation fournit des résultats préliminaires encourageants. Sur un jeu de 18 requêtes ($n = 18$), l’approche hybride (vectoriel + BM25 + RRF + cross-encoder) surpasse la baseline vectorielle pure sur toutes les métriques, avec un gain de +9,2 % sur le $nDCG@3$ et un $MRR = 1.0$. Ces scores doivent cependant être interprétés en tenant compte de la circularité du protocole d’auto-labellisation : les signaux utilisés pour annoter les chunks sont les mêmes termes et T-codes exploités par le retrieval, ce qui produit des scores structurellement favorables. L’évaluation de génération (RAGAS, $n = 3$) donne une Answer Relevancy de 0,92 et un Context Recall de 0,89, mais repose sur un échantillon trop réduit et un juge LLM identique au générateur pour constituer une conclusion définitive. Des travaux futurs devront étendre le jeu de test, introduire des jugements humains indépendants et utiliser un juge LLM plus puissant pour valider ces résultats. La double approche retrieval / génération adoptée reste pertinente pour isoler les sources de défaillance dans le pipeline.

Chapitre 8

Déploiement

Introduction

La phase de déploiement constitue l'aboutissement des travaux de modélisation et leur transition vers une utilisation concrète par les utilisateurs finaux. Après les étapes de conception, d'optimisation et de validation du pipeline RAG, l'objectif est désormais de rendre la solution opérationnelle au sein d'un environnement applicatif réel.

Dans le cadre de ce projet, la mise en production repose sur deux composantes principales : d'une part, le **système RAG** développé en Python et exposé sous forme d'API REST à l'aide de FastAPI, et d'autre part, l'**application web AutoMoni**, une plateforme de monitoring SAP développée avec SAP CAP et React, au sein de laquelle le chatbot intelligent a été intégré.

Ce chapitre présente l'architecture générale de déploiement, les différentes étapes de mise en service des composants, le mécanisme d'intégration du chatbot dans l'interface applicative, ainsi qu'une présentation de l'interface utilisateur finale. Il se conclut par une évaluation des performances du système à travers un benchmark comparatif.

8.1 Architecture globale de déploiement

8.1.1 Vue d'ensemble

Le système déployé repose sur une architecture à trois couches distinctes communiquant via des interfaces bien définies. La figure 8.2 illustre l'ensemble des composants et leurs interconnexions.

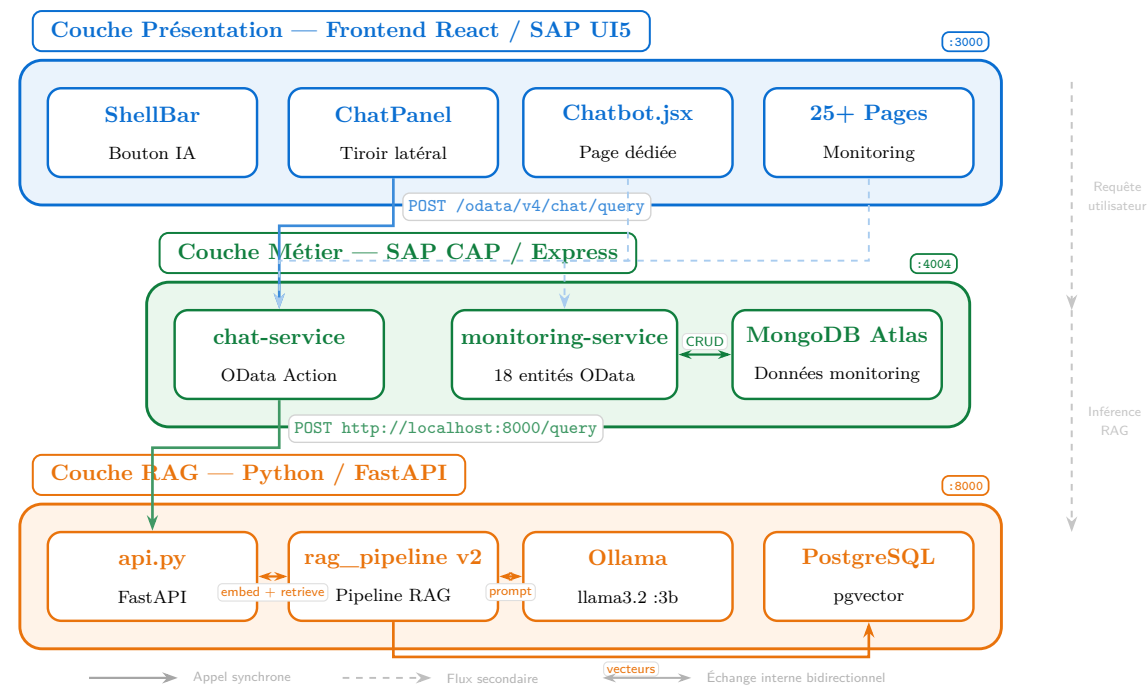


FIGURE 8.1 – Architecture globale de déploiement du système RAG intégré dans AutoMoni

8.1.2 Technologies utilisées

L'ensemble de la pile technologique mise en œuvre est résumé dans le tableau 8.1.

TABLE 8.1 – Pile technologique du système déployé

Couche	Technologie	Rôle
Frontend	React 19 + SAP UI5 Web Components	Interface utilisateur, affichage du chatbot
Backend	SAP CAP v8 + Express.js	Services OData, proxy vers l'API RAG (port 4004)
Base de données	MongoDB Atlas	Stockage des données de monitoring SAP (18 collections)
API RAG	FastAPI + Uvicorn	Exposition du pipeline RAG en REST (port 8000)
LLM	Ollama (llama3.2 :3b / qwen2.5 :7b)	Génération locale des réponses (port 11434)
Embeddings	BAAI/bge-large-en-v1.5	Vectorisation des requêtes et des documents
Base vectorielle	PostgreSQL + pgvector	Stockage et recherche hybride (port 5432)

8.2 Déploiement du système RAG

8.2.1 Configuration de l'environnement

Le système RAG est configuré à travers un fichier `.env` qui centralise les paramètres d'accès aux services externes. Les trois variables essentielles sont la chaîne de connexion à la base de données PostgreSQL, et les noms des modèles LLM disponibles via Ollama

8.2.2 Infrastructure de la base de données vectorielle

```
1 CREATE INDEX sap_chunks_hnsw ON sap_chunks
2   USING hnsw (embedding vector_cosine_ops)
3   WITH (m = 16, ef_construction = 64);
```

Listing 8.1 – Création de l'index vectoriel HNSW sur la table sap_chunks

8.2.3 Schéma de la base de données PostgreSQL

La base de données PostgreSQL du système héberge **21 tables** organisées en quatre groupes fonctionnels, chacun correspondant à un rôle distinct dans l'architecture RAG déployée :

- **Tables fondamentales RAG/CAG** : `sap_chunks` (76 463 vecteurs de dimension 1 024, index HNSW cosinus) et `qa_cache` (cache sémantique des réponses générées avec leurs embeddings de requête).
- **Tables d'interaction utilisateur** : `conversations` (historique de dialogue multi-tours avec détection d'intention et réécriture de requête) et `admin_feedback` (notations et corrections humaines avec vecteur de la question originale pour invalidation du cache).
- **Tables de monitoring SAP** : 12 tables alimentées en continu via PyRFC, correspondant aux T-codes de supervision : SM37, SM51, SM66, SM12, SM13, SM58, SMQ1, SOST, SP01, ST03N, ST13, ST22.
- **Tables de métriques système** : 5 tables de métriques OS et base de données (`cpu`, `memory`, `fsys`, `db02`, `mss_db`).

8.3 Diagramme de cas d'utilisation général

Dans cette section, les exigences fonctionnelles du système déployé sont représentées à l'aide d'un diagramme de cas d'utilisation conforme au langage de modélisation UML. Ce diagramme (voir figure 8.2) met en évidence les différents acteurs du système ainsi que les interactions possibles entre ces acteurs et les fonctionnalités offertes par l'application. Il permet ainsi de fournir une vision globale du comportement du système et des services qu'il propose aux utilisateurs.

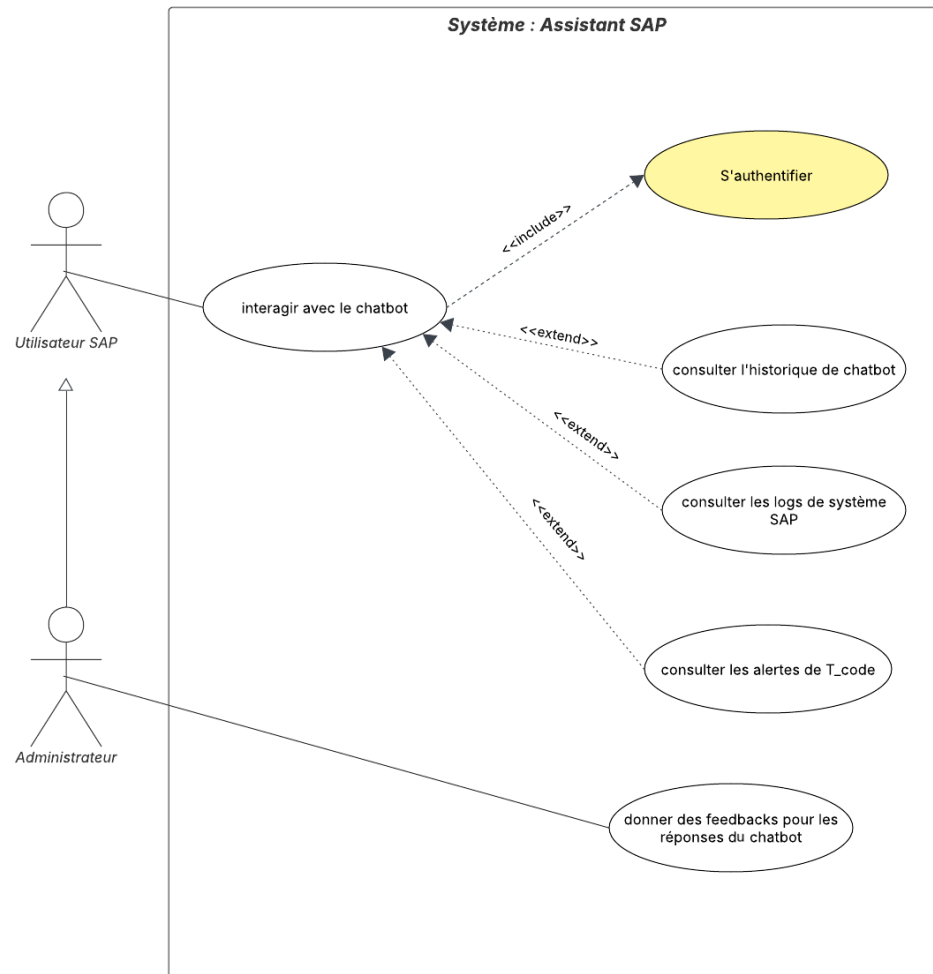


FIGURE 8.2 – Diagramme de cas d'utilisation général du système déployé

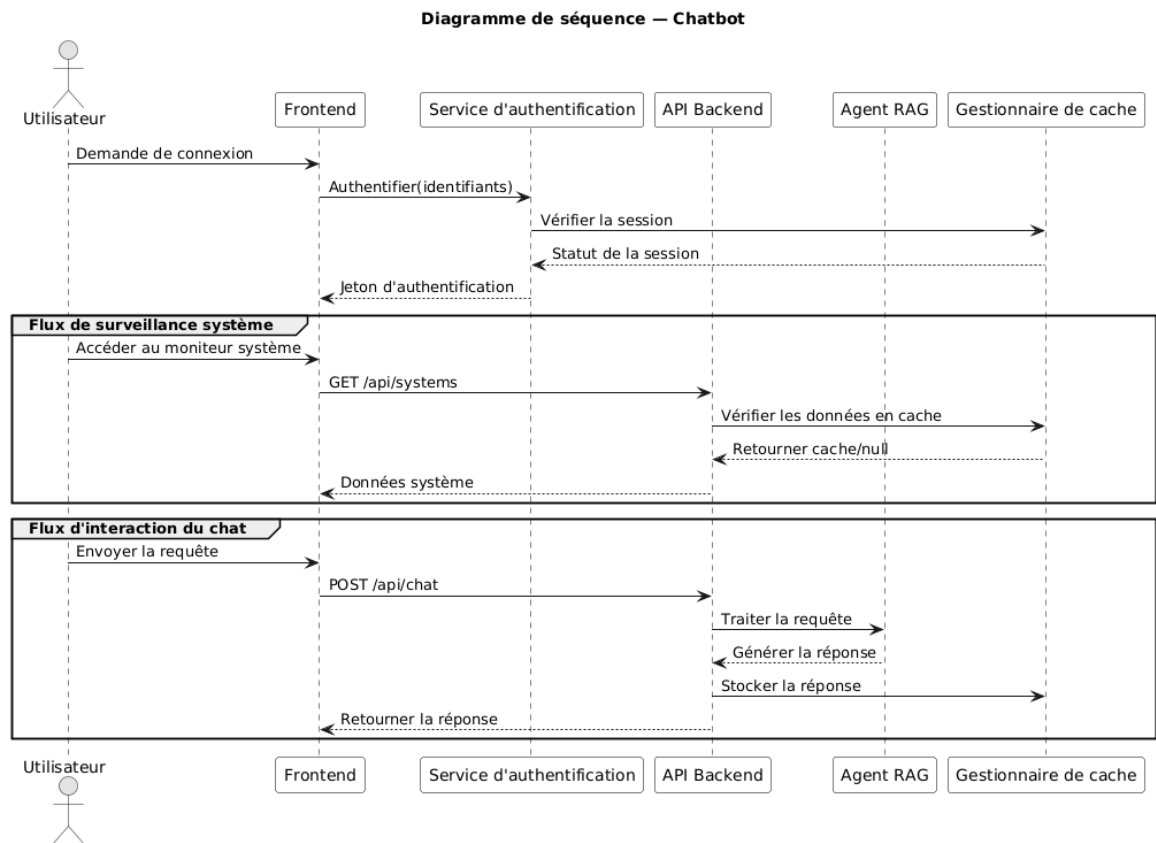


FIGURE 8.3 – Diagramme de séquence : cycles complets d’interaction : initialisation de session, surveillance SAP et chatbot RAG

La figure 8.4 présente le diagramme de classes détaillé des quatre tables fondamentales avec l’ensemble de leurs attributs et les associations logiques entre elles. La figure 8.4 présente les 17 tables de monitoring et de métriques dans un format compact.

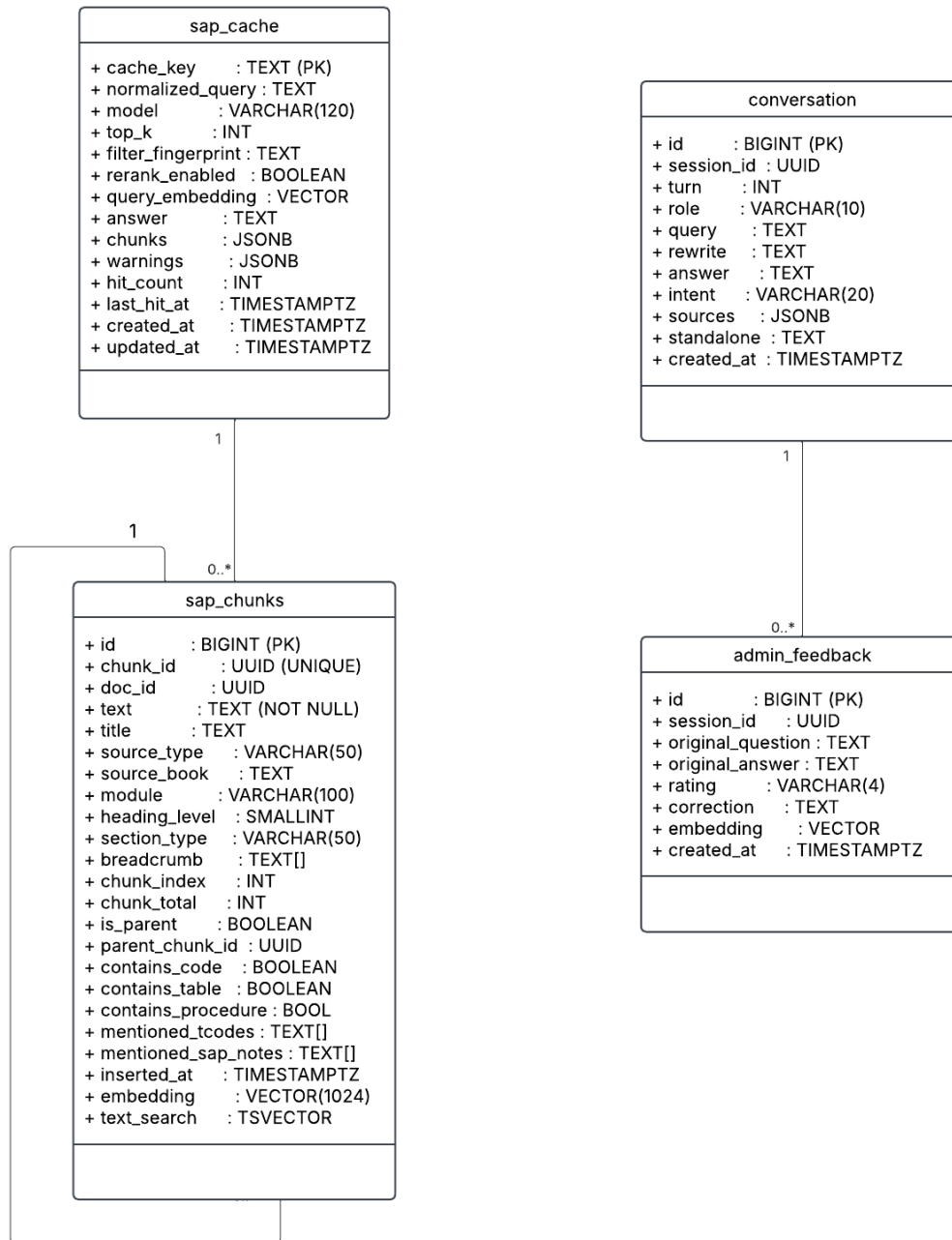


FIGURE 8.4 – Diagramme de classes : Tables fondamentales du système RAG (PostgreSQL)

8.3.1 Démarrage de l'API FastAPI

Le serveur FastAPI est lancé via **Uvicorn**, le serveur ASGI de référence pour les applications Python asynchrones :

```

1 cd C:\Users\Youss\Documents\RAG_SAP\rag
2 uvicorn api:app --host 0.0.0.0 --port 8000 --reload

```

Listing 8.2 – Commande de démarrage de l'API RAG

8.3.2 Endpoints REST exposés

L'API FastAPI expose trois endpoints documentés automatiquement via Swagger UI (<http://localhost:8000/docs>) :

TABLE 8.2 – Endpoints de l'API RAG (api.py)

Méthode	Endpoint	Description
POST	/query	Reçoit une question et un session_id, exécute le pipeline RAG, renvoie la réponse avec sources et avertissements
DELETE	/session/{session_id}	Supprime l'historique de conversation associé à une session
GET	/health	Vérifie la disponibilité du service et retourne le modèle actif

Le corps de la requête POST /query accepte les paramètres suivants :

```

1 class QueryRequest(BaseModel):
2     question: str
3     session_id: str = ""
4     top_k: int = TOP_K # default : 5
5     model: str = OLLAMA_MODEL # default : llama3.2:3
6     filter_tcode: Optional[str] = None
7     filter_module: Optional[str] = None
8     filter_source: Optional[str] = None

```

Listing 8.3 – Schéma de la requête POST /query (api.py)

8.3.3 Paramètres clés du pipeline

Le tableau 8.3 regroupe les hyperparamètres déterminants du pipeline RAG déployé en production :

TABLE 8.3 – Paramètres de configuration du pipeline RAG (rag_pipeline.py)

Paramètre	Valeur	Rôle
FETCH_K	15	Nombre de candidats récupérés par recherche vectorielle
RERANK_K	10	Nombre de candidats transmis au cross-encoder après RRF
TOP_K	5	Nombre final de chunks injectés dans le prompt LLM
CAG_MAX_CHUNKS	3	Chunks pré-chargés max pour les requêtes T-code (CAG)
MIN_SCORE	0.05 / 0.30	Seuil minimal de similarité cosinus (adaptatif selon l'intention)
MIN_RERANK_SCORE	2.0	Seuil de rejet par le cross-encoder (échelle -10 à +10)
MAX_CHUNKS_PER_SOURCE	3	Déduplication : maximum de chunks par source
W_{vec} / W_{BM25}	0.4 / 0.6	Poids de fusion RRF (BM25 privilégié pour corpus SAP)
NUM_CTX	4 096	Fenêtre de contexte du LLM Ollama (en tokens)
Temperature	0.1	Température de génération (réponses déterministes)

8.4 Intégration du chatbot dans l'application web

8.4.1 Vue d'ensemble de l'intégration

L'intégration du chatbot dans AutoMoni a été réalisée selon une architecture en trois niveaux : la **définition du service** en CDS (Core Data Services), l'**implémentation du handler** en JavaScript côté CAP, et les **composants React** côté frontend.

8.4.2 Définition du service OData (CAP CDS)

Le service de chatbot est défini dans le fichier `chat-service.cds` à l'aide du langage CDS de SAP. Il expose une action OData qui accepte la question de l'utilisateur et un identifiant de session, et retourne la réponse structurée du système RAG :

```

1 service ChatService {
2   action query(question: String, session_id: String) returns
3     {
4       answer:      String;
5       sources:     array of String;
6       warnings:    array of String;
7       session_id: String;
8   };
9 }
```

Listing 8.4 – Définition du service chatbot en CDS (`chat-service.cds`)

Cette définition génère automatiquement l'endpoint OData `POST /odata/v4/chat/query`, documenté et typé, directement accessible depuis le frontend React.

8.4.3 Composants React du chatbot

Le chatbot est implémenté côté frontend sous deux formes complémentaires :

- **Page dédiée** (`/chatbot`) : le composant `Chatbot.jsx` (515 lignes) offre une interface complète avec historique de conversations organisées en sessions, liste des conversations précédentes, zone de réponse avec rendu Markdown via `react-markdown`, et boutons d'action (nouvelle conversation, effacement).
- **ChatPanel — Tiroir latéral** : exporté depuis `Chatbot.jsx`, ce panneau coulissant est intégré dans le composant `AppShell.jsx` et rendu accessible depuis toutes les pages de l'application via le bouton IA dans la `ShellBar`. Il est monté paresseusement (*lazy mount*) à la première ouverture afin d'éviter un flash CSS indésirable :

```

1 const [isChatOpen, setIsChatOpen] = useState(false)
2 const [chatHasOpened, setChatHasOpened] = useState(false)
3
4 const onChatbotClick = () => {
5   if (!chatHasOpened) setChatHasOpened(true)
6   setIsChatOpen(prev => !prev)
7 }
8
9 {chatHasOpened && (
```

```

10   <ChatPanel open={isChatOpen} onClose={() =>
      setIsChatOpen(false)} />
11 })}

```

Listing 8.5 – Montage paresseux du ChatPanel dans AppShell.jsx

8.4.4 Chatbot et évaluation intégrée en temps réel

L'intégration du pipeline d'évaluation directement dans la boucle de traitement du chatbot constitue un apport majeur par rapport à une évaluation *offline* classique. Plutôt que d'utiliser un juge LLM coûteux à chaque appel (comme G-Eval), le système repose sur deux mécanismes complémentaires déclenchés automatiquement à chaque réponse :

1. **Détection déterministe des hallucinations** : la fonction `check_hallucinations()` analyse la réponse générée à la recherche de T-codes SAP et de numéros de notes SAP qui ne figurent pas dans les chunks récupérés. Toute mention identifiée comme non ancrée dans les sources est supprimée par `sanitize_answer()` avant renvoi à l'utilisateur. Ce mécanisme s'exécute en moins d'une milliseconde sans appel LLM supplémentaire.
2. **Attribution des sources** : les chunks utilisés pour construire la réponse sont retournés dans le champ `sources` de la réponse JSON, sous forme de références structurées (« *titre du livre > section* »).

Ces deux informations sont remontées dans l'interface utilisateur via le contrat d'API défini dans `chat-service.cds` :

```

1  returns {
2    answer:    String;           -- reponse nettooyee
3    sources:   array of String; -- references des chunks
                        utilisees
4    warnings:  array of String; -- hallucinations detectees et
                        supprimees
5    session_id: String;
6  }

```

Listing 8.6 – Champs d'évaluation retournés par le service RAG

Le composant React `BotMessageContent` interprète ces champs et affiche, sous chaque réponse du bot, un indicateur de qualité et la liste des sources consultées :

- **Badge vert « ✓ Réponse vérifiée »** : aucun avertissement détecté, la réponse est intégralement ancrée dans les sources récupérées.

- **Badge orange** « $\triangle$ **N** avertissement(s) » : au moins une affirmation a été filtrée pour risque d'hallucination, l'utilisateur est averti.
- **Section « Sources » dépliable** : liste des références documentaires utilisées, permettant à l'utilisateur de vérifier l'origine de chaque information.

```

1  const BotMessageContent = ({ message }) => (
2    <>
3      <Markdown>{message.text}</Markdown>
4      {(message.sources?.length > 0 || message.warnings?.length
5        > 0) && (
6        <div className="ag-eval-footer">
7          <span className={`ag-quality-badge ${
8            message.warnings?.length > 0
9              ? 'ag-quality-badge--warn'
10             : 'ag-quality-badge--ok'
11           }`} >
12            {message.warnings?.length > 0
13              ? `Warning: ${message.warnings.length}
14                hallucination(s) filtered`
15              : `Response verified`}
16          </span>
17          <details className="ag-sources">
18            <summary>{message.sources.length} source(s)</
19              summary>
20            <ul>{message.sources.map((src, i) => <li key={i}>{
21              src}</li>)}</ul>
22          </details>
23        </div>
24      )}
25    </>
26  )

```

Listing 8.7 – Rendu du badge qualité et des sources dans BotMessageContent (Chatbot.jsx)

Cette approche est délibérément **déterministe et légère** : contrairement à RAGAS ou G-Eval qui nécessitent un second appel LLM pour scorer la réponse, la détection par règles (T-codes, numéros de notes) est instantanée et 100 % reproductible. Elle constitue un mécanisme de premier niveau suffisant pour alerter l'utilisateur sur les cas les plus risqués, RAGAS étant réservé à l'évaluation *offline* en batch du pipeline.

8.4.5 Intelligence de navigation — T-codes vers pages monitoring

Une fonctionnalité avancée du chatbot est sa capacité à détecter dans le message de l'utilisateur une **intention de navigation** vers une page de monitoring, et à y rediriger automatiquement. Le composant `Chatbot.jsx` embarque un dictionnaire de 18 T-codes SAP mappés à leurs routes React :

```
1  const TCODE_ROUTES = {
2    st22: { path: '/st22', label: 'Runtime Errors (ST22)'
3          },
4    sm37: { path: '/sm37', label: 'Background Jobs (SM37)'
5          },
6    sm13: { path: '/sm13', label: 'Update Records (SM13)'
7          },
8    sm66: { path: '/sm66', label: 'Work Processes (SM66)'
9          },
10   db02: { path: '/db02', label: 'Database Space (DB02)'
11         },
12   sm58: { path: '/sm58', label: 'RFC Monitoring (SM58)'
13         },
14   smq1: { path: '/smq1', label: 'Outbound Queue (SMQ1)'
15         },
16   // ... 11 autres T-codes
17 }
```

Listing 8.8 – Mapping T-codes vers routes de navigation (Chatbot.jsx)

Lorsque l'utilisateur formule un message contenant à la fois un T-code reconnu et un mot-clé d'intention de navigation (`open`, `go to`, `ouvre`, `affiche`, etc.), le chatbot effectue une redirection directe vers la page correspondante, sans interroger l'API RAG. Cette détection supporte également l'extraction du **SID** (System ID, ex. PRD, P01) et d'un **filtre de date** depuis le message naturel.

8.5 Interface utilisateur — Présentation

8.5.1 Page chatbot dédiée

La page chatbot, accessible via la route `/chatbot`, propose une interface structurée en deux panneaux : à gauche, la liste des conversations historiques identifiées par un titre auto-généré, et à droite, la conversation active avec l'assistant. Parmi ses fonctionnalités principales :

- **Gestion des sessions** : chaque conversation est identifiée par un UUID de session unique, qui est transmis à l'API RAG afin de maintenir le contexte conversationnel côté serveur.
- **Quick Prompts** : trois suggestions de questions fréquentes (*Show top runtime errors today*, *Any failed background jobs right now ?*, *Summarize DB space alerts*) sont proposées à l'utilisateur pour faciliter la prise en main.
- **Indicateur de traitement** : un *BusyIndicator* SAP UI5 s'affiche pendant le temps de réponse du système.
- **Avertissements d'hallucinations** : si l'API RAG détecte un T-code ou une note SAP potentiellement halluciné(e), un avertissement est affiché sous la réponse.

8.5.2 ChatPanel : Tiroir latéral universel

Le composant `ChatPanel` est directement intégré au sein du shell applicatif `AppShell.jsx`. Il peut être ouvert depuis n'importe quelle page de l'application grâce au bouton IA (icône d'intelligence artificielle) positionné dans la `ShellBar`, en haut à droite de l'interface. Lors de son activation, le panneau apparaît latéralement depuis la droite via une animation de type *slide-in*, offrant à l'utilisateur la possibilité d'interagir avec l'assistant conversationnel tout en conservant le contexte de la page de monitoring actuellement affichée.

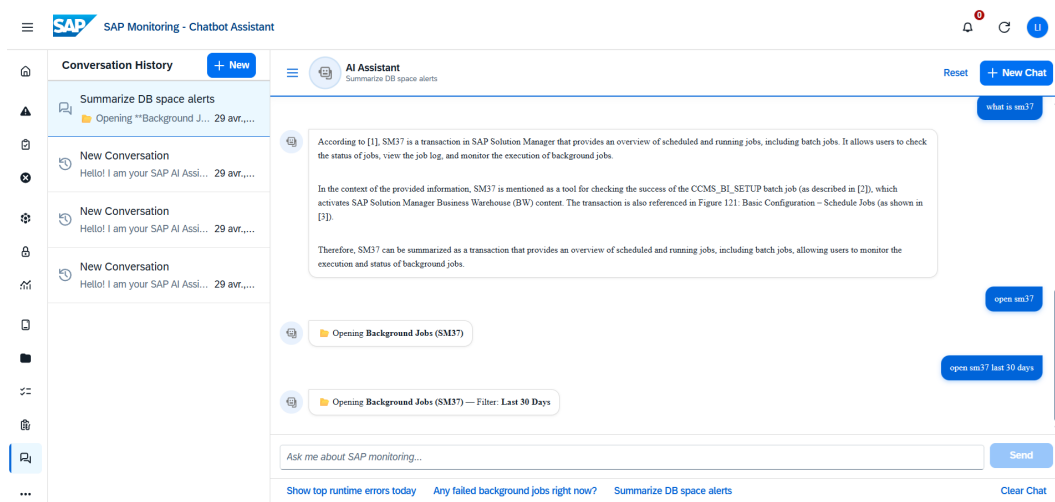


FIGURE 8.5 – Interface de la page chatbot dédiée avec historique de conversations

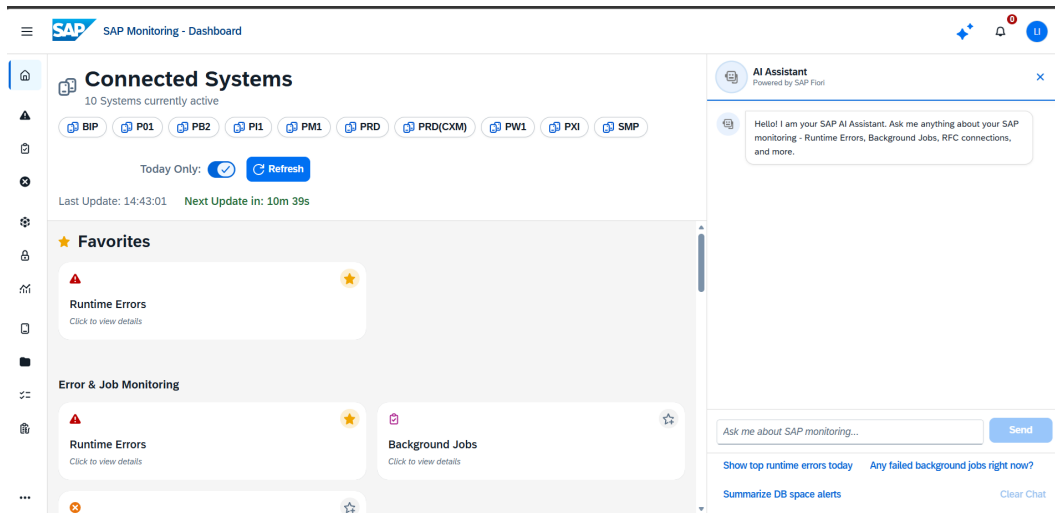


FIGURE 8.6 – ChatPanel intégré en tiroir latéral sur une page de monitoring SAP

8.6 Sélection du modèle LLM et benchmarking

8.6.1 Protocole d'évaluation

Afin de comparer objectivement les deux configurations LLM disponibles, un benchmark automatisé a été conduit le 12 mai 2026. Il porte sur un ensemble de **10 questions représentatives** des cas d'usage réels du système, couvrant des T-codes BASIS courants :

1. How to monitor background jobs in SM37 ?
2. How to analyze a short dump in ST22 ?
3. How to configure transport routes in STMS ?
4. How to manage user roles with SU01 and PFCG ?
5. How to create an ABAP program using SE38 ?
6. What is a BAPI and how to test it with SE37 ?
7. How to reprocess IDoc errors using BD87 ?
8. How to analyze authorization errors with SU53 ?
9. How to read system logs in SM21 ?
10. How to analyze work processes with SM50 ?

Les deux modèles évalués sont **LLaMA 3.2 :3b** (modèle rapide, 3 milliards de paramètres) et **Qwen 2.5 :7b** (modèle de qualité supérieure, 7 milliards de paramètres). Les métriques calculées sont le **Recall@K** (proportion de questions ayant au moins un chunk pertinent dans les K premiers résultats), le **MRR** (*Mean Reciprocal Rank*, position moyenne du premier chunk pertinent) et le **nDCG** (*Normalized Discounted Cumulative Gain*, qualité globale du classement). Les valeurs de K testées sont $K \in \{1, 3, 5, 10, 12\}$.

8.6.2 Résultats

La figure 8.7 présente les résultats visuels du benchmark comparatif entre les deux configurations LLM sur l'ensemble des métriques de retrieval.



FIGURE 8.7 – Résultats du benchmark RAG — LLaMA 3.2 :3b vs Qwen 2.5 :7b (12 mai 2026, 76 463 vecteurs)

8.6.3 Analyse des résultats

Les résultats du benchmark mettent en évidence plusieurs constats :

- **Recall@K croissant** : le rappel augmente logiquement avec K pour les deux modèles, atteignant des valeurs élevées à K=10 et K=12, confirmant que la base documentaire contient les informations nécessaires pour répondre aux questions posées.
- **Qualité de classement** : le nDCG reflète la capacité du pipeline à placer les chunks les plus pertinents en tête de liste. La fusion RRF avec pondération asymétrique (BM25 : 0.6, vectoriel : 0.4) favorise la précision lexicale, essentielle pour les T-codes SAP.
- **Impact des paramètres** : Qwen 2.5 :7b utilise un seuil de reranking (`MIN_RERANK_SCORE` = 2.0) et un `FETCH_K` = 40 plus élevés que LLaMA 3.2 :3b (`MIN_RERANK_SCORE` = 1.0, `FETCH_K` = 30), ce qui explique une sélectivité différente dans les résultats.
- **Compromis vitesse/qualité** : LLaMA 3.2 :3b offre des temps de réponse significativement plus courts, tandis que Qwen 2.5 :7b produit des réponses de meilleure qualité rédactionnelle. Le choix du modèle peut être configuré dynamiquement via la variable `model` de la requête API.

8.7 Limites et perspectives

8.7.1 Limites du déploiement actuel

Malgré les performances satisfaisantes observées lors du benchmark, plusieurs limitations du déploiement actuel méritent d'être soulignées :

- **CORS ouvert en développement** : La politique CORS de l'API FastAPI est actuellement configurée en mode permissif (`allow_origins=["*"]`), adapté à un environnement de développement. Une configuration restrictive sera nécessaire pour un déploiement en production.
- **Cache sémantique en mémoire** : Le cache sémantique du pipeline RAG est stocké uniquement en mémoire vive du processus. Son contenu est perdu à chaque redémarrage et ne peut pas être partagé entre plusieurs instances, ce qui limite son efficacité dans une architecture distribuée.
- **LLM local limité** : Le modèle `llama3.2:3b` déployé localement via Ollama, bien qu'adapté à un contexte *on-premise*, reste moins performant que les grands modèles hébergés dans le cloud (GPT-4o, Claude Sonnet) sur des questions complexes nécessitant un raisonnement approfondi.
- **Gestion des sessions en mémoire** : Les sessions de conversation de l'API FastAPI sont stockées dans un dictionnaire Python en mémoire. En cas de redémarrage du service, toutes les sessions actives sont perdues. La persistance des sessions dans la base de données PostgreSQL est déjà implémentée mais dépend d'un mécanisme asynchrone de sauvegarde.
- **Détection d'hallucinations par heuristiques** : La validation anti-hallucination repose sur des expressions régulières pour les T-codes et les numéros de notes SAP. Les affirmations factuellement erronées ne contenant pas d'identifiants SAP explicites ne sont pas détectées.

8.7.2 Perspectives d'amélioration

Plusieurs axes d'amélioration peuvent être envisagés pour renforcer le système à court et moyen terme :

- **Déploiement Cloud Foundry** : Les fichiers `mta.yaml` et `manifest.yml` sont déjà configurés pour un déploiement sur SAP BTP (*Business Technology Platform*) via Cloud Foundry. Cette étape permettrait de bénéficier d'une haute disponibilité, d'un équilibrage de charge et d'une authentification XSUAA intégrée.
- **Cache sémantique distribué** : L'intégration d'un cache Redis permettrait de partager le cache sémantique entre plusieurs instances de l'API RAG et de le persister entre les redémarrages.

- **Modèle LLM plus performant** : L'intégration d'un modèle de grande taille (qwen2.5 :7b déjà supporté, ou un modèle cloud via API) améliorerait la qualité des réponses sur les requêtes complexes.
- **Mise à jour incrémentale de la base de connaissances** : Un pipeline de synchronisation incrémentale (déjà prototypé dans `conv_databaseMONGODB/`) permettrait d'enrichir automatiquement la base documentaire à mesure que de nouvelles notes SAP ou documentations sont publiées.
- **Collecte de feedback utilisateur** : L'ajout d'un mécanisme de notation des réponses (pouce haut/pouce bas) permettrait de constituer un jeu de données d'évaluation continue et d'identifier les points faibles du système en production réelle.

Conclusion

Ce chapitre a présenté le déploiement complet du système RAG SAP dans le cadre de la méthodologie CRISP-DM. L'architecture développée repose sur trois couches complémentaires : un backend Python/FastAPI pour le moteur RAG, un service SAP CAP assurant l'exposition OData, et une interface React SAP UI5 intégrant le chatbot. Les résultats du benchmark ont confirmé l'efficacité du pipeline hybride de retrieval sur un corpus SAP volumineux de 76 463 vecteurs. En environnement de production, la solution permet aux administrateurs SAP BASIS d'obtenir rapidement des réponses techniques fiables directement depuis l'interface de monitoring. Enfin, les perspectives d'évolution, notamment l'intégration sur SAP BTP et l'utilisation de modèles LLM plus performants, ouvrent la voie à une industrialisation avancée de la plateforme.

Conclusion générale et perspectives

Bilan du projet

À l'issue de ce projet, nous avons réussi à créer un assistant conversationnel intelligent, destiné à satisfaire les exigences quotidiennes des administrateurs SAP BASIS et intégré directement sur la plateforme AutoMoni. La méthodologie CRISP-DM, adoptée dès le début, nous a permis de suivre un cheminement cohérent et itératif, de l'analyse des besoins jusqu'à la mise en œuvre effective de la solution.

Dans un premier temps, l'examen du contexte métier a montré que les administrateurs SAP consacrent beaucoup de temps à la recherche d'informations techniques dispersées dans une documentation abondante. Trois attentes principales ont émergé de cette analyse : disposer d'une réponse rapide et précise lors des incidents, bénéficier d'une assistance contextuelle sans interrompre le flux de travail, et pouvoir naviguer intuitivement entre les différentes vues de l'application. Ces exigences ont directement guidé nos décisions de conception.

La construction de la base documentaire a représenté un travail conséquent : **76 463 fragments** extraits de la documentation SAP officielle ont été vectorisés à l'aide du modèle `BAAI/bge-large-en-v1.5` (vecteurs de dimension 1024), puis stockés dans PostgreSQL via l'extension `pgvector`. L'indexation HNSW adoptée garantit des temps de recherche inférieurs à 100 ms même sur un corpus de cette taille.

Le cœur du projet réside dans le pipeline RAG élaboré spécifiquement pour la terminologie SAP. Au lieu de se baser sur une recherche uniquement sémantique, nous avons choisi d'adopter une approche hybride combinant BM25 et la recherche vectorielle, ajustée grâce à l'algorithme RRF. Un cross-encoder effectue ensuite un reranking précis des résultats, tandis qu'un module de validation déterministe vérifie la cohérence des T-codes et des numéros de notes SAP présents dans chaque réponse avant de les transmettre à l'utilisateur. Ce pipeline est disponible sous forme d'API REST grâce à FastAPI.

Côté application, l'intégration dans AutoMoni s'est appuyée sur l'architecture SAP CAP pour exposer le service de chatbot en OData, et sur React pour les composants d'interface. Qu'il soit utilisé depuis le tiroir latéral *ChatPanel* ou depuis la page dédiée,

l'assistant reste joignable à tout moment sans perturber la consultation des données de monitoring. Les tests comparatifs menés entre LLaMA 3.2 :3b et Qwen 2.5 :7b ont mis en lumière les différences de comportement entre les deux modèles et ont permis d'ajuster les paramètres du pipeline en conséquence.

Apports du projet

Ce travail apporte plusieurs contributions concrètes, tant sur le plan technique que sur le plan fonctionnel.

- Un pipeline RAG hybride, pensé pour les particularités du vocabulaire SAP, qui combine efficacement recherche lexicale et recherche sémantique pour retrouver les passages documentaires les plus pertinents.
- Une intégration transparente du chatbot dans une application métier en production, réalisée sans modification de l'expérience utilisateur existante, grâce au protocole OData et à l'architecture SAP CAP.
- Un mécanisme de contrôle qualité léger, embarqué directement dans la boucle de traitement, qui détecte et filtre les affirmations non ancrées dans les sources sans nécessiter d'appel LLM supplémentaire.
- Une fonctionnalité de navigation intelligente : lorsque l'utilisateur mentionne un T-code dans son message, le chatbot peut le rediriger automatiquement vers la page de monitoring correspondante, fusionnant ainsi assistance documentaire et navigation applicative.

Perspectives

Les résultats enregistrés durant la phase de déploiement ouvrent la voie à plusieurs prolongements envisageables à différents horizons temporels.

Industrialisation de la solution. Le passage vers une infrastructure cloud via SAP BTP et Cloud Foundry constitue la suite logique du projet. Les fichiers de déploiement (`mta.yaml`, `manifest.yaml`) ont été préparés dans cette optique. Ce changement apporterait la haute disponibilité, l'authentification XSUAA et la montée en charge automatique, trois éléments indispensables pour un usage en production à grande échelle.

Amélioration des performances. Le cache sémantique actuel, limité à la mémoire du processus, gagnerait à être externalisé dans une instance Redis partagée entre les différentes instances de l'API. Par ailleurs, l'utilisation d'un modèle LLM hébergé dans le cloud (tel que Qwen 2.5 :7b ou un modèle GPT) permettrait d'élever le niveau de qualité rédactionnelle des réponses sur les requêtes les plus exigeantes.

Maintien de la base documentaire. La documentation SAP évolue régulièrement avec la publication de nouvelles notes et mises à jour. Un processus de synchronisation incrémentale, déjà prototypé durant le projet, pourrait être mis en place pour enrichir continuellement le corpus sans repartir d'un indexage complet.

Retour utilisateur et amélioration continue. Intégrer un système de notation directement dans l'interface du chatbot permettrait de recueillir des signaux qualitatifs en situation réelle. Ces retours constitueraient un jeu de données précieux pour affiner les paramètres du pipeline et détecter les cas où le système produit des réponses insatisfaisantes.

Extension du périmètre fonctionnel. L'architecture développée n'est pas spécifique à SAP BASIS ; elle pourrait être adaptée à d'autres modules SAP comme FI/CO, MM ou SD, voire à des référentiels documentaires d'autres domaines métiers. Cette ouverture témoigne de la capacité de généralisation de l'approche retenue.

En résumé, ce projet illustre qu'il est envisageable d'établir un système d'assistance intelligent, performant et en conformité avec les exigences de confidentialité des entreprises, en utilisant des modèles open-source déployés localement et une architecture modulaire bien conçue. Les opportunités d'amélioration relevées représentent simplement une suite logique, et chacune d'elles pourrait donner lieu à un développement ultérieur complet.

Bibliographie

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.
- [2] Brian J. Chan, Chao-Ting Chen, Jui-Hung Cheng, and Hen-Hsen Huang. Don't do RAG : When cache-augmented generation is better for knowledge tasks. *arXiv preprint arXiv :2412.15605*, 2024.
- [3] Data Science Process Alliance. CRISP-DM : Still the most popular process for data science, data mining, or analytics, 2021. URL <https://www.datascience-pm.com/crisp-dm-still-most-popular/>. Consulté le 5 juin 2026.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT : Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 2019.
- [5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RA-GAS : Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv :2309.15217*, 2023.
- [6] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Stanford University, 3rd (online draft) edition, 2024. URL <https://web.stanford.edu/~jurafsky/slp3/>.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.

- [8] Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. *Proceedings of EMNLP-IJCNLP*, 2019.
- [9] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR : A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2021.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.